Faculty of Computer Science and Information Technology

LPG IOT MEASUREMENT DEVICE WITH DATA ANALYTICS

Muhammad Azri Hafiz bin Roslan

Bachelor of Computer Science with Honours

(Software Engineering)

2020

LPG IOT MEASUREMENT DEVICE WITH DATA ANALYTICS

MUHAMMAD AZRI HAFIZ BIN ROSLAN

This project is submitted in partial fulfilment of the

requirement for the degree of

Bachelor of Computer Science with Honours

(Software Engineering)

Faculty of Computer Science and Information Technology

UNIVERSITI MALAYSIA SARAWAK

2020

PERANTI PENGUKUR IOT LPG DENGAN PENGANALISIS DATA

MUHAMMAD AZRI HAFIZ BIN ROSLAN

Projek ini merupakan salah satu keperluan untuk

Ijazah Sarjana Muda Sains Komputer dengan Kepujian

(Kejuruteraan Perisian)

Fakulti Sains Komputer dan Teknologi Maklumat

UNIVERSITI MALAYSIA SARAWAK

2020

UNIVERSITI MALAYSIA SARAWAK

THESIS STATUS ENDORSEMENT FORM

TITLE LPG IOT MEASUREMENT DEVICE WITH DATA ANALYTICS

ACADEMIC SESSION: 2019/2020

MUHAMMAD AZRI HAFIZ BIN ROSLAN

(CAPITAL LETTERS)

hereby agree that this Thesis* shall be kept at the Centre for Academic Information Services, Universiti Malaysia Sarawak, subject to the following terms and conditions:

1. The Thesis is solely owned by Universiti Malaysia Sarawak
2. The Centre for Academic Information Services is given full rights to produce copies for educational purposes only
3. The Centre for Academic Information Services is given full rights to do digitization in order to develop local content database
4. The Centre for Academic Information Services is given full rights to produce copies of this Thesis as part of its exchange item program between Higher Learning Institutions [or for the purpose of interlibrary loan between HLI]
5. ** Please tick (√)

[/]	CONFIDENTIAL	(Contains classified information bounded by the OFFICIAL SECRETS ACT 1972)
[ ]	RESTRICTED	(Contains restricted information as dictated by the body or organization where the research was conducted)
[ ]	UNRESTRICTED	

Validated by SELEVIAWATI TARMIZI
Senior Lecturer
Faculty of Computer Science and Information Technology
Universiti Malaysia Sarawak

On Behalf of Mr Ahmad Hadinata (CTG)

(AUTHOR'S SIGNATURE)

(SUPERVISOR'S SIGNATURE)

Permanent Address

NO. 370, LORONG ASTANA 2, JLN ASTANA, PETRA JAYA, 93050, KUCHING, SARAWAK.

Date: 15/8/2020

Date: 7/8/2020

Note * Thesis refers to PhD, Master, and Bachelor Degree

** For Confidential or Restricted materials, please attach relevant documents from relevant organizations / authorities

DECLARATION

I hereby declare that the thesis of LPG IoT measurement device with Data Analytics is based on my original work except for quotations and citations which have been duly acknowledging. I also declare that no portion of the work referred in this report has been submitted in support of an application for another degree at Universiti Malaysia Sarawak (UNIMAS).

…….

Muhammad Azri Hafiz bin Roslan (58804)

Software Engineering (WC10)

Faculty of Computer Science and Information Technology

Universiti Malaysia Sarawak (UNIMAS) Date: ……

ACKNOWLEDGEMENT

Alhamdulillah, all praises to Allah S.W.T. for the good health and wellbeing that were necessary to complete this project. I would like to express my sincere thanks to my supervisor and examiner, Mr. Ahmad Hadinata bin Fauzi and Dr Cheah Wai Shiang for the continuous guidance and encouragement throughout this project. I am also grateful to the MakerSpace management for providing me with all the necessary facilities for this project. Finally, I would like to thank my parents and friends for unceasing support throughout this project.

TABLE OF CONTENTS

DECLARATION ..... **i**

ACKNOWLEDGEMENT ..... **ii**

TABLE OF CONTENTS ..... **iii**

LIST OF FIGURES ..... **vi**

LIST OF TABLES ..... **x**

LIST OF EQUATIONS ..... **xi**

ABSTRACT ..... **xii**

ABSTRAK ..... **xiii**

Chapter 1 – INTRODUCTION **1**

1.1 – INTRODUCTION 1

1.2 – PROBLEM STATEMENT 2

1.3 – SCOPE 3

1.4 – OBJECTIVE 3

1.5 – PROJECT OVERVIEW 4

1.6 – BRIEF METHODOLOGY 4

1.7 – SIGNIFICANCE OF PROJECT 6

1.8 – PROJECT SCHEDULE 7

1.9 – EXPECTED OUTCOME 11

Chapter 2 – LITERATURE REVIEW **12**

2.1 – INTRODUCTION 12

2.2 – BACKGROUND 12

2.3 – REVIEW OF THE EXISTING TECHNIQUES 13

2.3.1 – CONVENTIONAL TECHNIQUE (USING WARM WATER) 13
2.3.2 – USING SOUND VIBRATION 14
2.3.3 – USING PRESSURE SENSOR 17
2.4 – COMPARISON OF ALL REVIEWED SYSTEM/TECHNIQUE 20
2.5 – ANALYSIS OF PROPOSED TECHNIQUE AND REVIEW TECHNIQUE 22
2.6 – SUMMARY 23
Chapter 3 – METHODOLOGY 24
3.1 – INTRODUCTION 24
3.2 – REQUIREMENTS ANALYSIS 24
3.2.1 - SOFTWARE REQUIREMENTS 24
3.2.2 - HARDWARE REQUIREMENTS 30
3.2.3 – QUESTIONNAIRES 33
3.3 – FUNCTIONAL REQUIREMENT 40
3.4 – NON-FUNCTIONAL REQUIREMENT 40
3.5 – SYSTEM DESIGN 41
3.5.1 – BLOCK DIAGRAM 41
3.5.2 – USE CASE DIAGRAM 42
3.5.3 – SEQUENCE DIAGRAM 47
3.5.4 – CONTEXT DIAGRAM 52
3.5.5 – ACTIVITY DIAGRAM 52
3.5.6 – ENTITY RELATIONSHIP DIAGRAM (ERD) 55
3.6 – DATABASE DESIGN 56
3.7 – APPLICATION INTERFACE & DEVICE PROTOTYPE DESIGN 58
3.7.1 – ANDROID APPLICATION INTERFACE DESIGN 58
3.7.2 – DEVICE PROTOTYPE DESIGN 60
3.8 – METHODOLOGY 61
3.8.1 – REQUIREMENTS PLANNING 61
3.8.2 – USER DESIGN 62
3.8.3 – CONSTRUCTION 63

3.8.4 – CUTOVER 63

3.9 – CONCLUSION 63

Chapter 4 – IMPLEMENTATION 64

4.1 - INTRODUCTION 64

4.2 – IOT DEVICE 64

4.2.1 – THE WIRING CONNECTIONS 64

4.2.2 – THE DEVICE CASING (ENCLOSURE) & LOAD CELL STAND 65

4.2.3 – CODING 68

4.3 – REST API 76

4.3.1 – CONNECT.PHP 76

4.3.2 – API.PHP 77

4.3.3 – APIOPERATION.PHP 79

4.3.4 – SEQUENCE DIAGRAM OF THE API 80

4.4 – ANDROID APPLICATION 81

4.4.1 – SIGN IN 81

4.4.2 – SIGN UP 83

4.4.3 – HOME 86

4.4.3 - USAGE 89

4.4.4 – PROFILE 91

4.4.5 – UTILITY CLASSES 101

4.4.6 – LIBRARIES USED 105

4.5 – SUMMARY 106

Chapter 5 - TESTING 107

5.1 - INTRODUCTION 107

5.2 – IOT DEVICE 107

5.2.1 – FUNCTIONALITY 107

5.2.2 – RELIABILITY 110

5.2.3 – EFFICIENCY 111

5.3 - API 112

5.3.1 – FUNCTIONALITY 112
5.3.2 – RELIABILITY 118
5.3.3 – EFFICIENCY 119
5.4 – ANDROID APPLICATION 120
5.4.1 – FUNCTIONALITY 120
5.4.2 – RELIABILITY 124
5.4.3 – EFFICIENCY 125
5.5 – SUMMARY 126
Chapter 6 – CONCLUSION AND FUTURE WORKS 127
6.1 – ACHIEVEMENTS 127
6.2 – PROJECT LIMITATIONS 127
6.3 – FUTURE WORK 128
6.4 – CONCLUSION 128
REFERENCES 129
FIGURE REFERENCES 130
APPENDIX 131

LIST OF FIGURES

Figure 1-1. Overview diagram of the proposed project 4

Figure 1-2. Rapid Application Development (RAD) methodology diagram 5

Figure 1-3. Gantt Chart of FYP 1 9

Figure 1-4. Gantt Chart of FYP 2 10

Figure 1-5. Sample design of the expected outcome, the measuring device, and a mobile application. 11

Figure 2-1. The conventional technique of measuring gas level in a gas cylinder 13

Figure 2-2. Contents inside the device used to measure the LPG level and how it is installed on an LPG tank 15

Figure 2-3. Knocking mechanism from pulling to pushing action 16

Figure 2-4. LPG tank with RFID device 17

Figure 2-5. Overview diagram of the pressure sensor system 18

Figure 3-1. PlatformIO IDE 25

Figure 3-2. Firebase Console 26

Figure 3-3. TinkerCAD's user interface 27

Figure 3-4. Repetier-Host's user interface 28

Figure 3-5. Android Studio IDE. 29

Figure 3-6. LoLin NodeMCU ESP8266 v3 32

Figure 3-7. 10kg straight bar load cell 32

Figure 3-8. HX711 load cell amplifier 33

Figure 3-9. Pie Chart of responses from question 1 34

Figure 3-10. Pie Chart of responses from question 2 34

Figure 3-11. Pie Chart of responses from question 3 35

Figure 3-12. Pie Chart of responses from question 4 36

Figure 3-13. Pie Chart of responses from question 5 36

Figure 3-14. Pie Chart of responses from question 6 37

Figure 3-15. Bar graph of responses from question 7 37

Figure 3-16. Bar graph of responses from question 8 38

Figure 3-17. Bar graph of responses from question 9 38

Figure 3-18. Bar graph of responses from question 10 39

Figure 3-19. Block Diagram of the proposed project 41

Figure 3-20. Use Case Diagram of the system 42

Figure 3-21. View LPG Usage sequence diagram 47

Figure 3-22. Set LPG threshold sequence diagram 48

Figure 3-23. Fetch weight readings sequence diagram 49

Figure 3-24. Notify users sequence diagram 49

Figure 3-25. Store weight readings sequence diagram 50

Figure 3-26. Send weight readings sequence diagram 51

Figure 3-27. Measure weight sequence diagram 51

Figure 3-28. Context Diagram of the proposed project 52

Figure 3-29. Activity Diagram on ESP8266 side 52

Figure 3-30. Activity Diagram on Firebase side 53

Figure 3-31. Activity Diagram on Application side 54

Figure 3-32. ERD of the proposed project 55

Figure 3-33. Interface design of the Home screen 58

Figure 3-34. Interface design of the Usage screen 59

Figure 3-35. The device prototype design 60

Figure 3-36. Rapid Application Development (RAD) diagram 61

Figure 4-1. The wiring connections between ESP8266, HX711, and load cell 64

Figure 4-2. The casing design 66

Figure 4-3. Load Cell stand 66

Figure 4-4. When casing and stand are screwed together 67

Figure 4-5. How the device looks like after assembled 67

Figure 4-6. Device with cover on top 68

Figure 4-7. Device code snippet #1 69

Figure 4-8. Device code snippet #2 69

Figure 4-9. Device code snippet #3 70

Figure 4-10. Device code snippet #4 71

Figure 4-11. Device code snippet #5 72

Figure 4-12. Device code snippet #6 73

Figure 4-13. Get device settings API response 73

Figure 4-14. Device code snippet #7 74

Figure 4-15. Device code snippet #8 74

Figure 4-16. Device code snippet #9 75

Figure 4-17. Device code snippet #10 75

Figure 4-18. Connect.php code snippet 76

Figure 4-19. API.php code snippet #1 77

Figure 4-20. API.php code snippet #2 78

Figure 4-21. API.php code snippet #3 78

Figure 4-22. APIOperation.php code snippet #1 79

Figure 4-23. APIOperation.php code snippet #2 80

Figure 4-24. Overview of API 80

Figure 4-25. Sign In activity 82

Figure 4-26. Sign In activity code snippet 82

Figure 4-27. Sign Up activity 83

Figure 4-28. Sign Up code snippet #1 84

Figure 4-29. Sign Up activity code snippet #2 85

Figure 4-30. Home fragment 86

Figure 4-31. Home fragment code snippet #1 87

Figure 4-32. Home fragment code snippet #2 88

Figure 4-33. Usage fragment 89

Figure 4-34. Usage fragment code snippet #1 90

Figure 4-35. Profile fragment 91

Figure 4-36. Change Password activity 92

Figure 4-37. Change Password activity code snippet #1 93

Figure 4-38. Change Password activity code snippet #2 94

Figure 4-39. IoT Device settings activity 95

Figure 4-40. IoT Device settings code snippet #1 96

Figure 4-41. Device info activity 97

Figure 4-42. Device Info code snippet #1 98

Figure 4-43. Device Info activity (Delete device) 99

Figure 4-44. Device Info code snippet #2 100

Figure 4-45. API class 101

Figure 4-46. SessionManager code snippet #1 101

Figure 4-47. SessionManager code snippet #2 102

Figure 4-48. SessionManager code snippet #3 102

Figure 4-49. SessionManager code snippet #4 102

Figure 4-50. Tool code snippet 103

Figure 4-51. VolleySingleton code snippet 104

Figure 4-52. Gradle code snippet #1 105

Figure 4-53. Gradle code snippet #2 105

Figure 4-54. Gradle code snippet #3 105

LIST OF TABLES

Table 1-1. Project Schedule 8

Table 2-1. Comparison of all reviewed systems/techniques 21

Table 3-1. Laptop's specifications 30

Table 3-2. Smartphone's specifications 31

Table 3-3. View LPG Usage use case table 43

Table 3-4. Set LPG threshold use case table 43

Table 3-5. Fetch weight readings use case table 44

Table 3-6. Notify User use case table 44

Table 3-7. Store weight readings use case table 45

Table 3-8. Send weight reading use case table 45

Table 3-9. Measure weight use case table 46

Table 3-10. Data dictionary for table 'User' 56

Table 3-11. Data dictionary for table 'LPG' 56

Table 3-12. Data dictionary for table 'Usage' 57

Table 13. Test case TC_DEVICE_01 107

Table 14. Test case TC_DEVICE_02 108

Table 15. Test case TC_DEVICE_03 108

Table 16. Test case TC_DEVICE_04 109

Table 17. Test case TC_DEVICE_05 109

Table 18. Reliability testing for IoT device 111

Table 19. Efficiency testing for IoT device 111

Table 20. Test case TC_API_01 112

Table 21. Test case TC_API_02 113

Table 22. Test case TC_API_03 113

Table 23. Test case TC_API_04 114

Table 24. Test case TC_API_05 114

Table 25. Test case TC_API_06 115

Table 26. Test case TC_API_07 115

Table 27. Test case TC_API_08 116

Table 28. Test case TC_API_09 116

Table 29. Test case TC_API_10 117

Table 30. Test case TC_API_11 117

Table 31. Reliabiliy testing for API 118

Table 32. Efficiency testing for API 119

Table 33. Test case TC_APP_01 120

Table 34. Test case TC_APP_02 121

Table 35. Test case TC_APP_03 122

Table 36. Test case TC_APP_04 122

Table 37. Test case TC_APP_05 123

Table 38. Reliability testing for application 124

Table 39. Efficiency testing for application 125

LIST OF EQUATIONS

Equation 4-1. Formula of contentPerPercent 69

Equation 4-2. readingVal formula when the value is between fullWeight and emptyWeight. 75

ABSTRACT

LPG is one of the important sources of energy for many homes and business in Malaysia. LPG is typically used as fuel in cooking equipment. For LPG owners, it is important for them to keep track of their LPG usage to ensure that their LPG does not run out when they are using it. Besides, it is also beneficial for the owners if they are notified when their LPG almost runs out. Thus, the proposed project is to develop an IoT device to measure the LPG level and an application that can notify the owners when their LPG is running out. Moreover, the owners can check their LPG usage daily, weekly, monthly, and yearly. With this proposed project, it helps the LPG owners to plan on when to replace their LPG.

ABSTRAK

LPG merupakan salah satu sumber tenaga penting untuk rumah dan bisnes di Malaysia. LPG kebiasaannya digunakan sebagai bahan api dalam peralatan memasak. Bagi pemilik LPG, menjejaki tahap kegunaan LPG adalah merupakan suatu perkara yang penting bagi memastikan mereka tidak kehabisan LPG sewaktu mereka menggunakannya. Selain itu, ia juga berfaedah untuk pemilik-pemilik tersebut jika mereka diberitahu sekiranya LPG mereka hampir habis. Oleh itu, projek ini mencadangkan untuk membangunkan sebuah peranti IoT untuk mengukur tahap LPG dan aplikasi untuk memberitahu pemilik-pemilik tersebut sekiranya LPG mereka hampir habis. Tambahan pula, pemilik-pemilik tersebut boleh menyemak penggunaan LPG mereka sepanjang hari, minggu, bulan, dan tahun tersebut. Dengan projek yang dicadangkan ini, ia akan membantu pemilik-pemilik tersebut untuk merancang lebih awal untuk menggantikan LPG mereka.

CHAPTER 1 – INTRODUCTION

1.1 – INTRODUCTION

Internet of Things (IoT) has been a popular trend nowadays. It is a system that connected to computing devices that has the capability of sending data over a network without the need of interaction between human and computer (Rouse, 2019). With IoT devices, it can help us to monitor things automatically.

One of the problems that restaurant owners usually faced is they must make sure that the cooking gas tank has enough gas to cook orders from the customers. To do that, the owners need to know how much the amount of LPG is left inside the LPG tank. The conventional way to check the amount of LPG left inside the LPG tank is by pouring warm water down the side of the LPG tank and feel the temperature change by placing own hand at where the water is poured. The colder part indicates the LPG level left inside the gas tank. This requires several steps just to check one LPG tank. It will be difficult for a restaurant owner who has more than one gas tank.

Therefore, instead of using the conventional technique, an IoT device with mobile application that can constantly monitor the amount of LPG left inside the LPG tank needs to be developed. It will help to reduce time and effort of the person to check the amount of LPG left and gives a more accurate reading. To achieve that, load cell or also known as weight sensor is used to constantly weigh the LPG tank. The weight of the LPG tank is then transferred to the application via cloud for the user to check the amount of LPG left inside the LPG tank.

With this device and mobile application, it can help the LPG tank owners to constantly measure the amount of LPG left inside their tanks. On the other hand, it will help them to plan earlier when they need to replace their LPG tank with a new one when it is almost empty.

1.2 – PROBLEM STATEMENT

Measuring the amount of LPG in an LPG tank is difficult. It is immeasurable and is limited to knowing the LPG tank is empty or not.

For example, restaurant owners need to make sure their LPG tank has enough LPG to cook all the customers' orders before it runs out. Besides, it is hard to get LPG supplies when it is out of office hours. Restaurant owners usually buy a couple extra of LPG tanks to overcome this problem.

There is a conventional technique of checking the amount of LPG inside the LPG tank. That technique requires them to pour warm water down on the side of LPG tank and feel the temperature change at the side of the LPG tank using their hand. The technique is not reliable as it depends on them if they do feel the temperature change. Different people might feel differently. Thus, using this technique is inaccurate. Apart from that, it is difficult and dangerous for them to frequently check the amount of LPG left inside the LPG tank if it is being used. On the other hand, this technique is time consuming and risky as their hand might get burnt if the warm water is not handled properly.

1.3 – SCOPE

The scope of this project is to measure the amount of LPG inside an LPG tank by using load cell and microcontroller with built in WiFi. Since getting the actual LPG tank is laborious, this project will be focusing on developing a smaller scale as prototype for proof of concept. A water bottle is used to represent the LPG tank. As for the LPG, it will be replaced with water or any liquid.

For the Android app, it is developed with minimum SDK Android 4.4 (KitKat) and target SDK of Android 8.0 (Oreo). Besides, Wi-Fi will be used as a medium of transferring data from the microcontroller to Google's Firebase on the Internet.

1.4 – OBJECTIVE

The main objective of this project is to develop an IoT device that can measure the amount of LPG inside an LPG tank. The device will give an accurate percentage of the LPG left in the tank.

- The amount of LPG in LPG tank can be measured using load cell.
- The LPG usage statistics can be viewed on Android mobile application.
- The mobile application can send push notification when the amount of LPG is below a certain threshold set by the user.

1.5 – PROJECT OVERVIEW

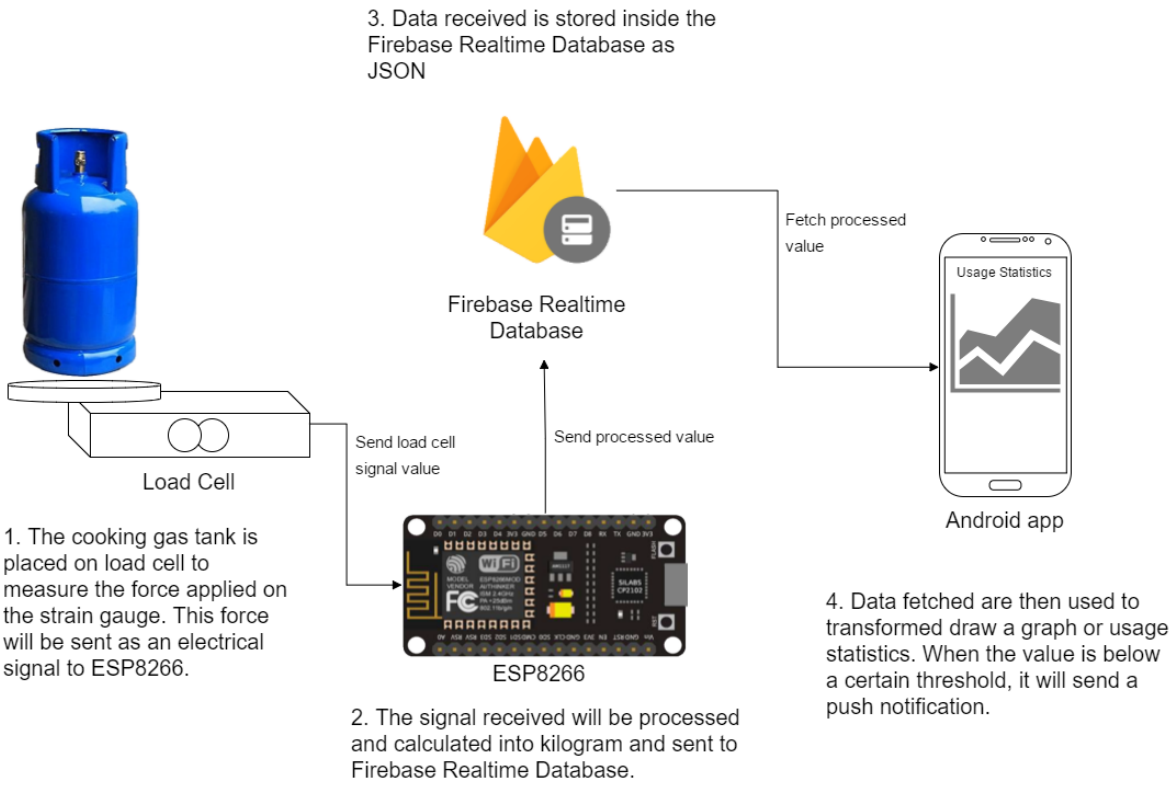


Figure 1-1. Overview diagram of the proposed project

1.6 – BRIEF METHODOLOGY

In this project, Rapid Application Development (RAD) methodology is going to be used to develop both IoT device and mobile application. Anderson (2019) stated that RAD is a form of Agile methodology that focuses on developing the software and fulfilling user needs. Besides, this methodology allows the developer to maximize the development of the project's prototype. Below is the diagram of the process of RAD.

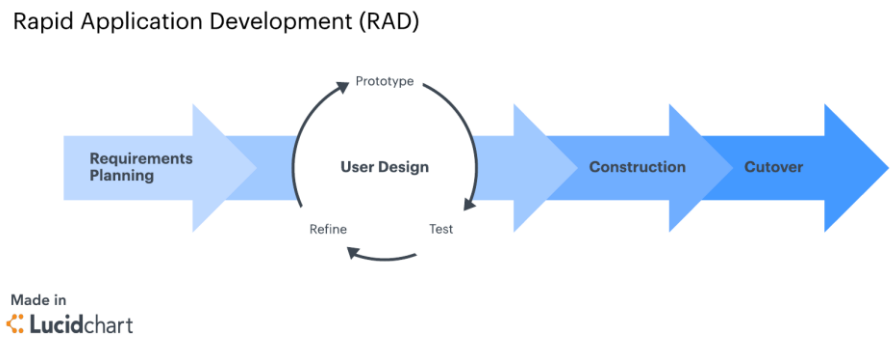


Figure 1-2. Rapid Application Development (RAD) methodology diagram. *Reprinted from* 4 Phases of Rapid Application Development Methodology*, by Lucidchart Content Team, 2018, Retrieved from https://www.lucidchart.com/blog/rapid-application-development-methodology. Copyright 2018 by Lucidchart. Reprinted with permission.*

Phase 1: Requirements planning

During this phase, the requirements and scope of this project are being discussed with the supervisor and client to identify everything that needs to be done to solve the problem. The requirements and scopes are then recorded and documented.

Phase 2: User Design

In this phase, several prototypes will be built out to meet the requirements and scope that has been discussed previously. At first, a prototype will be developed and then tested out by the client to see whether it meets the requirements and to get feedbacks. Once it has been tested, the prototype is then modified and improved according to the feedbacks obtained. The phase of prototyping, testing, and refining will be repeated until all the requirements are met.

Phase 3: Construction

In this phase, a final working model will be developed as the problems and changes addressed by the client has been thoroughly considered during the previous phase. Therefore, it will be faster to develop the final working model. Some alterations or changes will also be made during this phase if it is needed.

Phase 4: Cutover

The completed product is then launched during this phase. A training will be done for the user on how to use the IoT device and mobile application developed.

1.7 – SIGNIFICANCE OF PROJECT

The primary targets that will achieve throughout this project are:

- Impact on the study:
    - Provides an accurate measurement of the amount of LPG inside an LPG tank in terms of percentage.
    - Provides a new way of measuring the amount of LPG inside an LPG tank using IoT devices, load cell and ESP8266.
    - Provides easy remote monitoring of the LPG inside an LPG tank through mobile application.
    - Completing the documentation of measuring the amount of LPG inside an LPG tank using load cell and ESP8266.

- Impact on society:
    - Provides an efficient and autonomous way of measuring the amount of LPG inside an LPG tank.
    - Allows society to plan accordingly based on the LPG usage and amount left.

1.8 – PROJECT SCHEDULE

TASK	DAYS	START	END
FYP 1 Briefing			
Looking for Supervisor	1	9/13/19	9/13/19
FYP title discussion with supervisor	1	9/14/19	9/14/19
Working on Brief Project Description	2	9/15/19	9/16/19
Submit Brief Project Description to supervisor for	2	9/17/19	9/18/19
Fixing Brief Project Description according to supervisor's feedbacks	2	9/19/19	9/20/19
Submission of Brief Project Description	1	9/21/19	9/21/19
Approval and Comments by Reviewers	1	9/22/19	9/22/19
Working on Full Proposal	12	9/23/19	10/4/19
Submission of Full Proposal on FYP System	1	10/5/19	10/5/19
Chapter 1			
Working on Chapter 1	20	10/6/19	10/25/19
Submission of Chapter 1 on FYP System	1	10/26/19	10/26/19
Chapter 2			
Working on Chapter 2	20	10/27/19	11/15/19
Submission of Chapter 2 on FYP System	1	11/16/19	11/16/19
Chapter 3			
Working on Chapter 3	18	11/17/19	12/4/19

Submission of Chapter 3 on FYP System	1	12/5/19	12/5/19
Submission of Full FYP 1 Report	1	12/12/19	12/12/19
Final Year Project Symposium	2	12/17/19	12/18/19
Submission of final report	1	1/11/20	1/11/20
FYP 2 Briefing	1	2/11/20	2/11/20
FYP Prototype Development			
Working on IoT Device	14	2/10/20	2/23/20
Working on PHP REST API	7	2/17/20	2/23/20
Working on Android App	30	2/22/20	3/22/20
Chapter 4			
Working on Chapter 4	42	2/10/20	3/22/20
Submission of Chapter 4 on FYP System	1	3/23/20	3/23/20
Chapter 5			
Working on Chapter 5	14	3/23/20	4/5/20
Submission of Chapter 5 on FYP System	1	4/6/20	4/6/20
Chapter 6			
Working on Chapter 6	9	4/6/20	4/14/20
Submission of Chapter 6 on FYP System	1	4/15/20	4/15/20
Submission of Full FYP Report	1	4/15/20	4/15/20
Final Year Project 2 Symposium	2	5/5/20	5/6/20
Submission of final report	1	5/29/20	5/29/20

Table 1-1. Project Schedule

GANTT CHART

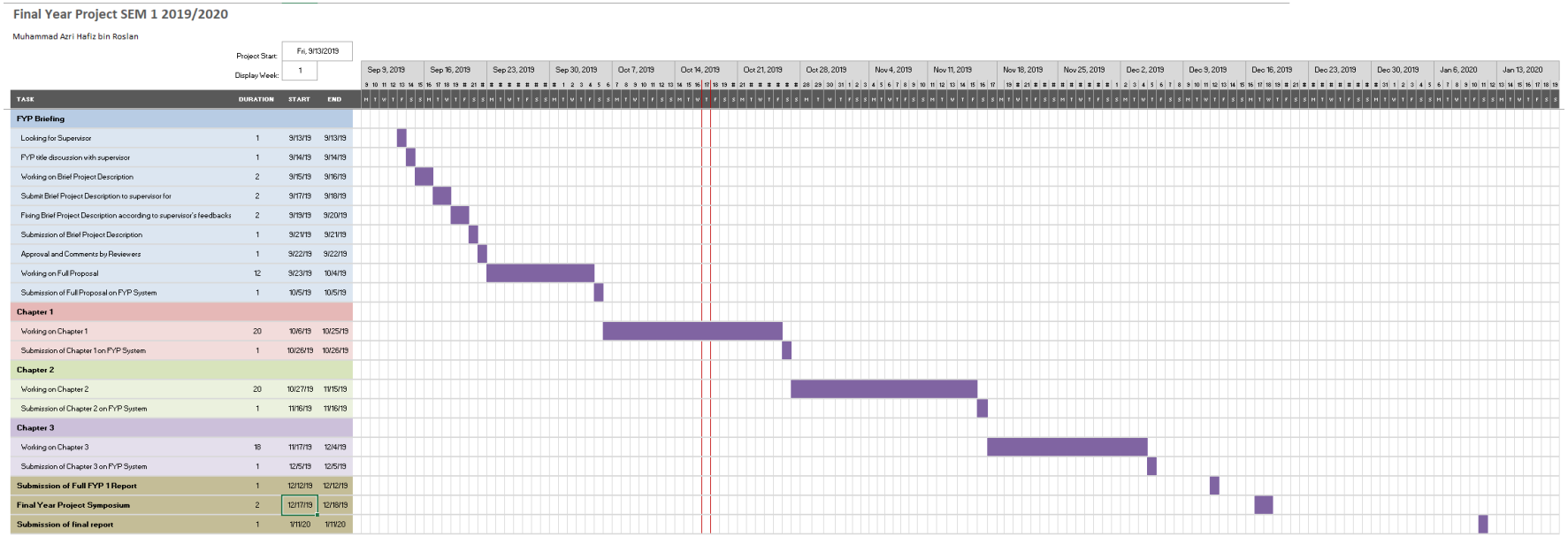


Figure 1-3. Gantt Chart of FYP 1

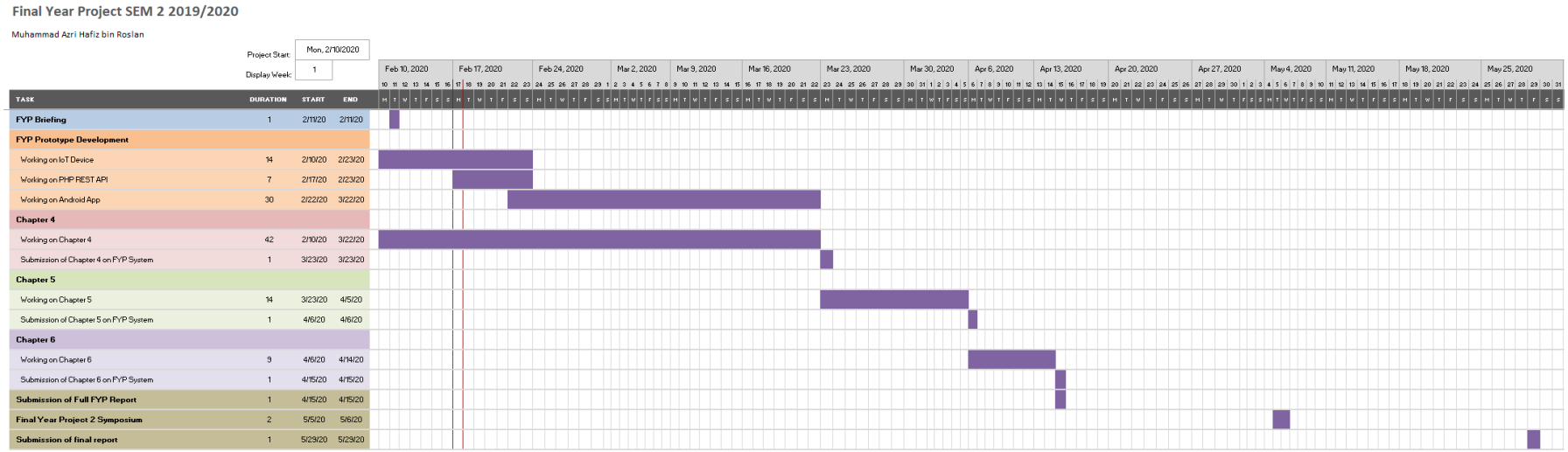


Figure 1-4. Gantt Chart of FYP 2

1.9 – EXPECTED OUTCOME

At the end of this project, an IoT device to measure the weight of the LPG in an LPG tank is developed. The device can measure the weight of the LPG accurately and constantly. It will greatly help the owners and users of the IoT device and mobile application to get the accurate measurement of LPG level left inside their LPG tank without having to weigh it themselves.

Next, users of the mobile application can view their LPG usage statistics. This could help them to plan or check on which certain period they need to lower down their usage if they are on a tight budget. Other than that, based on the statistics shown on the mobile application, they can plan when they need to replace their LPG with a fully filled LPG tank.

Lastly, users do not have the time to constantly check their amount of LPG left through the mobile application. Therefore, the mobile application must be able to notify the users if their LPG reached or below a certain threshold.

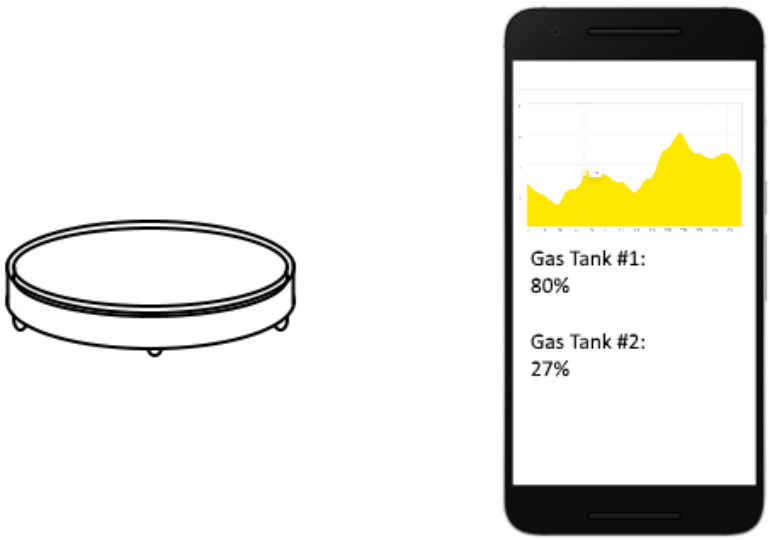


Figure 1-5. Sample design of the expected outcome, the measuring device, and a mobile application.

CHAPTER 2 – LITERATURE REVIEW

2.1 – INTRODUCTION

In this chapter, the existing related projects are analysed, evaluated, and compared with the proposed solution. To achieve the objectives and requirements of the proposed project, a study of existing related projects is carried out. These literature reviews use various sources, for example, articles, books, journals, and websites to collect information on how to measure the LPG level in an LPG tank. Apart from that, it will be used to identify the strengths and weaknesses of the related existing projects and then compared them with the proposed project.

2.2 – BACKGROUND

Previously in Chapter 1, it has been identified that the LPG level in an LPG tank is immeasurable and is limited to knowing the LPG tank is empty or not. Therefore, the LPG tank owners usually will buy a couple extra of LPG tanks to overcome this problem.

Regardless of having extra LPG tanks, they still need to know when they need to replace their LPG tank if they want to plan their budget. Therefore, developing an IoT device that utilises load cell to measure the LPG level in an LPG tank will help them to plan early whether they need to replace their LPG tank or not. The load cell can accurately measure the LPG level based on the weight of the LPG tank. The lighter the LPG tank means the lesser the amount of LPG inside it.

Even though there is a conventional technique of measuring the LPG level, it requires the owner to feel the temperature change using their own hand after they pour warm water on the side of

the LPG tank. This technique is unreliable and laborious as it depends on the person, whether they can feel the temperature change or not. On the other hand, the technique is considered dangerous because the person needs to be near to the LPG tank.

2.3 – REVIEW OF THE EXISTING TECHNIQUES

Several techniques have been developed to measure the LPG level in a LPG tank starting from the conventional technique using warm water to the latest techniques of using different methods to measure the LPG level such as by knocking the LPG tank and measuring the pressure inside the LPG tank using IoT devices. These techniques are reviewed in this subsection.

2.3.1 – CONVENTIONAL TECHNIQUE (USING WARM WATER)

Figure 2-1. The conventional technique of measuring gas level in a gas cylinder. *Reprinted from* ELGAS, *by E. Hahn, 2019, Retrieved from* https://www.elgas.com.au/blog/353-how-much-gas-is-left-in-the-gas-bottle. *Copyright 2019 by ELGAS. Reprinted with permission.*

Conventional technique is a technique that is commonly being used by people. In this case, the conventional technique to measure the LPG level is by using warm water. According to Hahn (2019), to measure the LPG level, they must pour the warm water down the side of the LPG tank. Then, using their hand, touch the part where they poured the water on LPG tank from top to bottom until they feel the LPG tank is colder than before. The part where they feel the temperature change is where the LPG level is.

This technique requires only warm water to measure the LPG level. However, this technique is not reliable as it depends on them if they can feel the temperature change. Apart from that, the technique is inefficient because they must pour warm water and feel the temperature change every time they want to measure the LPG level. On the other hand, it is difficult and dangerous for the person to measure the LPG level when the LPG tank is being used.

2.3.2 – USING SOUND VIBRATION

2.3.2.1 - INTRODUCTION

In 2016, Wu, Hung, Chang, and Yang proposed a technique of measuring the LPG level in an LPG tank using sound vibration. According to Wu, Hung, Chang, and Yang (2016), this technique measures the LPG level using the tone frequencies produced by knocking on the outside surface of the LPG tank. The algorithm used to calculate the tone frequency is the bending vibration algorithm based on Euler-Bernoulli beam theory.

2.3.2.2 - SYSTEM DESIGN

In this technique, the device is placed on the side of the LPG tank, as shown in Figure 2-2. The device will knock the side of the LPG tank to produce sound and then uses the sound vibration produced to measure the LPG level inside the LPG tank.

Figure 2-2. Contents inside the device used to measure the LPG level and how it is installed on an LPG tank. *Reprinted from* Analysis and Implementation of Noncontact Level Sensing for a Pressurized Cylinder, *by H. P. Wu, C. Hung, S. Chang, Z. Yang, 2016, Taiwan:Journal of Sensors. Copyright 2016 by the Hindawi Publishing Corporation. Reprinted with permission.*

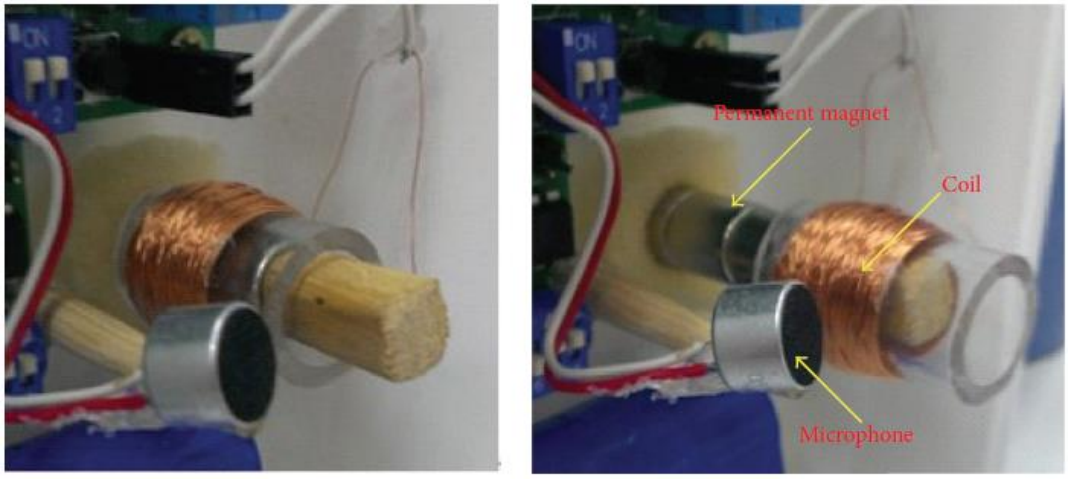


Figure 2-3. Knocking mechanism from pulling to pushing action. *Reprinted from* Analysis and Implementation of Noncontact Level Sensing for a Pressurized Cylinder, *by H. P. Wu, C. Hung, S. Chang, Z. Yang, 2016, Taiwan:Journal of Sensors. Copyright 2016 by the Hindawi Publishing Corporation. Reprinted with permission.*

Based on Wu, Hung, Chang, and Yang (2016), the sound vibration is obtained by using a microphone that is placed beside the coil. The coil is working with a strong magnet to produce pushing(knocking) and pulling action. The actions are controlled by the direction of current flowing through the coil. The knocking action is executed once per day at a specified time. The device will send out an alarm signal if the LPG tank is close to empty.

2.3.3 – USING PRESSURE SENSOR

2.3.3.1 – INTRODUCTION

In 2014, Juvanna and Meenakshi proposed a technique of measuring the LPG level and leakage detection in a LPG tank using a built-in pressure sensor in RFID device. The device will alarm the owner if the sensor detects a leakage. But for this project purposes, we will be focusing on to how they measure the LPG level in an LPG tank.

2.3.3.2 – SYSTEM DESIGN

This technique utilises a built-in pressure sensor in an RFID module to measure the LPG level in an LPG tank. According to Juvanna and Meenakshi (2014), the pressure sensor will be directly mounted on the transport cap of the LPG tank as shown in Figure 4.

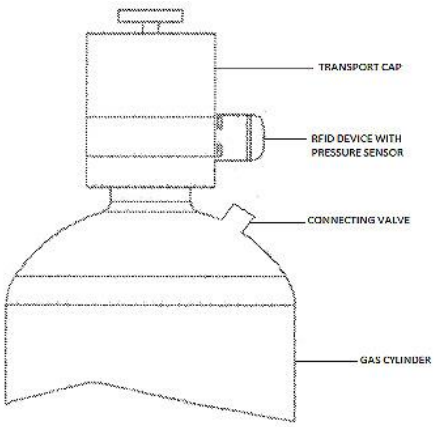
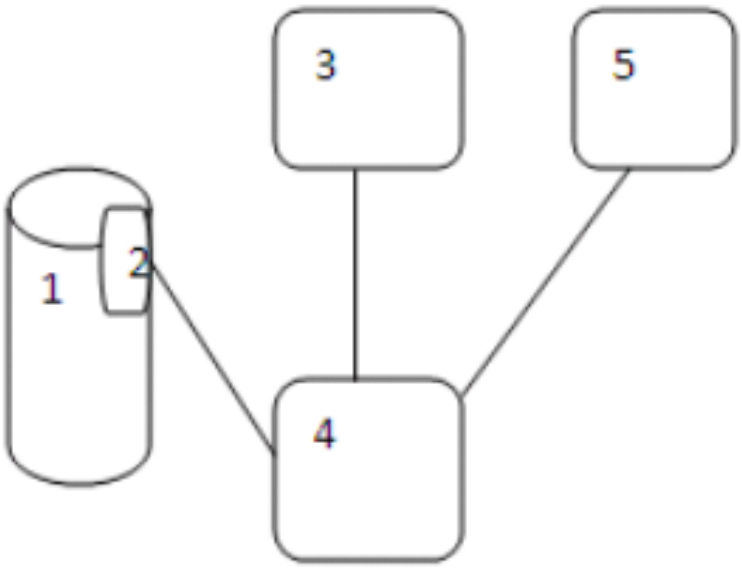


Figure 2-4. LPG tank with RFID device. *Reprinted from* Gas Level Detection and Leakage Monitoring System using a Specific Technique, *by I. Juvanna and N. Meenakshi, 2014, Padur, Chennai:International Journal of Computer Science and Mobile Computing. Copyright 2014 by the Hindustan Institute of Technology and Science. Reprinted with permission.*

The pressure sensor continuously measures the pressure inside the LPG tank. Then, the reading obtained is sent to the system using radio frequency technology. The pressure sensor will produce an analog signal as a reading output. The signal then will strike the cylindrical surface which produces electromagnetic (EM) wave. The RFID will read the EM wave and send it to the controller. When the pressure reaches a certain threshold, the controller will send the signal to an auto dialler. The auto-dialler will dial the owner to inform whether the LPG tank needs to be replaced or there is leakage.



1. Cylinder, 2. RFID Tag and Pressure sensor, 3. The LCD display, 4. PIC controller, 5. Auto dialler

Figure 2-5. Overview diagram of the pressure sensor system. *Reprinted from Gas Level Detection and Leakage Monitoring System using a Specific Technique, by I. Juvanna and N. Meenakshi, 2014, Padur, Chennai:International Journal of Computer Science and Mobile Computing. Copyright 2014 by the Hindustan Institute of Technology and Science. Reprinted with permission.*

Figure above shows the overall diagram of the system. The built-in pressure sensor with RFID is attached to the LPG tank to measure the amount of LPG level inside it. The reading is then sent to a PIC controller to be analysed and check what action needs to be executed next according to the reading received by the PIC controller. The reading is displayed on the LCD display. If the reading reaches a certain threshold, it will send a signal to the auto dialler.

2.4 – COMPARISON OF ALL REVIEWED SYSTEM/TECHNIQUE

TITLE	TECHNIQUE PROPOSED	STRENGTH	WEAKNESS	TOOLS
The conventional technique (Using warm water)	-	• Low cost, as it only requires warm water	• Unable to constantly measure • Highly depending on if the person able to feel the temperature change • Unable to notify the user if the gas is empty or not	• Warm water

Analysis and Implementation of Noncontact Level Sensing for a Pressurized Cylinder	Using sound vibration that utilizes bending vibration algorithm based on Euler-Bernoulli beam theory to measure the LPG level	• Able to notify the user using the alarm if the LPG tank is near empty.	• Only check the LPG tank once per day • Only able to detect if the LPG tank is near empty or not.	• Microphone • Alarm • Knocking mechanism (using magnets and coil)
Gas Level Detection and Leakage Monitoring System using a Specific Technique	Measuring the LPG level by using the reading obtained from the pressure sensor	• Able to continuously measure the LPG level • Able to notify the user using auto dialler if the LPG tank reaches a certain threshold	• Only applicable to certain types of LPG tanks	• Pressure sensor • RFID • LCD • PIC controller • Auto dialler

Table 2-1. Comparison of all reviewed systems/techniques

2.5 – ANALYSIS OF PROPOSED TECHNIQUE AND REVIEW TECHNIQUE

In this section, the three reviewed existing techniques are compared and explained in detail with the proposed technique. Based on the limitations of existing techniques, there are several things that set this proposed technique apart from them and why this technique should be implemented.

The proposed technique provides an affordable, autonomous, simple, and with more functionalities, to measure the LPG level in an LPG tank compared to the existing techniques. It uses the IoT sensor and module called load cell and ESP8266 to measure the LPG level according to the weight of the LPG tank. Both are cheap and easy to obtain. However, the system that uses sound vibration requires numerous items to develop a device that measures the LPG level. As for the technique that uses pressure to measure the LPG level, it needs to use a specific module which is an RFID module with a built-in pressure sensor in it. Both are costly and hard to obtain.

Besides, the LPG level in the LPG tank can be measured and continuously monitored with the proposed technique compared to the conventional technique and the technique that uses sound vibration. Even though the sound vibration technique does measure the LPG level daily, however, it can only detect whether the LPG tank is near empty or not. As for the proposed technique, it uses the LPG tank's weight to measure the LPG level inside it. Therefore, we can place the LPG tank on top of the load cell to measure its weight regularly. Apart from that, it can help to tell the LPG tank owners the accurate estimated LPG level in their LPG tank since the weight is measured regularly.

Furthermore, it notifies the owners if their LPG level reaches below a certain threshold with the proposed technique. Both techniques that use sound vibration and pressure sensor do also notify the owners. However, the method of notifying for the sound vibration technique is by using an alarm. The owners will not be able to hear the alarm if they are not close to the LPG tank. On the other hand, the pressure sensor technique uses an auto dialler to notify the owners or suppliers. This method is not efficient as it needs to dial the owner's or supplier's telephone number to send a notification. With this proposed technique, it solves both problems from the existing techniques by sending a notification via mobile application. It utilises Firebase push notification service which is faster than dialling a telephone number.

Lastly, the reviewed techniques do not provide the owners with any means of checking their LPG usage. With the proposed technique, the owners can check their usage statistics to help them plan when they need to replace their LPG tank.

2.6 – SUMMARY

To conclude, three existing techniques that are related to the proposed technique are reviewed within this chapter. The reviews are done based on their system design, strengths, and weaknesses. Load cell is used to measure the LPG level in an LPG tank in this project. This project will produce an IoT device to measure the LPG level using a microcontroller with built in WiFi called ESP8266. Finally, a mobile application will also be developed to display the usage of LPG level in the LPG tank and notify the owners if their LPG level is below a specified threshold.

CHAPTER 3 – METHODOLOGY

3.1 – INTRODUCTION

This chapter discusses in detail the methodology used to develop the LPG level measurement device using the proposed technique. All the requirements, whether it is software or hardware, are also discussed in this chapter.

3.2 – REQUIREMENTS ANALYSIS

In this project, there are two essential requirements involved namely, software and hardware requirements. Software requirements refer to the software or tool used to develop the device. As for the hardware requirements, it refers to the hardware used in developing the device to measure the LPG level. This project focuses on developing a device that can measure the LPG level using its weight.

3.2.1 - SOFTWARE REQUIREMENTS

Several software applications are being used in this project. Those software applications are:

- PlatformIO (version 1.10.0)
- Firebase
- TinkerCAD
- Repetier-Host
- Android Studio 4.0

3.2.1.1 – PlatformIO

PlatformIO is an open-source and cross-platform IDE. PlatformIO is available to be installed as an extension in Visual Studio Code. PlatformIO provides users with the functionality of writing and uploading Arduino codes directly from Visual Studio Code to Arduino boards. Apart from that, PlatformIO has Intelligent Code Completion and Smart Code Linter which helps in rapid development. In this project, PlatformIO will be used to write the Arduino codes into NodeMCU ESP8266.

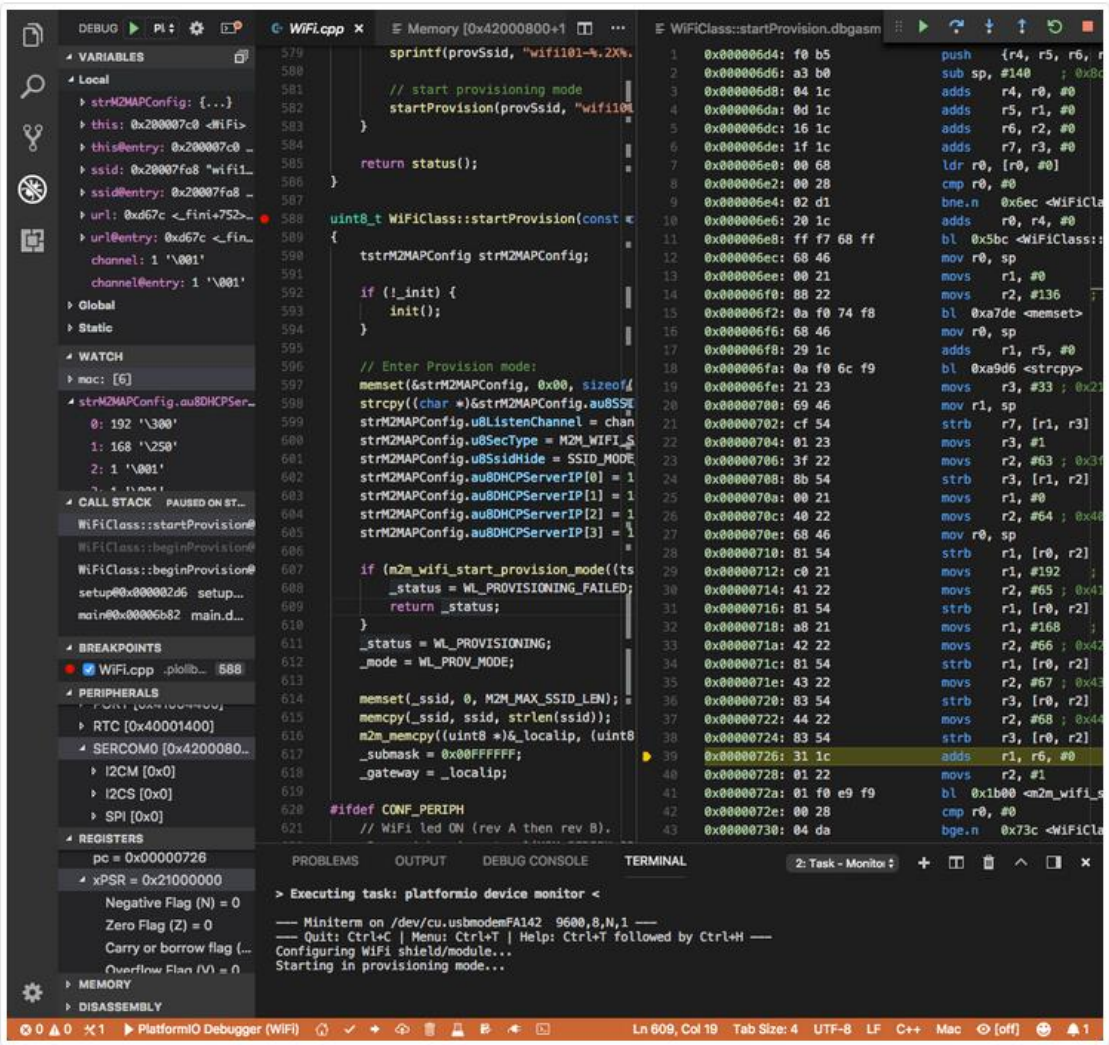


Figure 3-1. PlatformIO IDE. *Reprinted from* PlatformIO, *by PlatformIO, 2019, Retrieved from https://platformio.org/. Copyright 2019 by PlatformIO. Reprinted with permission.*

3.2.1.2 – Firebase

Firebase is a cloud service platform for mobile and web application development by Google. Firebase provides various types of services such as real-time database and other backend services. These services provide an API to developers that allows them to synchronise their application data across clients and to be stored on the Firebase's cloud storage. In this project, Firebase will be used to handle the backend services between NodeMCU ESP8266 and Android application.

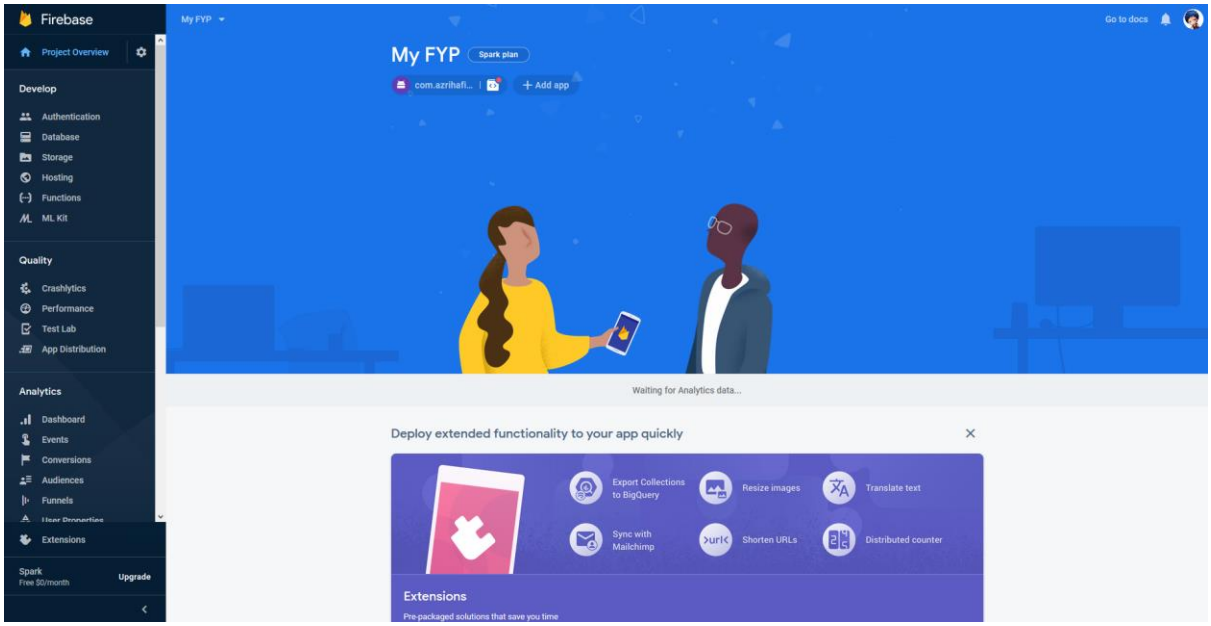


Figure 3-2. Firebase Console

3.2.1.3 – TinkerCAD

TinkerCAD is an online 3D modelling platform offers by Autodesk, Inc. TinkerCAD provides users with an easy to use user interfaces which is suitable for those who are new to 3D modelling. Apart from that, TinkerCAD can be used directly from a web browser, and their models are saved on TinkerCAD's cloud. It means that users can access their 3D models anywhere and anytime. TinkerCAD allows users to export their 3D models into an STL, OBJ and more file formats. In this project, TinkerCAD will be used to design a 3D model LPG tank and casing for the device to measure the LPG level.

Figure 3-3. TinkerCAD's user interface

3.2.1.4 – Repetier-Host

Repetier-Host is a 3D printing software developed by Hot-World GmbH & Co. KG. Repetier-Host helps in slicing a 3D model to be used for 3D printing. Repetier-Host allows users to load various types of 3D model file formats such as STL, OBJ, and more. In this project, Repetier-Host is used to slice the 3D model to be prepared for 3D printing.

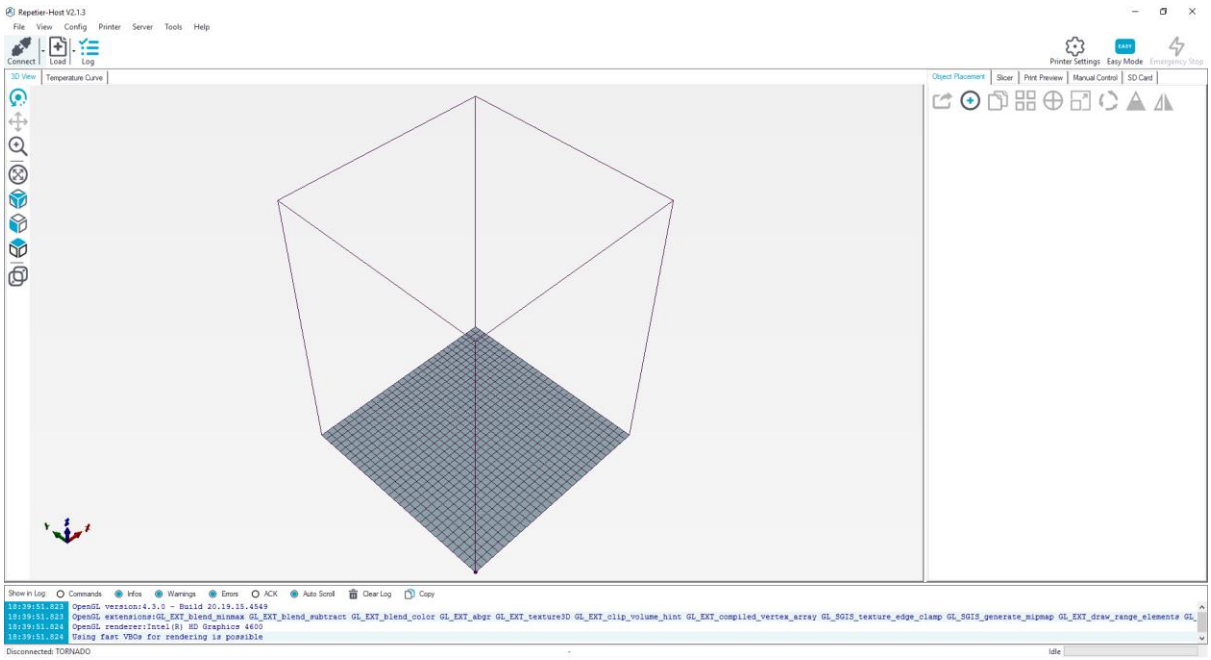


Figure 3-4. Repetier-Host's user interface

3.2.1.5 – Android Studio

Android Studio is an IDE for developing an Android application. Android Studio is based on IntelliJ IDEA. Android Studio is compatible with various operating systems, namely Windows, macOS, Linux and Chrome OS. Besides, Android Studio has various functionalities to aid the developers in developing their applications. One of the main features that can help to make Android application development easier is its rich layout editor. It allows users to drag and drop the user interface components into their application layout without having to code the layout components one by one manually. Also, users can preview their layout on different screen sizes using the layout editor. Android Studio also allows users to run their app on an Android emulator directly from Android Studio. In this project, Android Studio is used to develop the Android application to monitor the LPG level. Android Studio provides an option for developers whether they want to develop the application using Kotlin or Java. For this project, Java will be used to develop the application. Apart from that, the Android emulator will be used to check whether the application supports different Android phone screen size.

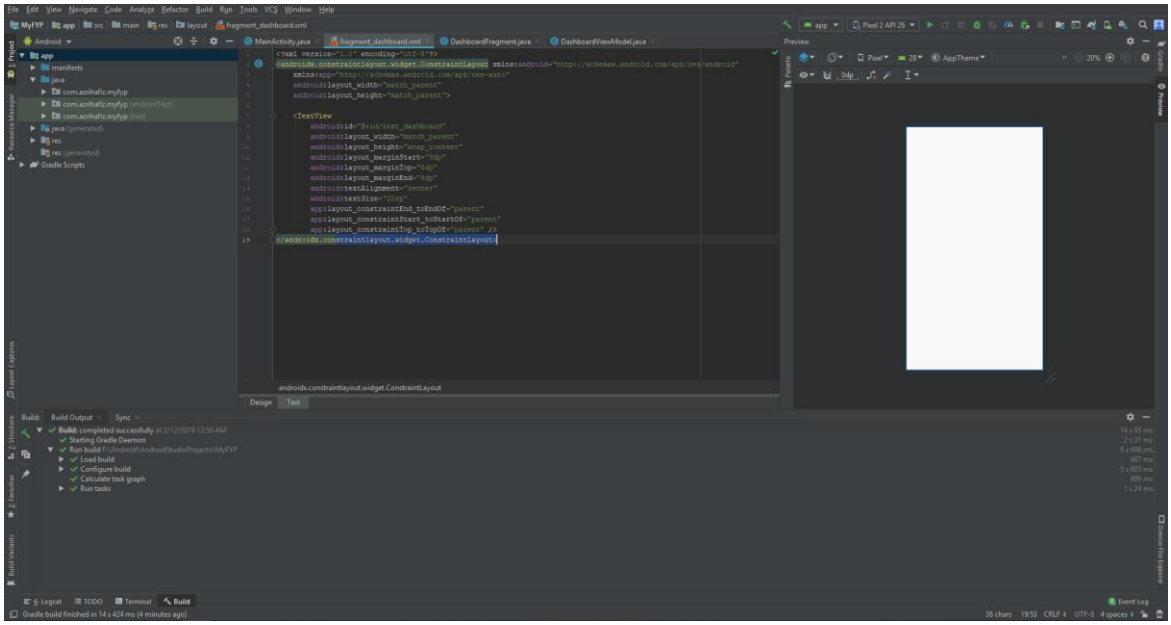


Figure 3-5. Android Studio IDE.

3.2.2 - HARDWARE REQUIREMENTS

To develop the proposed project, several hardware components are required. Those hardware are:

- Laptop
- Android phone
- Arduino components
- 3D printer

3.2.2.1 – Laptop

Throughout this project, Acer Aspire V15 Nitro laptop will be used to run the required software. The laptop's specifications are enough to run the required software. The specifications are as follows:

Specification	Description
Laptop model	Acer Aspire V15 Nitro Black Edition (Aspire VN7-591G)
Operating System	Windows 10 Home Single Language
Processor	Intel Core i7-4720HQ (2.60GHz)
RAM	8.00 GB 1600MHz
GPU	Intel HD Graphics 4600 NVIDIA GTX860M

Table 3-1. Laptop's specifications

3.2.2.2 – Android phone

To test and deploy the Android application, this project requires an Android phone. In this project, the phone that is used with its specifications is as the table below:

Specification	Description
Device name	HUAWEI nova 2i (RNE-L22)
Android version	8.0.0 (Oreo)
CPU	Hisilicon Kirin 659
RAM	4.0 GB

Table 3-2. Smartphone's specifications

3.2.2.3 – Arduino IoT components

To develop the LPG level measurement device, it requires several components. One of the main components needed is a microcontroller. In this project, the weight reading is sent into Firebase's Realtime Database. Therefore, to make the reading able to be sent into Firebase's Realtime Database, it requires a board that can connect to the Internet. Hence, LoLin NodeMCU ESP8266 v3 is used in this project as it has a built-in WiFi module that allows itself to connect to the Internet.

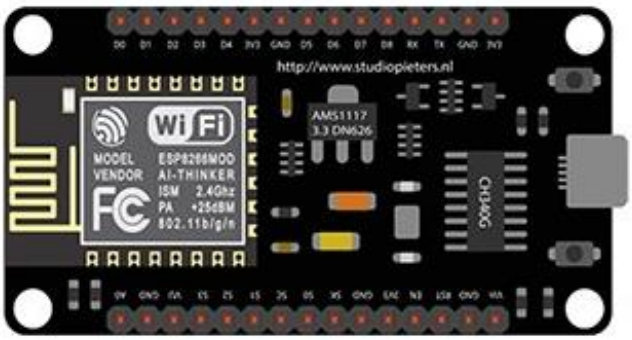


Figure 3-6. LoLin NodeMCU ESP8266 v3. *Reprinted from* Quick start with NodeMCU v3 (ESP8266), Arduino ecosystem and PlatformIO IDE, *by D. Loginov, 2018, Retrieved from https://medium.com/@loginov_rocks/quick-start-with-nodemcu-v3-esp8266-arduino-ecosystem-and-platformio-ide-b8415bf9a038. Copyright 2018 by Medium. Reprinted with permission.*

Next, to measure the weight, it requires a load cell. Omega (n.d.) stated that a load cell is a type of transducer that converts force into an electrical signal. There are different types of load cells, but the one used in this project is a straight bar type load cell. For developing a smaller scale of prototype, this project will be using a 10kg straight bar load cell.

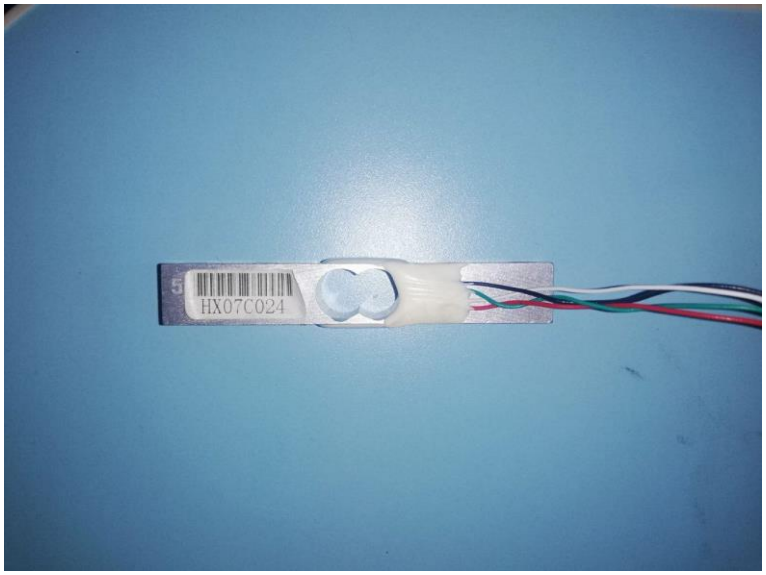


Figure 3-7. 10kg straight bar load cell

However, the signal strength is not strong enough to be read by ESP8266. Therefore, it needs to use an amplifier to boost the signal so ESP8266 can read it.

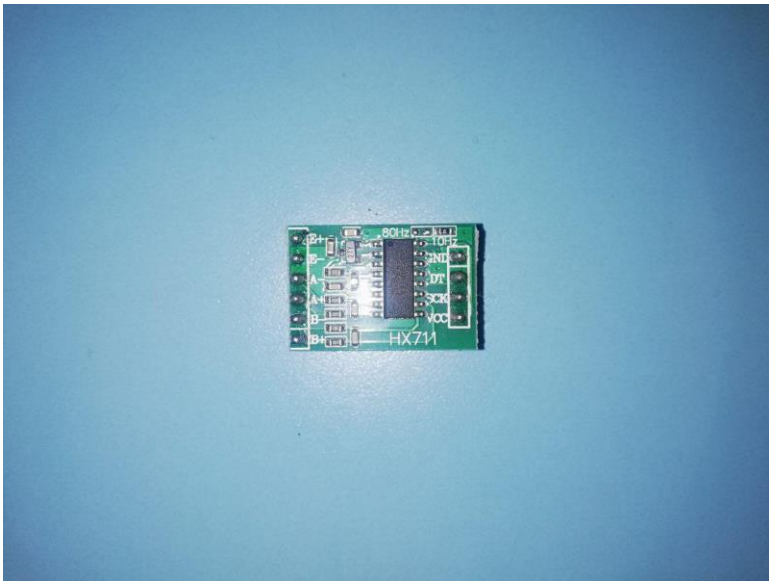


Figure 3-8. HX711 load cell amplifier

3.2.2.4 – 3D printer

To create a prototype model for the LPG tank, a 3D printer is used to 3D print the gas models. In this project, the 3D printer used is TEVO Tornado 3D printer.

3.2.3 – QUESTIONNAIRES

To ensure that this proposed project is practical, a survey has been conducted to study the problems faced by LPG tank owners. This section discusses in detail the responses obtained from the questionnaire.

Question 1: Do you own LPG tank(s)?

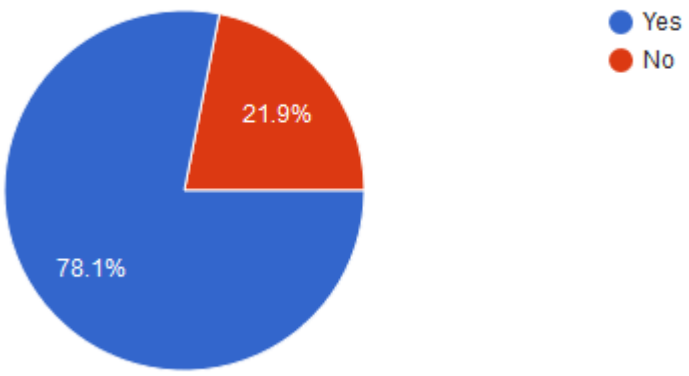


Figure 3-9. Pie Chart of responses from question 1

Based on the responses, it shows that most of the respondents do own cooking LPG tank(s). 78.1% own LPG tanks while 21.9% do not own LPG tank.

Question 2: How often do you replace your LPG tank(s)?

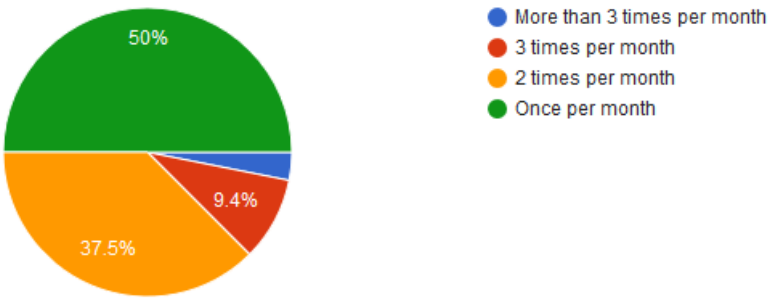


Figure 3-10. Pie Chart of responses from question 2

Based on the responses obtained, 50% of the respondents replace their LPG tank(s) once per month, 37.5% replace their LPG tank(s) two times per month, 9.4% replace their LPG tank(s) 3 times per month, and the rest is more than 3 times per month.

Question 3: Do you know how to check the LPG level?

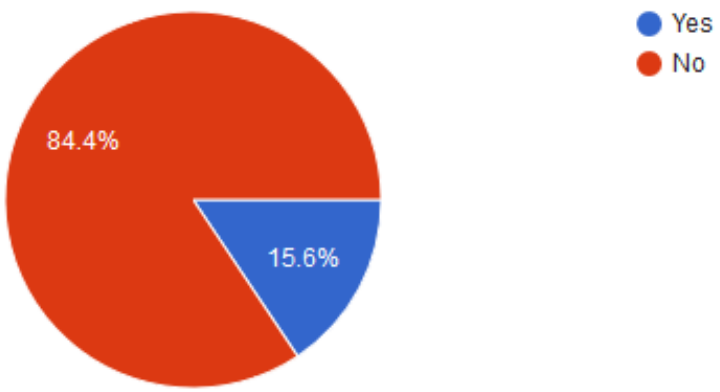


Figure 3-11. Pie Chart of responses from question 3

Based on the responses, most of the respondents do not know how to check their LPG level. 84.4% do not know how to check their LPG level, while 15.6% do know how to check their LPG level using the conventional technique.

Question 4: Do you find it inconvenient that you are limited to knowing whether your LPG tank is empty or not?

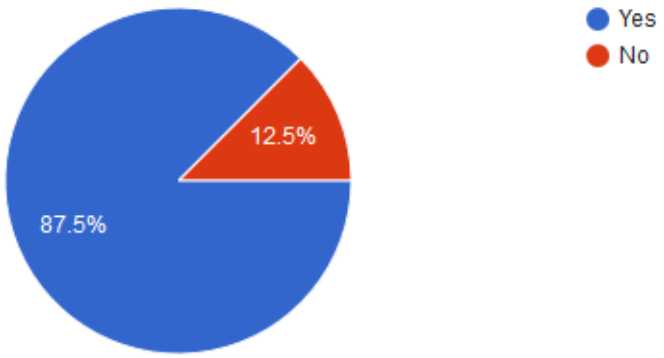


Figure 3-12. Pie Chart of responses from question 4

Based on the responses, it shows that most of the respondents find that it is inconvenient that they are limited to knowing whether their LPG tank is empty or not.

Question 5: How do you solve the problem in Q4?

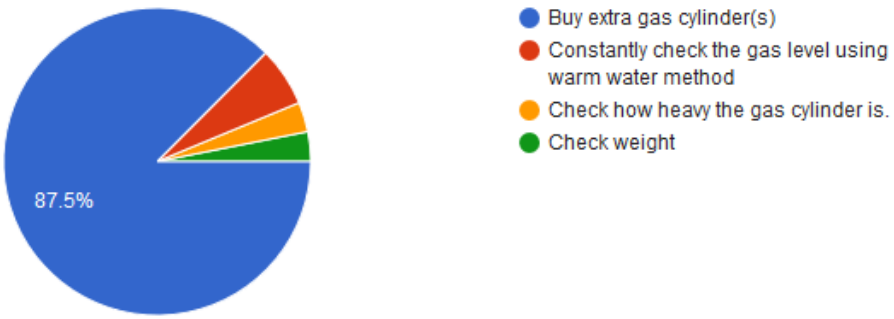


Figure 3-13. Pie *Chart of responses from question 5*

Based on the responses, most of the respondents buy extra LPG tank(s) to overcome the problem on the previous question. Some of them do check their LPG level using the warm water method and check the weight of the LPG tank.

Question 6: Will it be beneficial for you if there exists a device/system that can help you to constantly measure your LPG level?

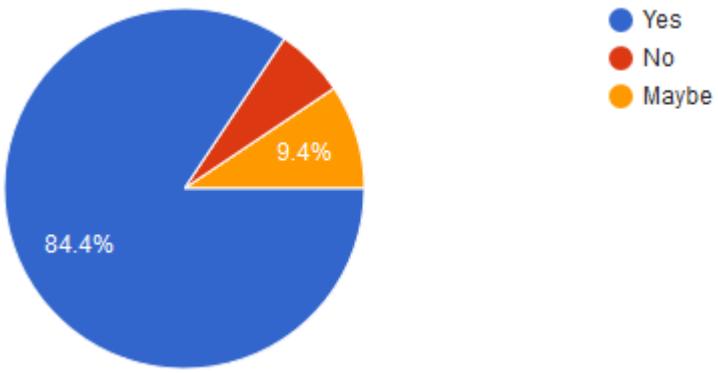


Figure 3-14. Pie Chart of responses from question 6

Based on the responses, it proves that most of the respondents find that it is beneficial for them if there exists a device/system that can help them to measure their LPG level regularly.

Question 7: The device/system should be able to notify when the LPG level is low (from Strongly disagree(1) to Strongly agree(5))

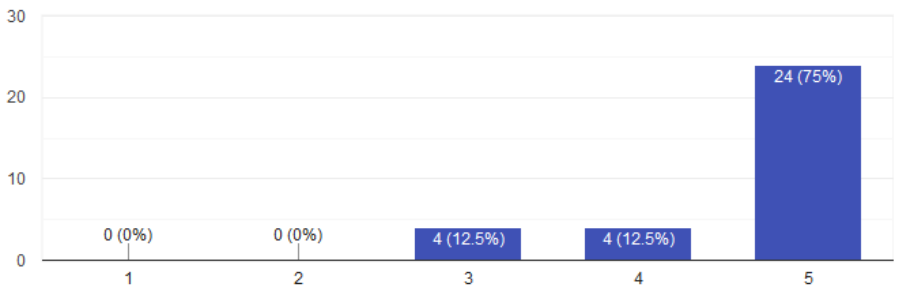


Figure 3-15. Bar graph of responses from question 7

Based on the responses, this shows that most of the respondents agree that the system should be able to notify them when their LPG level is low.

Question 8: A mobile application should be developed specifically for this device/system (from Strongly disagree(1) to Strongly agree(5))

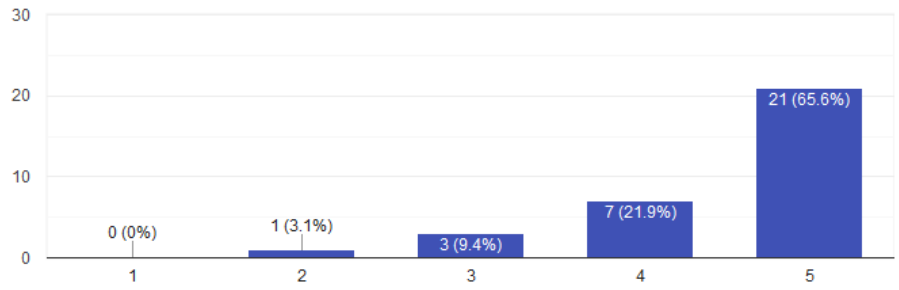


Figure 3-16. Bar graph of responses from question 8

Based on the responses obtained, most of the respondents want an application to be developed specifically for this system.

Question 9: The device/system should notify you through the application (from Strongly disagree(1) to Strongly agree(5))

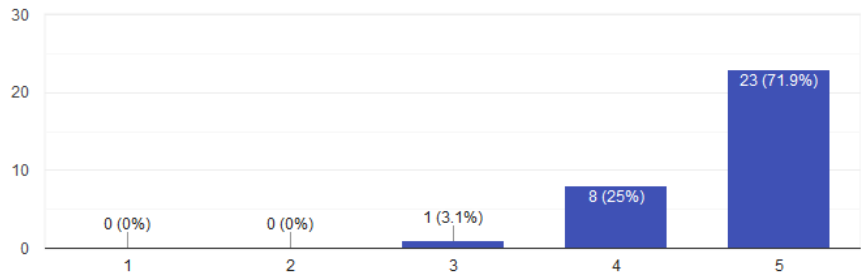


Figure 3-17. Bar graph of responses from question 9

Based on the responses, it shows that most of the respondents do agree that they should be notified through the application.

Question 10: The device/system can show your LPG tank(s) usage over the years through the application (from Strongly disagree(1) to Strongly agree(5))

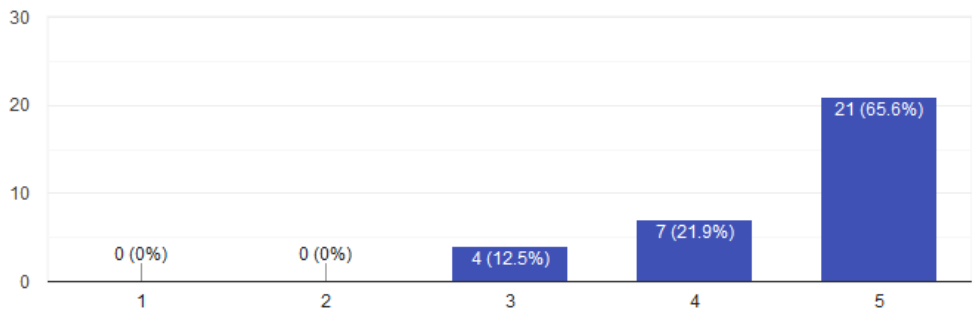


Figure 3-18. Bar graph of responses from question 10

Based on the responses, this shows that the respondents want the application to show them their usage over the years through the application.

3.3 – FUNCTIONAL REQUIREMENT

Below is the list of functional requirements of this proposed project:

1. User can set the threshold for push notification on the application.
2. User can view the LPG level in the LPG tank on the application.
3. User can view the LPG usage graph on the application.

3.4 – NON-FUNCTIONAL REQUIREMENT

Below is the list of non-functional requirements of this proposed project:

1. The system can measure the weight of the LPG tank every 5 minutes.
2. The system can send the reading to Firebase's Realtime Database after it measures the weight.
3. The system can send push notification to its user if the LPG level reaches a certain threshold.
4. The android application can fetch the reading from Firebase's Realtime Database.
5. The android application can generate a graph of LPG usage.

3.5 – SYSTEM DESIGN

This section will discuss in detail the data flow diagram (DFD), entity relationship diagram (ERD) and data dictionary used to design the system flows. According to Computer Hope (2017), DFD is used to present the data flow of each process and what are the inputs and outputs.

3.5.1 – BLOCK DIAGRAM

In this project, the weight of the LPG tank is measured using the load cell. The reading from the load cell is amplified using HX711 and sent to ESP8266. The ESP8266 then will send the reading to Firebase Realtime Database and will send a trigger to Firebase Push Notification service if needed. The readings from Realtime Database is displayed on Android application. Also, a push notification is sent to the application if the reading reaches a certain threshold.

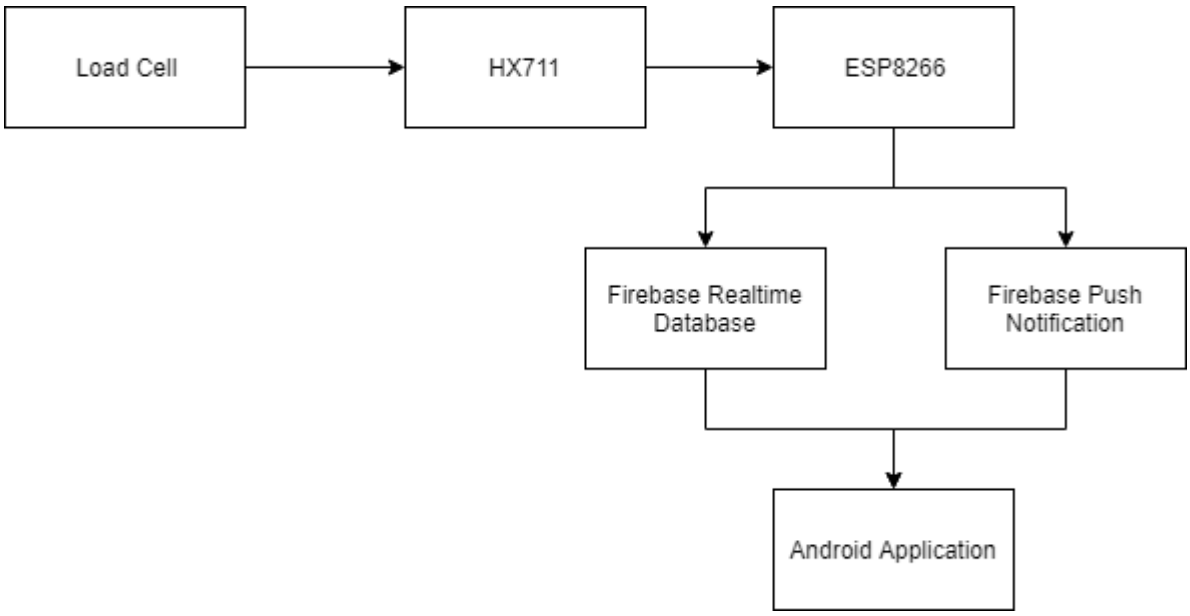


Figure 3-19. Block Diagram of the proposed project

3.5.2 – USE CASE DIAGRAM

In this system, the users can view the LPG usage and set the LPG threshold for the system to notify them. Besides, Firebase system will notify the users if the LPG level reaches the threshold set by them via push notification service. Firebase also stores the weight readings in its Realtime Database in the form of JSON structure. On the other hand, ESP8266 will measure the weight of the LPG tank and send the weight readings to Firebase.

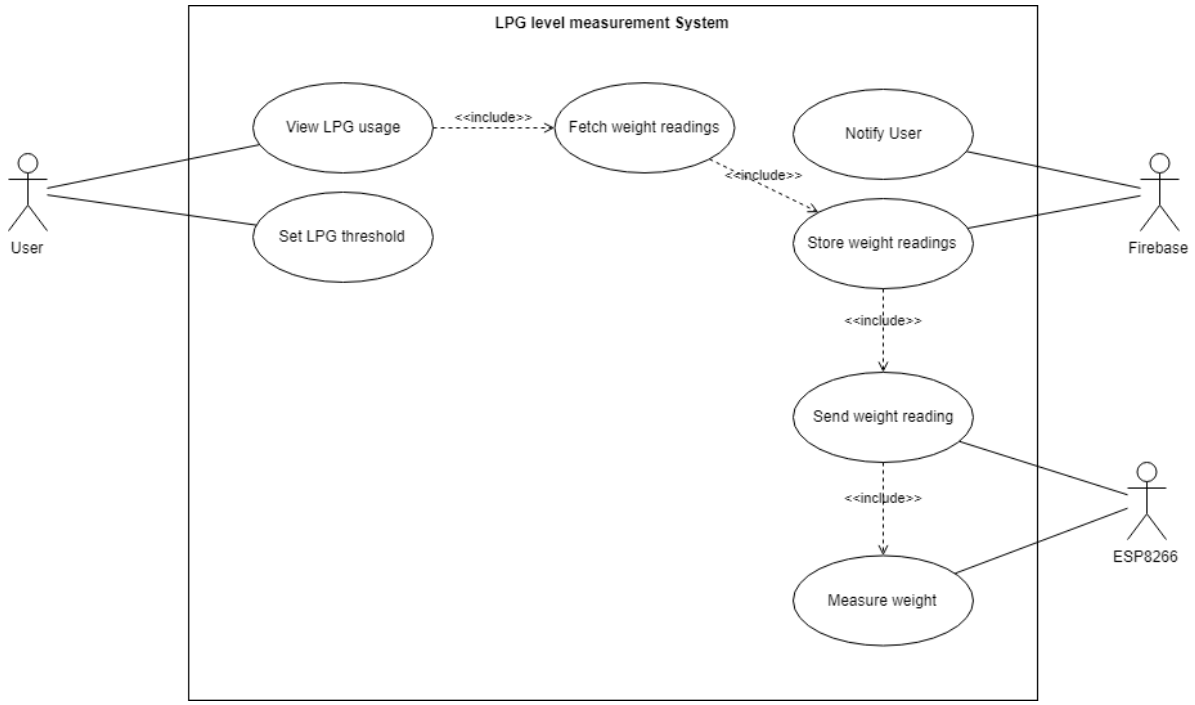


Figure 3-20. Use Case Diagram of the system

3.5.2.1 – View LPG Usage use case

Use Case	View LPG Usage
Short Description	The user views the LPG usage
Actors	User
Requirements	Fetch weight readings
Pre-Conditions	The readings need to be fetched from the Firebase's Realtime Database.
Post-Conditions	-
Main Flow	(1) The user chooses to view the LPG usage (2) The user chooses to view either monthly or weekly
Alternate Flow	-
Exception Flow	(E1) The reading in Firebase's Realtime Database is empty.

Table 3-3. View LPG Usage use case table

3.5.2.2 – Set LPG threshold use case

Use Case	Set LPG threshold
Short Description	The user sets the threshold for push notification.
Actors	User
Requirements	-
Pre-Conditions	-
Post-Conditions	The users get notified when the LPG level reaches the threshold that they set.
Main Flow	(1) The user chooses to set the threshold. (2) The user sets the threshold. (3) The user clicks on the save button.
Alternate Flow	-
Exception Flow	(E3) The threshold set is a negative value or not numerical.

Table 3-4. Set LPG threshold use case table

3.5.2.3 – Fetch weight readings use case

Use Case	Fetch weight readings
Short Description	The android app fetches weight readings from Firebase's Realtime Database.
Actors	-
Requirements	Store weight readings
Pre-Conditions	The application is connected to Firebase's Realtime Database via an Internet connection.
Post-Conditions	The readings are fetched and displayed on the android application.
Main Flow	(1) The android application is connected to Firebase's Realtime Database. (2) The android application fetches the weight readings from the database.
Alternate Flow	-
Exception Flow	(E2) The android phone is not connected to the Internet.

Table 3-5. Fetch weight readings use case table

3.5.2.4 – Notify User use case

Use Case	Notify User
Short Description	Firebase sends Push Notification service to the android application.
Actors	Firebase
Requirements	-
Pre-Conditions	The latest reading reaches the threshold set by the user.
Post-Conditions	The android application notifies the user.
Main Flow	(1) The current reading reaches the threshold set by the user. (2) The Firebase push notification service sends a push notification to the android application. (3) The user gets notified by the application.
Alternate Flow	-
Exception Flow	(E2) The reading does not reach the threshold set. (E3) The Firebase push notification failed to send push notification service.

Table 3-6. Notify User use case table

3.5.2.5 – Store weight readings use case

Use Case	Store weight readings
Short Description	The readings are stored into the Firebase's Realtime Database
Actors	Firebase
Requirements	Send weight reading
Pre-Conditions	The weight readings are sent from ESP8266.
Post-Conditions	The readings are stored into the database in JSON structure.
Main Flow	(1) ESP8266 sends weight reading. (2) Firebase Realtime Database store the reading.
Alternate Flow	-
Exception Flow	(E1) There is no internet connection.

Table 3-7. Store weight readings use case table

3.5.2.6 – Send weight readings use case

Use Case	Send weight reading
Short Description	The weight reading is sent from ESP8266 to Firebase Realtime Database
Actors	ESP8266
Requirements	Measure weight
Pre-Conditions	The weight reading is measured using the load cell.
Post-Conditions	The readings are sent to Firebase Realtime Database.
Main Flow	(1) The weight reading is measured. (2) The reading is sent to Firebase Realtime Database via internet connection.
Alternate Flow	-
Exception Flow	(E2) There is no internet connection.

Table 3-8. Send weight reading use case table

3.5.2.7 – Measure weight use case

Use Case	Measure weight
Short Description	The weight of the LPG tank is measured.
Actors	ESP8266
Requirements	-
Pre-Conditions	-
Post-Conditions	The weight of the LPG tank is measured.
Main Flow	(1) LPG tank's weight is measured using the load cell.
Alternate Flow	-
Exception Flow	-

Table 3-9. Measure weight use case table

3.5.3 – SEQUENCE DIAGRAM

Sequence diagram represents how a system communicate with one another to execute a task and the order of which interactions happens when a particular use case is carried out (Athuraliya, 2019). In this section, each use cases will have its sequence diagram.

3.5.3.1 – View LPG Usage

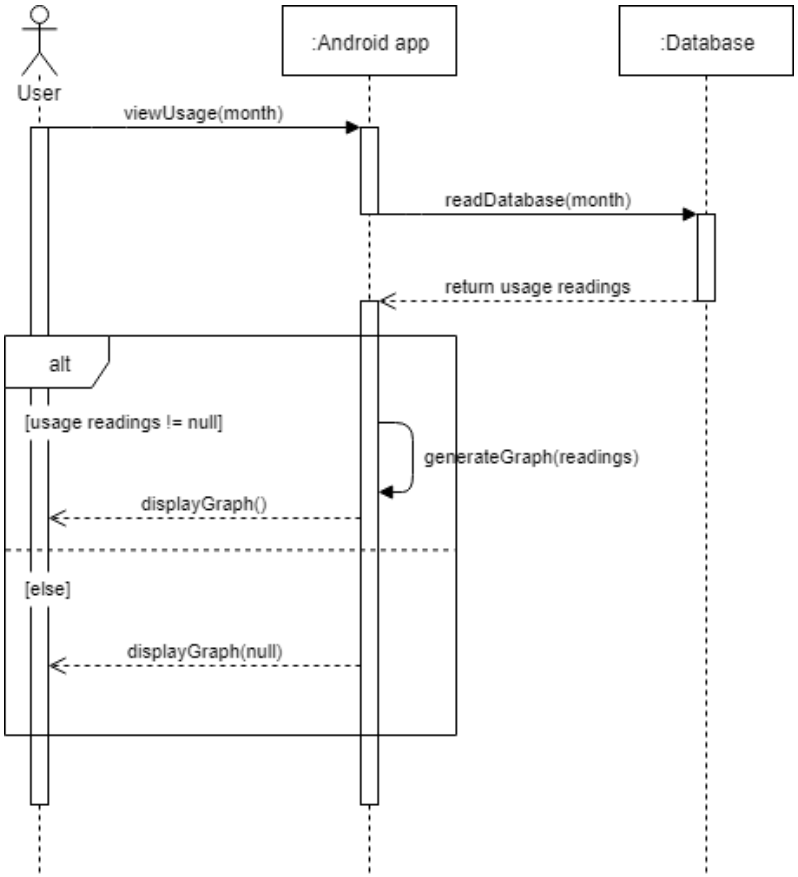


Figure 3-21. View LPG Usage sequence diagram

The user sends a request to view the LPG usage for a specified month. Next, the application will fetch the readings under the specified month from the database. The readings fetched is then used to generate a graph and displayed to the user.

3.5.3.2 – Set LPG threshold

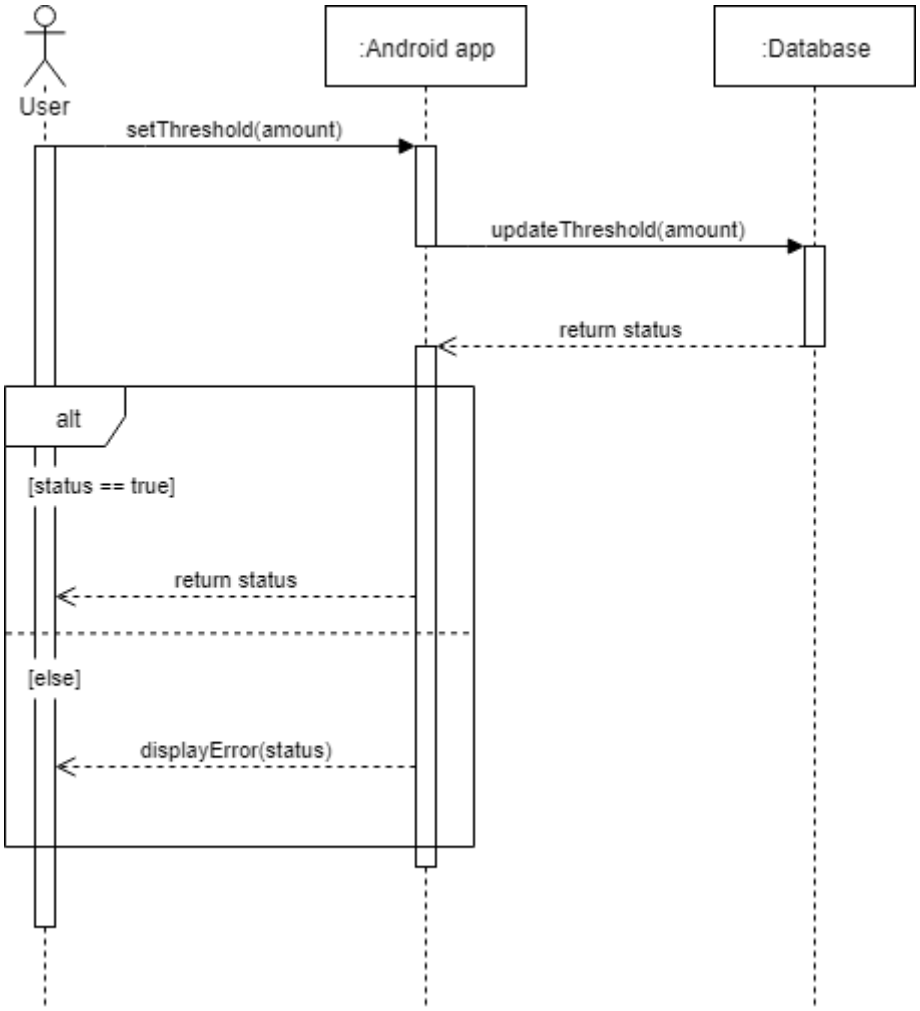


Figure 3-22. Set LPG threshold sequence diagram

The user sets the threshold. The amount set is sent to the database. The database returns the update status. If the status is true, then it will return the status normally. Else, it displays the error message together with the status code.

3.5.3.3 – Fetch weight readings

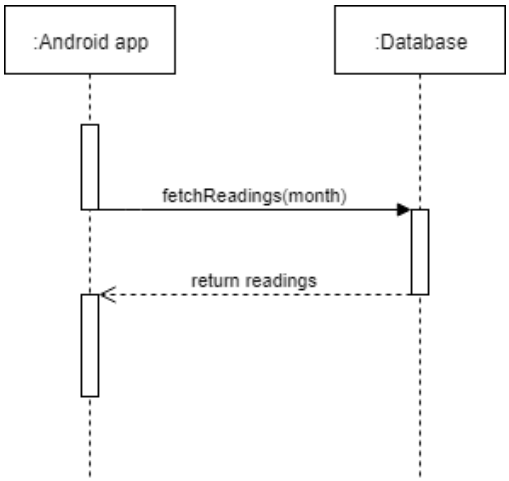


Figure 3-23. Fetch weight readings sequence diagram

The android app fetches weight readings from the database. The database then returns the readings to the application.

3.5.3.4 – Notify users

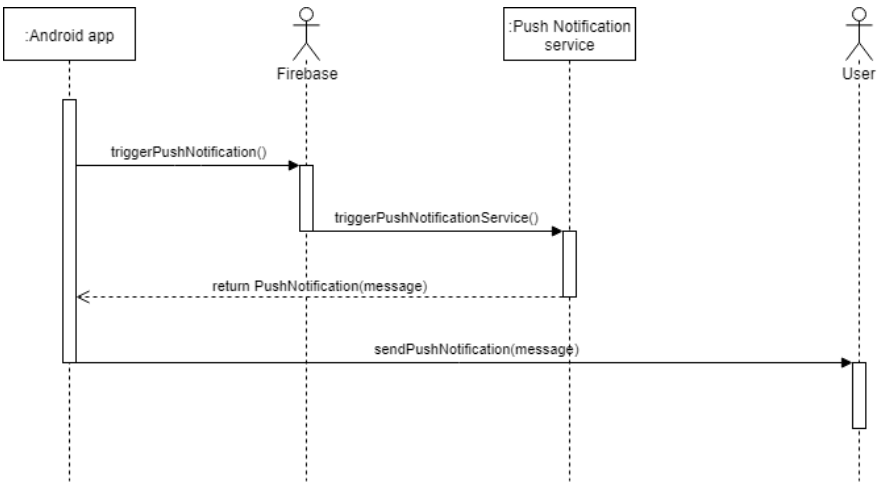


Figure 3-24. Notify users sequence diagram

The android application triggers push notification by sending the signal to Firebase. Then, Firebase triggers a push notification service. The push notification service then returns push notification to the application. The application then notifies the user.

3.5.3.5 – Store weight readings

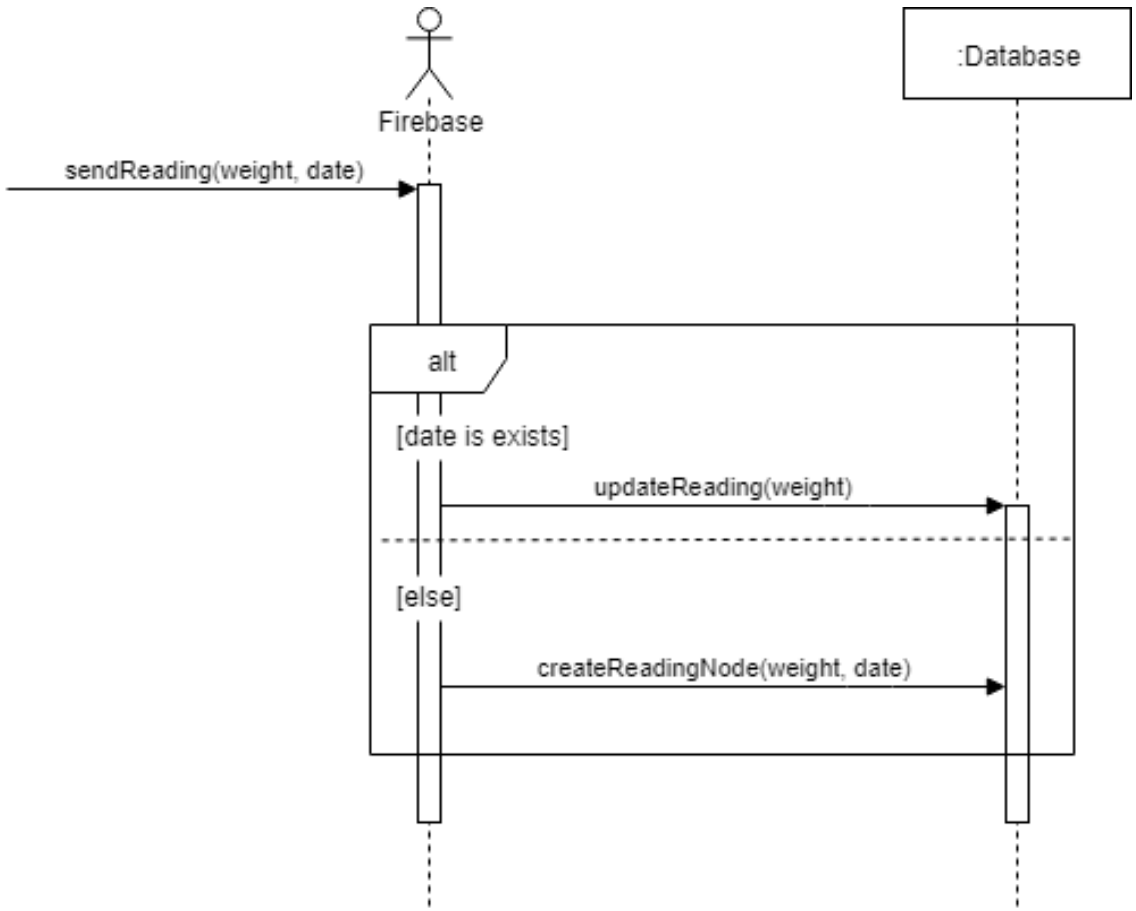


Figure 3-25. Store weight readings sequence diagram

When Firebase receives reading from ESP8266, it will check whether the date exists or not. If it exists, it will update the reading under the same date. Else, it will create a new node under the readings node in the database.

3.5.3.6 – Send weight readings

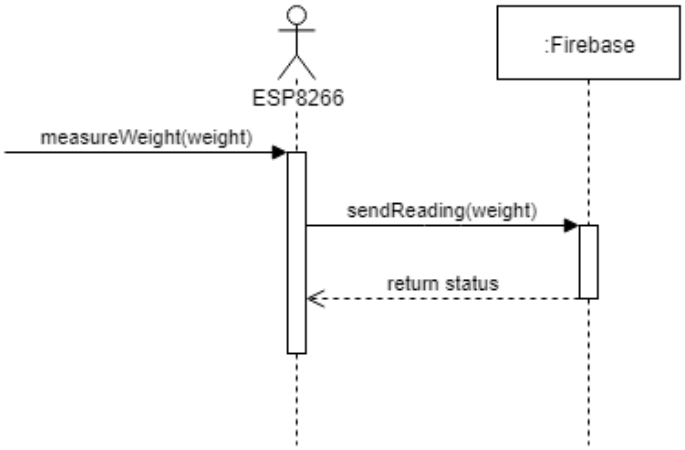


Figure 3-26. Send weight readings sequence diagram

When ESP8266 receives the amplified weight reading from HX711, it will send the reading to Firebase. Then, Firebase returns the status.

3.5.3.7 – Measure weight

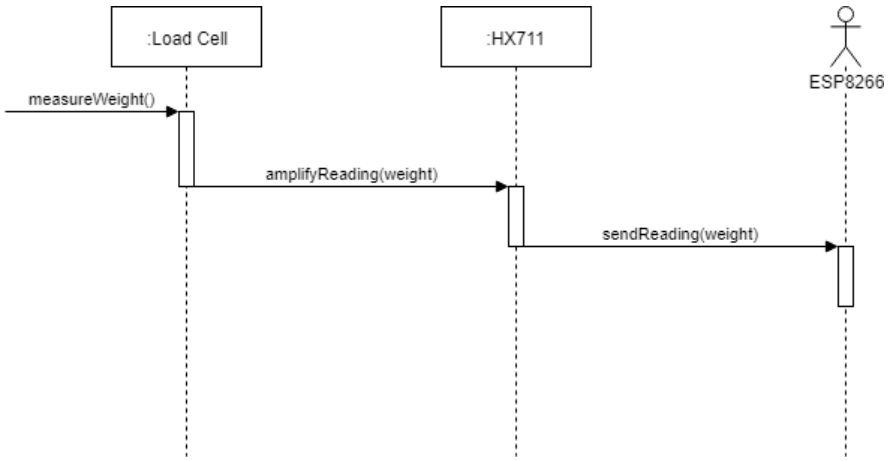


Figure 3-27. Measure weight sequence diagram

After load cell measures the weight, it sends the reading to HX711. Then, HX711 amplifies the reading and send it to ESP8266.

3.5.4 – CONTEXT DIAGRAM

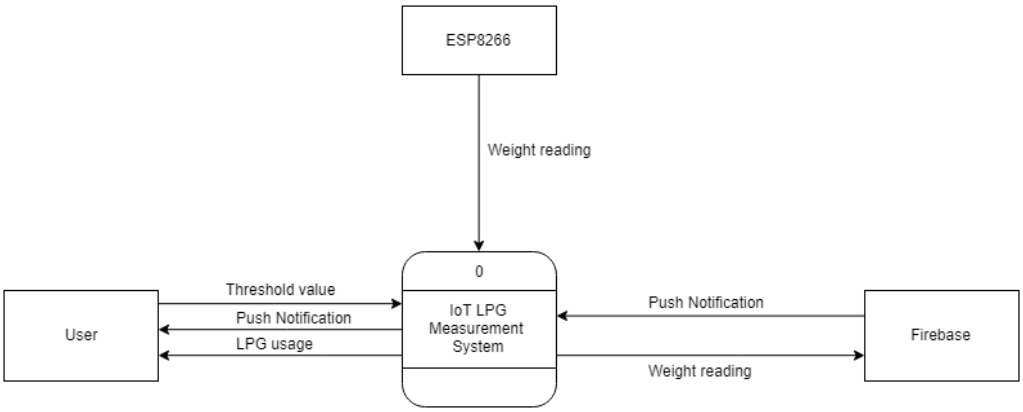


Figure 3-28. Context Diagram of the proposed project

3.5.5 – ACTIVITY DIAGRAM

3.5.5.1 – ESP8266 Side

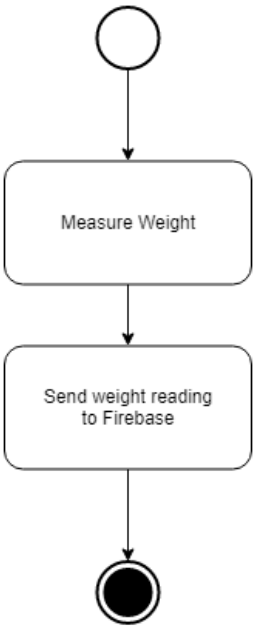


Figure 3-29. Activity Diagram on ESP8266 side

ESP8266 measures the weight. Then it sends the weight reading to Firebase.

3.5.5.2 – Firebase Side

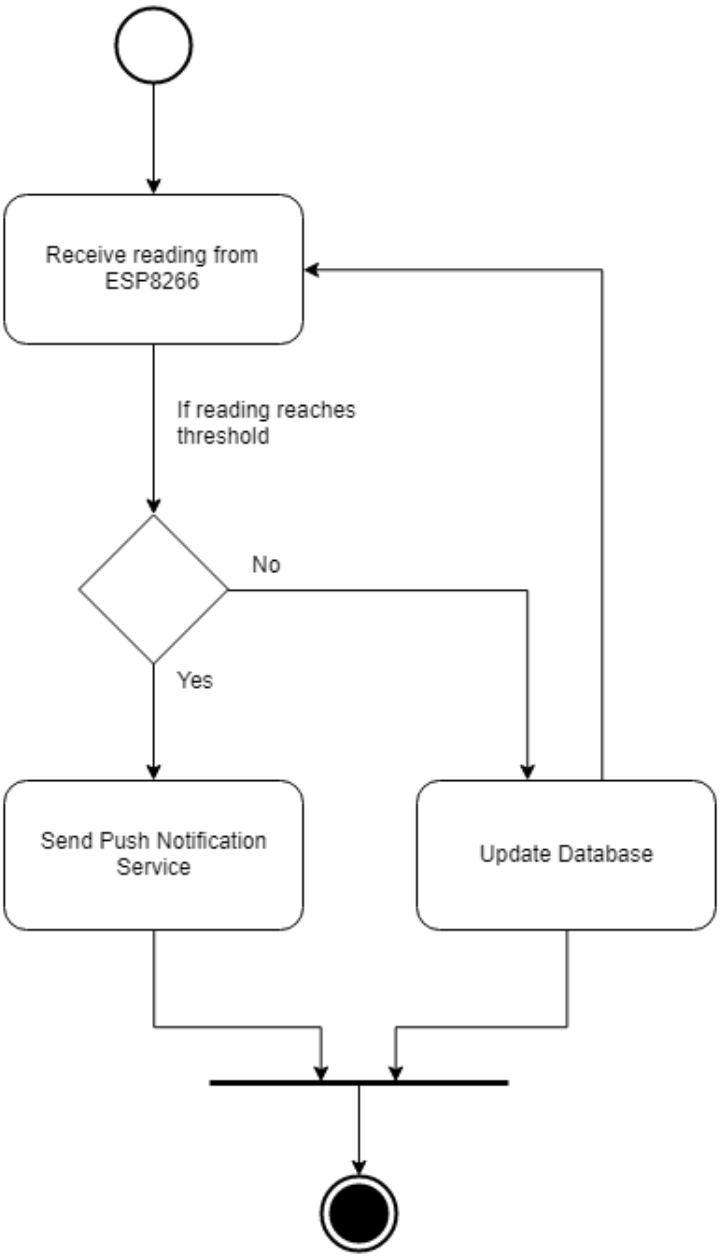


Figure 3-30. Activity Diagram on Firebase side

When Firebase receives reading from ESP8266, it will check if it reaches the threshold. If it reaches the threshold, it will send a push notification service. Else, it will update the database and recheck if it receives any reading from ESP8266.

3.5.5.3 – Application Side

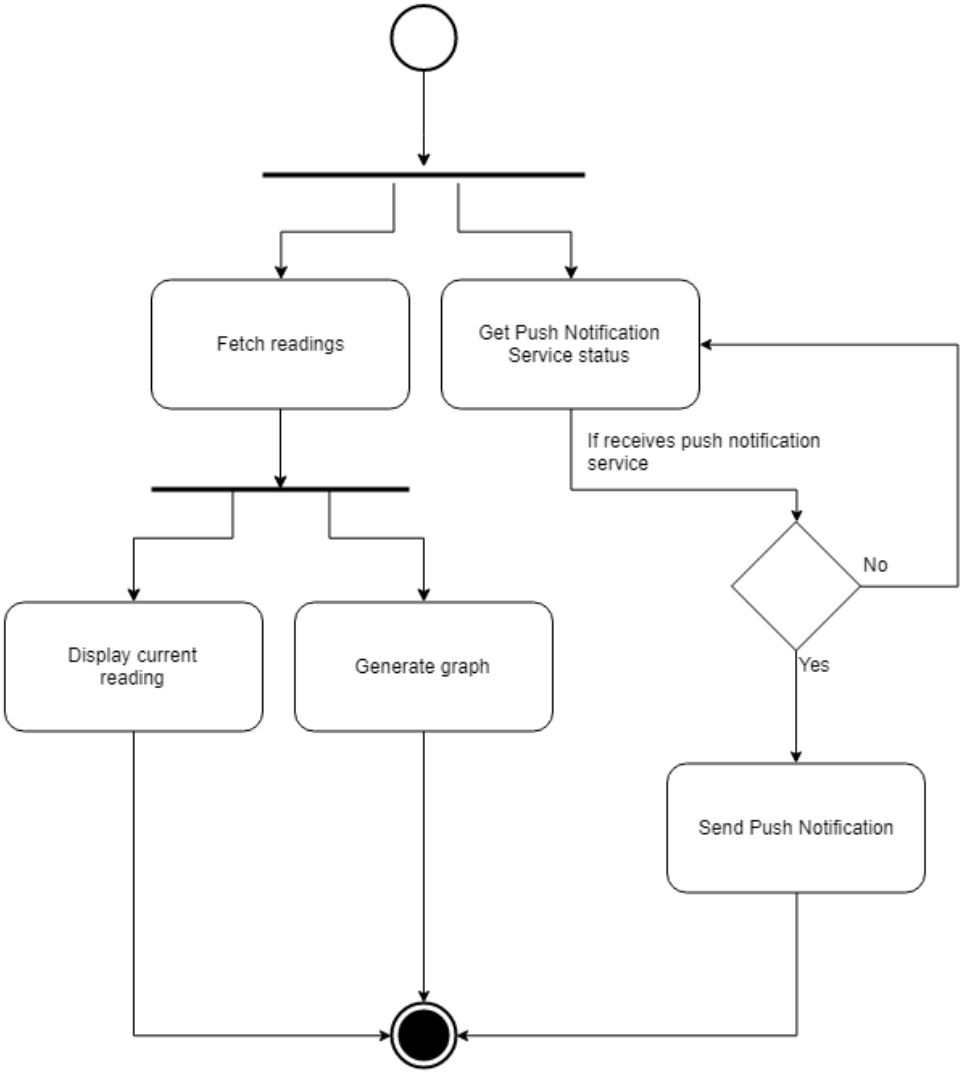


Figure 3-31. Activity Diagram on Application side

On the application, two processes will run simulatenously which are fetching readings from the database and get push notification status from Firebase. For push notification, if the status is true, it will send push notification. Else, it will recheck for the status. As for fetching readings, it will display the current reading and generate a graph based on the readings obtained from the database.

3.5.6 – ENTITY RELATIONSHIP DIAGRAM (ERD)

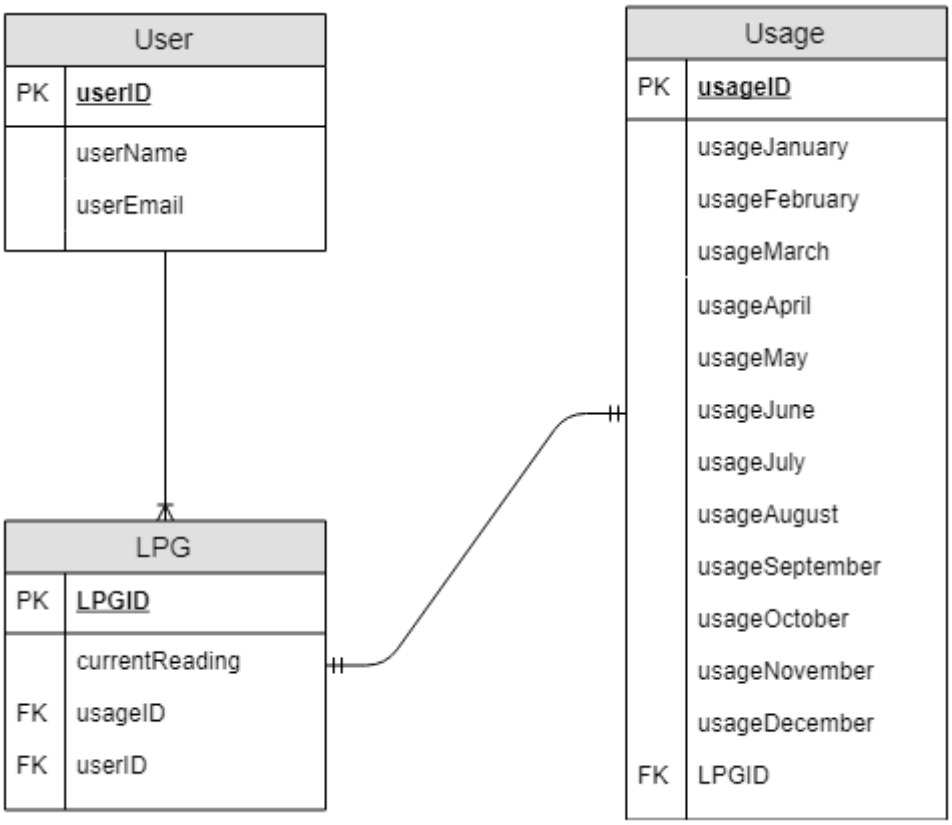


Figure 3-32. ERD of the proposed project

Table "User" is used to store the users' information. It consists of userID that is used to ensure each user has their unique ID, username and userEmail. As for table "LPG", it is used to store the LPGs' information. It consists of LPGID as primary key, currentReading, usageID and userID as the foreign key. The two foreign keys are used to determine who is the owner of the LPG and what is its usage. Lastly, for "Usage" table, it has several attributes to store the LPG usage over the year. It consists of usageID as its primary key, attribute usageJanuary to usageDecember, and LPGID as its foreign key.

3.6 – DATABASE DESIGN

Table 'User'

Field Name	Data Type	Data Format	Field Size	Description
userID	INT	AUTO_INCREMENT	5	Stores ID of the user
userName	VARCHAR	-	50	Stores the user's name
userEmail	VARCHAR	-	50	Stores the user's email

Table 3-10. Data dictionary for table 'User'

Table 'LPG'

Field Name	Data Type	Data Format	Field Size	Description
LPGID	INT	AUTO_INCREMENT	5	Stores the LPG ID
currentReading	INT	-	3	Stores the current percentage of LPG level
usageID	INT	AUTO_INCREMENT	5	Stores the usage ID
userID	INT	AUTO_INCREMENT	5	Stores the user ID

Table 3-11. Data dictionary for table 'LPG'

Table 'Usage'

Field Name	Data Type	Data Format	Field Size	Description
usageID	INT	AUTO_INCREMENT	5	Stores the usage ID
usageJanuary	INT	-	3	Stores the average usage of January
usageFebruary	INT	-	3	Stores the average usage of February
usageMarch	INT	-	3	Stores the average usage of March
usageApril	INT	-	3	Stores the average usage of April

usageMay	INT	-	3	Stores the average usage of May
usageJune	INT	-	3	Stores the average usage of June
usageJuly	INT	-	3	Stores the average usage of July
usageAugust	INT	-	3	Stores the average usage of August
usageSeptember	INT	-	3	Stores the average usage of September
usageOctober	INT	-	3	Stores the average usage of October
usageNovember	INT	-	3	Stores the average usage of November
usageDecember	INT	-	3	Stores the average usage of December
LPGID	INT	AUTO_INCREMENT	5	Stores the LPG ID

Table 3-12. Data dictionary for table 'Usage'

3.7 – APPLICATION INTERFACE & DEVICE PROTOTYPE DESIGN

3.7.1 – ANDROID APPLICATION INTERFACE DESIGN

3.7.1.1 – Home Screen

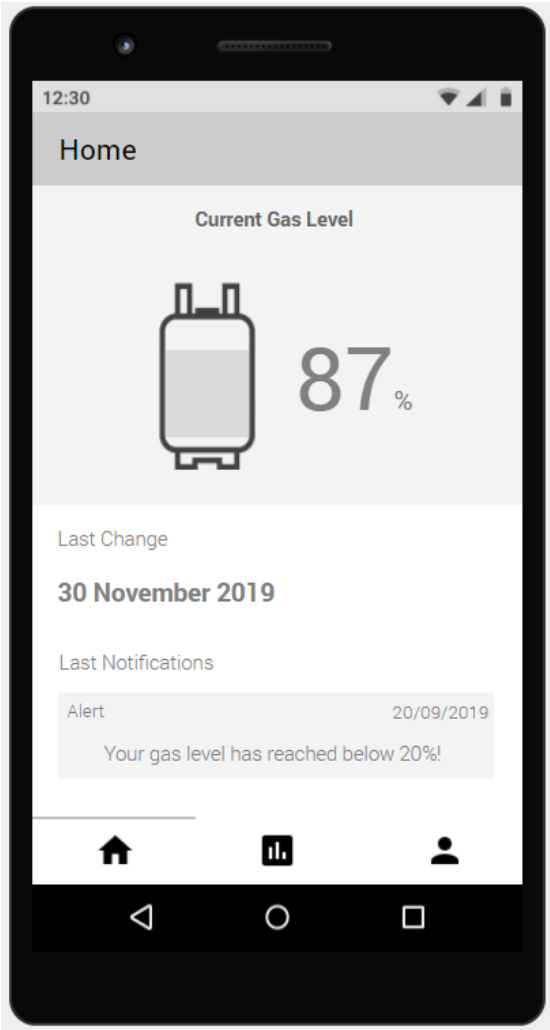


Figure 3-33. Interface design of the Home screen

Figure 3.7.1.1 shows the interface design of the application's home screen. In this screen, users can check their current LPG level in percentage. Users also can check when did they replace their LPG tank. Moreover, users can also check what is the latest notification they get.

3.7.1.2 – Usage Screen

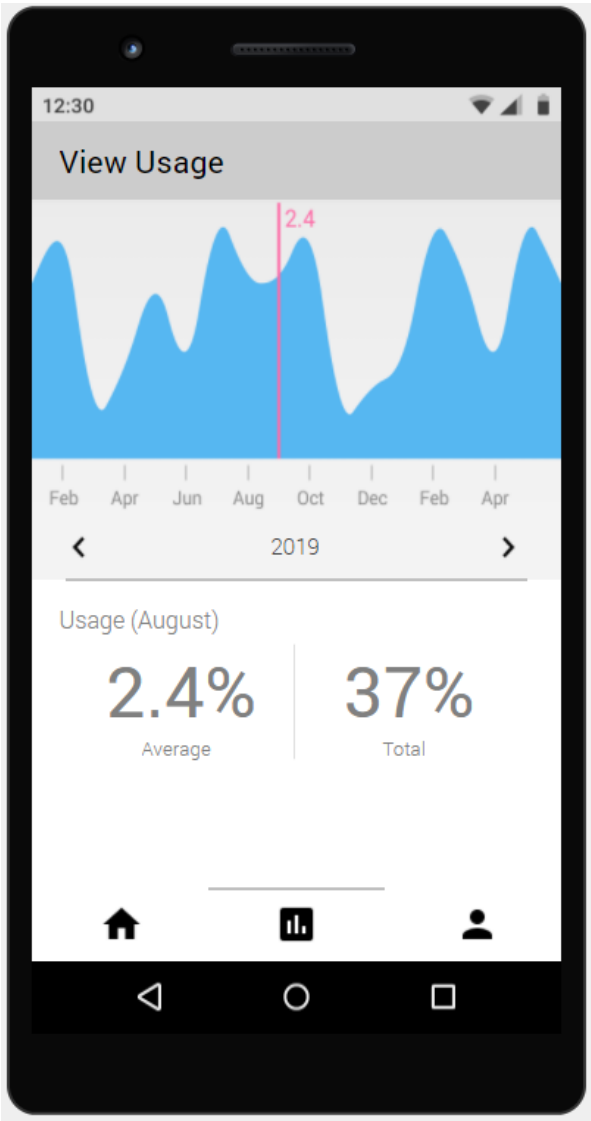


Figure 3-34. Interface design of the Usage screen

In this screen, users can check their LPG usage throughout the year. The readings obtained from the database is used to generate a graph to make it easier for them to see their LPG usage.

3.7.2 – DEVICE PROTOTYPE DESIGN

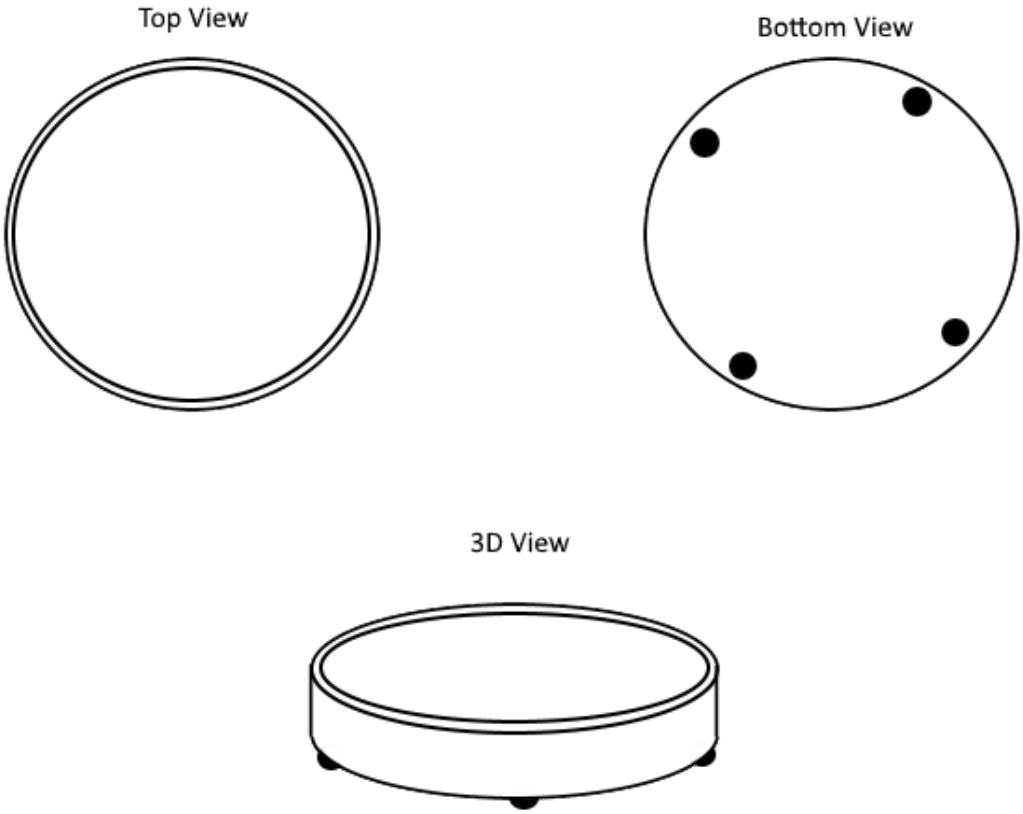


Figure 3-35. The device prototype design

The figure above shows the prototype design of the device used to measure the LPG level. The LPG tank is placed on top of the device.

3.8 – METHODOLOGY

Throughout this project, Rapid Application Development (RAD) methodology is used to develop the LPG level measurement device and the Android app. Below is the diagram of the process of RAD.

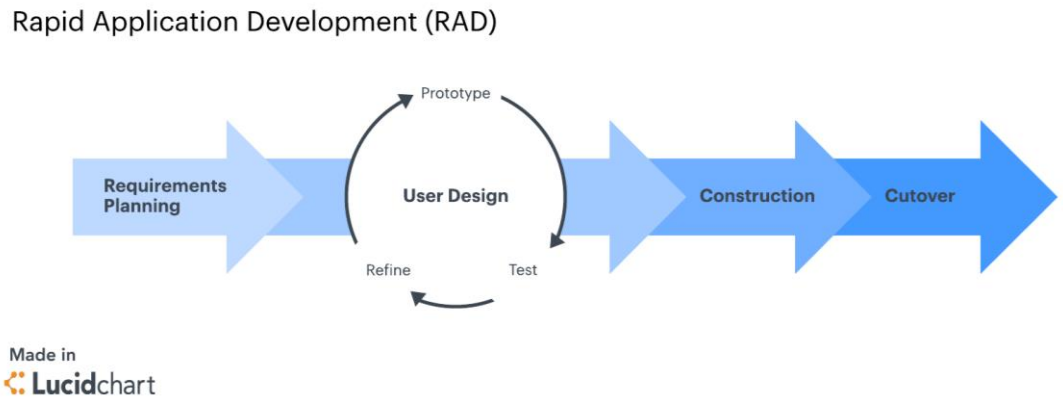


Figure 3-36. Rapid Application Development (RAD) diagram. *Reprinted from* 4 Phases of Rapid Application Development Methodology, *by Lucidchart Content Team, 2018, Retrieved from https://www.lucidchart.com/blog/rapid-application-development-methodology. Copyright 2018 by Lucidchart. Reprinted with permission.*

3.8.1 – REQUIREMENTS PLANNING

At the beginning of the project, the requirements elicitation begins with a thorough investigation of the current problems faced by people nowadays that can be solved by using IoT devices. In this development-based project, the requirements questions will be through an online form filled by users to determine whether the proposed project is practical or not. The feedbacks and answers are recorded and documented.

Next, the technologies used to develop the proposed project are determined. The technologies and tools used in each stage will be different. The estimated time to complete the proposed project is determined, that is two semesters of the course.

3.8.2 – USER DESIGN

After the requirements have been investigated, the project proceeded to the development phase. During this phase, several prototype iterations are conducted to build out the user design and determine whether it meets the requirements. These iterations will be repeated until all the requirements are met.

3.8.2.1 – Prototype

In this phase, the prototype device to measure the LPG level is developed according to the requirements defined on the current iteration.

3.8.2.2 – Test

In this phase, the prototype device is tested with users. User acceptance testing is conducted to gather feedbacks and to determine whether it meets the requirement defined. New requirements and bugs are analysed and discussed whether it needs to be implemented and fixed on the next iteration. User acceptance testing is also conducted on the last iteration to ensure that all the requirements are met.

3.8.2.3 – Refine

Based on the feedback results acquired from the user acceptance testing, the prototype is modified and improved according to what needs to be worked on and fixed.

3.8.3 – CONSTRUCTION

Based on the requirements and feedbacks acquired after several iterations from the user design phase, a final version of the working model is developed. It is easier to construct a final working model in this phase as the issues and modifications have been addressed during the thorough iterative user design phase. Some modifications and bug fixes are also made during this phase if needed.

3.8.4 – CUTOVER

In this phase, the completed product is released. Also, maintenance is conducted to ensure the reliability of the system. Moreover, training users and porting the old system to a new one is also conducted.

3.9 – CONCLUSION

In conclusion, the detailed methodology of developing the proposed project is discussed in this chapter. Rapid Application Development methodology is used in this project to ensure that the device developed meets the requirements on every iteration.

CHAPTER 4 – IMPLEMENTATION

4.1 - INTRODUCTION

This chapter discusses in detail the implementation process of this project. There are three subsections will be discussed in this chapter. Those three subsections are the IoT device, REST API, and Android application.

4.2 – IOT DEVICE

The IoT device is used to measure weight of the LPG tank and based on the weight measured, an estimated percentage value of LPG level is calculated. There are three main components used to develop the device. Those are 10kg straight bar load cell, HX711 load cell amplifier, and NodeMCU V3 ESP8266 12-E.

4.2.1 – THE WIRING CONNECTIONS

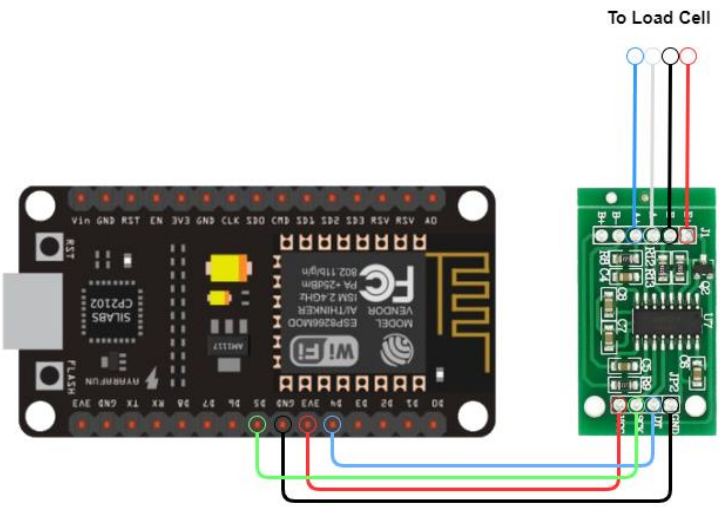


Figure 4-1. The wiring connections between ESP8266, HX711, and load cell

In HX711, there are four pins will be connected to ESP8266. Those pins are ground pin (GND), data pin (DT), system clock pin (SCK), and voltage in pin (VCC). These four pins are connected to ESP8266's GND pin, D4 pin, D5 pin, and 3.3V pin, respectively. On the other side of HX711 has six pins which are E+, E-, A-, A+, B-, and B+ pins. In this project, only four of those pins will be used since the type of load cell used requires only those four pins. Usually straight bar load cell comes with four wires labelled with different colours (black, red, white, and blue). These four wires are connected to the HX711's E+, E-, A-, and A+ according to the figure above.

4.2.2 – THE DEVICE CASING (ENCLOSURE) & LOAD CELL STAND

To protect the electronic components from getting in contact with water or substances that could damage the components, a casing for the components is designed and 3D printed. Besides, the load cell needs a stand for it to work.

4.2.2.1 – DEVICE CASING

The device casing is designed as hollow cylinder with an open top. The base is 4mm thick to ensure the screw that is being used does not pass through the casing when the parts are being screwed. The diameter of the case is 154mm and the cylinder wall thickness is 2mm.

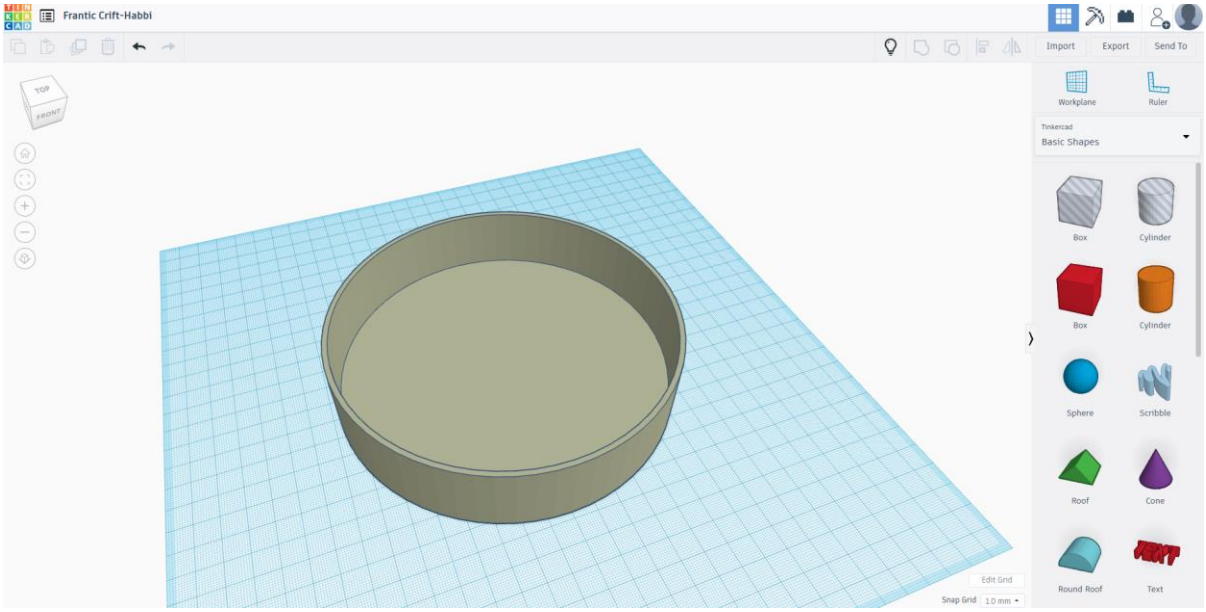


Figure 4-2. The casing designs

4.2.2.2 – Load Cell stand

Below is the design for load cell stand. The stand is designed with four M2.5 screw holes to screw the stand and casing together. The two M3 screw holes on top of the stand on the other hand is for screwing load cell together with the stand.

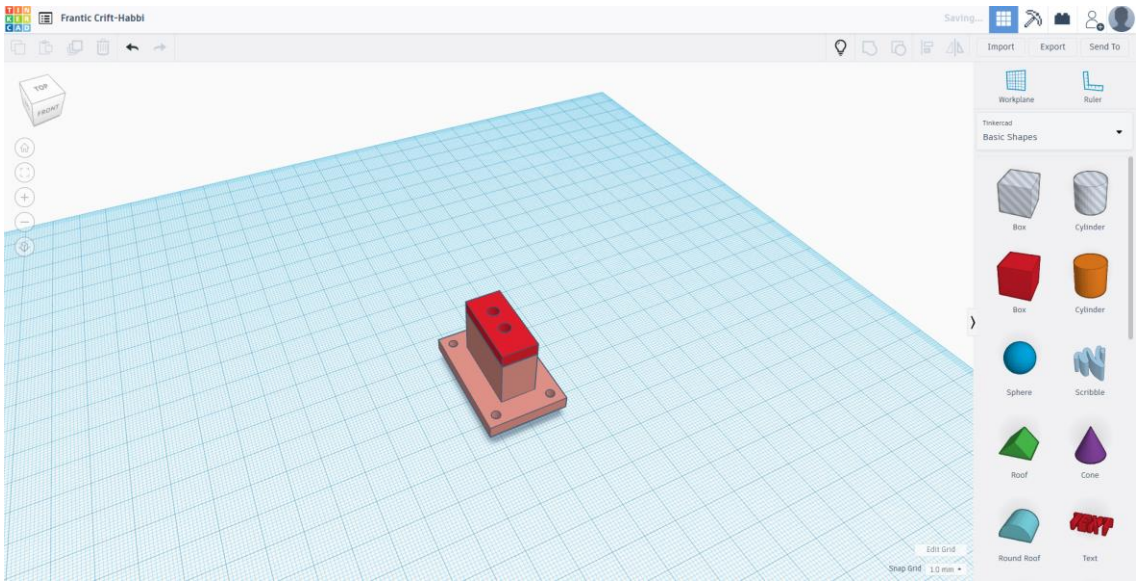


Figure 4-3. Load Cell stand

Figure below shows how the case and stand look like when it has been attached together.

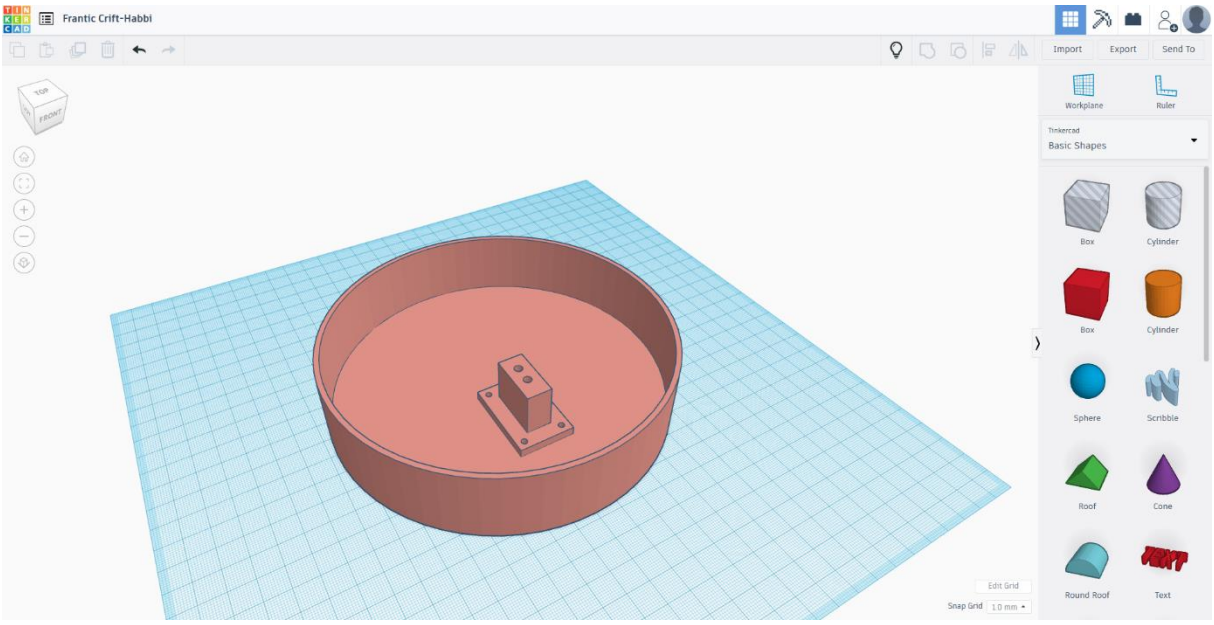


Figure 4-4. When casing and stand are screwed together

Figure 4-5. How the device looks like after assembled

Figure 4-6. Device with cover on top

4.2.3 – CODING

In this project, there are four libraries used. One of them is Firebase ESP8266 Client by Mobizt. This library is used to connect ESP8266 module with Firebase Realtime Database and Cloud Messaging service. The second library used is HX711_ADC by Olaf Kallhovd for integrating a 24-bit ADC HX711 with load cell. The last two libraries used are ArduinoHttpClient by Arduino and ArduinoJson by Benoit Blanchon. These two libraries are used to make a HTTP request to PHP REST API and parsing the return content when needed.

4.2.3.1 – Settings

```
//Load Cell & Gas Cylinder settings
const float calibrationFactor = 696.0;
const float fullWeight = 23.0;
const float emptyWeight = 2.0;
const float contentPerPercent = (fullWeight - emptyWeight)/100;
```

Figure 4-7. Device code snippet #1

These variables are used to set the load cell up. The full weight of the prototype LPG tank and when it is empty needs to be determined first and declared in the variables. The amount of fuel per one percent is calculated using formula:

$$contentPerPercent = \frac{fullWeight - emptyWeight}{100}$$

Equation 4-1. Formula of contentPerPercent

```
//Device settings
int thresholdPercentage = 10;
int tickerTime = 300; //300 seconds = 5 minutes
```

Figure 4-8. Device code snippet #2

Figure above shows the device default settings. There are two of them. These two variables will change after the device successfully fetched its settings through the PHP REST API. The thresholdPercentage refers to the percentage value of when should the device send push notification when the fuel reaches that thresholdPercentage. In this case, the default value of it is 10%. 'tickerTime' on the other hand refers to the amount of time that the device should send the readings to Firebase. In this case, the value is 300 seconds which is equivalent to 5 minutes. The device will send its reading for every 5 minutes.

4.2.3.2 – HTTP POST Request function

```
String httpPOSTRequest(String url, String params) {
  HTTPClient http;

  http.begin(url, API_URL_FINGERPRINT);
  http.addHeader("Content-Type", "application/x-www-form-urlencoded");

  int httpResponse = http.POST(params);
  String payload = "{}";

  if (httpResponse > 0) {
    Serial.println("[HTTP] Successfully executed POST request!");
    Serial.print("[HTTP] Response Code: ");
    Serial.println(httpResponse);
    Serial.print("[HTTP] URL: ");
    Serial.println(url);
    payload = http.getString();
  }
  else {
    Serial.print("[HTTP] Error: ");
    Serial.println(http.errorToString(httpResponse));
  }

  http.end();
  return payload;
}
```

Figure 4-9. Device code snippet #3

The function above is used to send HTTP POST request to a specified server URL. It takes two parameters which are the URL and the parameter(s) that wants to be passed. The function will return string payload which it received from the PHP REST API return content.

4.2.3.3 – Send Reading function

```
void sendReading() {
  if (WiFi.status() != WL_CONNECTED) {
    Serial.println("Error sending reading! Reason: Wifi disconnected!");
  }
  else {
    Serial.println("Sending reading to MySQL database...");

    String readingPercentage = "readingValue=" + String(readingVal) + "&device
ID=" + BOARD_ID;
    String reqResponse = httpPOSTRequest(DEVICE_SEND_READINGS, readingPercenta
ge);

    size_t capacity = JSON_OBJECT_SIZE(2) + 50;
    DynamicJsonDocument doc(capacity);
    deserializeJson(doc, reqResponse);

    String message = doc["message"];
    Serial.println(message);

    Serial.println();
    Serial.println("Sending reading to Firebase Realtime database...");
    String path = BOARD_ID + "/";
    if (Firebase.setInt(firebaseData, path + "gas_level", readingVal)) {
      Serial.println("Successfully sent reading to Firebase.");
    }
    else {
      Serial.println("Failed to send reading to Firebase!");
      Serial.println("Reason: " + firebaseData.errorReason());
    }
  }
}
```

Figure 4-10. Device code snippet #4

This function runs when every 'tickerTime' seconds. It sends HTTP POST request to MySQL database server to log the readings. Apart from that, it also sends the reading to Firebase Realtime Database.

4.2.3.4 – Send Push Notification

```
void sendPushNotification() {
  if (WiFi.status() != WL_CONNECTED) {
    Serial.println("Error sending reading! Reason: Wifi disconnected!");
  }
  else {
    firebaseData.fcm.setNotifyMessage("LPG IoT", "Your gas level has reached b
elow " + String(thresholdPercentage) + "%");

    if (Firebase.sendMessage(firebaseData, 0)) {
      Serial.println(firebaseData.fcm.getSendResult());
    }
    else {
      Serial.println(firebaseData.errorReason());
    }
  }
}
```

Figure 4-11. Device code snippet #5

This function is executed when the reading reaches 'thresholdPercentage'. It sends push notification to Firebase Cloud Messaging service.

4.2.3.5 – Setup function

In setup function, the device will fetch its settings from MySQL database through PHP REST API. The settings are thresholdPercentage and tickerTime. If it fails to fetch the reading, it will use the default value which is 10% for thresholdPercentage and 5 minutes for tickerTime.

```
Serial.println("[Settings] Fetching settings from online database..");
String boardID = "boardID=" + BOARD_ID;
String reqResponse = httpPOSTRequest(DEVICE_GET_SETTINGS, boardID);

size_t capacity = JSON_ARRAY_SIZE(2) + JSON_OBJECT_SIZE(3) + 110;
DynamicJsonDocument doc(capacity);
deserializeJson(doc, reqResponse);

String message = doc["message"];
thresholdPercentage = doc["deviceSettings"]["thresholdPercentage"];
tickerTime = doc["deviceSettings"]["tickerTime"];
Serial.print("[Settings] ");
Serial.println(message);
Serial.print("[Settings] Threshold Percentage (%): ");
Serial.println(thresholdPercentage);
Serial.print("[Settings] Ticker Time (s): ");
Serial.println(tickerTime);
```

Figure 4-12. Device code snippet #6

The response get from the POST request is in a form of JSON. Below is how the response looks like:


```
{
    "error": false,
    "message": "Successfully fetched device01 settings",
    "deviceSettings": {
        "thresholdPercentage": 15,
        "tickerTime": 600
    }
}
```


Figure 4-13. Get device settings API response

The response is then de-serialized using one of ArduinoJson library's function, 'deserializeJson()'. The value of 'thresholdPercentage' and 'tickerTime' in the device is set to the one received from the response. In this case, the value will be 15% for 'thresholdPercentage' and 600 seconds for 'tickerTime'.

```
ticker.attach(tickerTime, toggleSendReading);
```

Figure 4-14. Device code snippet #7

The last line of code in setup function is to attach a function toggleSendReading which will run for every 'tickerTime' seconds.

4.2.3.6 – Loop function

Under loop function, the device will keep on reading the load cell value and execute some functions when needed.

```
float i = LoadCell.getData();
Serial.print("[Load Cell] Output val: ");
Serial.print(i);

int roundedVal = round(i);
Serial.print("  Rounded val: ");
Serial.println(roundedVal);
```

Figure 4-15. Device code snippet #8

In figure above, the reading from load cell is round off to the nearest integer using *round()* function from 'math.h' header file.

```
if (roundedVal >= fullWeight) {
   readingVal = 100;
}
else if (roundedVal <= emptyWeight) {
   readingVal = 0;
}
else {
   readingVal = (roundedVal/contentPerPercent) - (emptyWeight/contentPerPercen
t);
}
```

Figure 4-16. Device code snippet #9

After the value is rounded off to its nearest integer, it will check if the value is more than the 'fullWeight', the 'readingVal' will stay 100 which means 100%. If it reaches below than the 'emptyWeight', it will be 0%. This is to ensure that the value given by 'readingVal' does not exceed 100% or below 0%. If it is in between empty and full weight, it will be calculated using a formula:

$$readingVal = \left(\frac{roundedVal}{contentPerPercent}\right) - \left(\frac{emptyWeight}{contentPerPercent}\right)$$

Equation 4-2. readingVal formula when the value is between fullWeight and emptyWeight

** Credit goes to my younger brother for helping me out to come up with this formula*

```
if (allowSendReading) {
   sendReading();
   previousVal = readingVal;
   allowSendReading = false;
}
```

Figure 4-17. Device code snippet #10

In this part, when the variable allowSendReading is true, it will send the reading by executing sendReading() function and the allowSendReading is set back to false after it has executed to ensure it will not keep on sending the reading later on.

4.3 – REST API

For the device and application to communicate with the MySQL database, they need to communicate via a Representational State Transfer Application Programming Interface (REST API). Since both device and application can execute HTTP request, therefore this REST API will use PHP as its language. The API will return its response in JSON encoded message.

There are three different PHP files used to develop this REST API. Those are *API.php*, *APIOperation.php*, and *Connect.php*. These three files have their own separate functionalities.

4.3.1 – CONNECT.PHP

```php
<?php

define('host', 'localhost');
define('user', '*****');
define('pass', '*****');
define('database', '*****');

class Connect {
    private $con;

    function __construct() { }

    function connect() {
        $this->con = new mysqli(host, user, pass, database);

        if (mysqli_connect_errno()) {
            echo "Failed to connect to MySQL: " . mysqli_connect_error();
        }
        return $this->con;
    }
}
```

Figure 4-18. Connect.php code snippet

This file is used to create a connection with database using specified credentials. It is written under a class and returns variable *$con* that will be used to process query functions.

4.3.2 – API.PHP

When the device and application call for REST API function, it will use the URL of where this file is kept. For example:

$$http://www.myfyp.com/API.php?apicall = apicallname$$

Before an API call is executed, function *checkParams()* will check if the request passed has enough parameters to execute the API call. If there is missing parameter(s), it will stop the API call process and return JSON encoded error message.

```
function checkParams($params) {

    $available = true;
    $missingparams = "";

    foreach($params as $param){
        if(!isset($_POST[$param]) || strlen($_POST[$param])<=0) {
            $available = false;
            $missingparams = $missingparams . ", " . $param;
        }
    }

    if(!$available) {
        $response = array();
        $response['error'] = true;
        $response['message'] = 'Parameters' . substr($missingparams, 1, strlen
($missingparams)) . ' missing';

        echo json_encode($response);
        die();
    }
}
```

Figure 4-19. API.php code snippet #1

When an API is called, it will check if the API call exists in the list of API calls. For example, if API call for *signin* is called, it will proceed to execute *signIn()* function under *APIOperation.php*.

```php
case 'signin':
     checkParams(array('userName', 'userPassword'));
     $db = new APIOperation();

      $result = $db->signIn(
          $_POST['userName'],
          $_POST['userPassword']
      );

      if ($result) {
         $response['error'] = false;
         $response['message'] = "Successfully signed in!";
         $response['user'] = $db->getUserDetails($_POST['userName']);
      }
      else {
         $response['error'] = true;
         $response['message'] = "Failed to sign in!";
      }
break;
```

Figure 4-20. API.php code snippet #2

Under *signin* API call, it will check if parameters *userName* and *userPassword* are exists or not when the API is called. Then, it will execute *signIn()* function under *APIOperation* class. The function will return Boolean value which refers to the status of executing the function. When it is true, it will execute another function, *getUserDetails()* to fetch the user details and save it under *user* in JSON.

```php
} else {
    $response['error'] = true;
    $response['message'] = 'Invalid API Call';
}

echo json_encode($response);
```

Figure 4-21. API.php code snippet #3

If an invalid API call is called, the API will return message with error state true. The responses are encoded into JSON using *json_encode()* function.

4.3.3 – APIOPERATION.PHP

The API calls use functions under *APIOperation.php*. For example, when the API call is *signin*, it will execute *signIn()* function and then *getUserDetails()* if the result from *signIn()* function is true.

```
function signIn($userName, $userPassword) {
    $stmt = $this->con-
>prepare("SELECT * FROM users WHERE userName=? AND userPassword=?");
    $userPassword = md5($userPassword);
    $stmt->bind_param("ss", $userName, $userPassword);
    $stmt->execute();
    $stmt->store_result();

    if ($stmt->num_rows == 1)
        return true;

    return false;
}
```

Figure 4-22. APIOperation.php code snippet #1

Under *signIn()* function, a query to database is done by using an SQL statement. In this case, it is used to look for a user account with given inputs of *userName* and *userPassword*. The SQL statement is executed using MySQLi Prepared Statement method. The result of executing the SQL statement is stored using *store_result()* function. Then it will check if the number of rows in the result is equal to one using *num_rows*. If it is equal to one, then the function will return true as a result, else it will return false.

```
function getUserDetails($userName) {
    $stmt = $this->con-
>prepare("SELECT userName, userEmail FROM users WHERE userName=?");
    $stmt->bind_param("s", $userName);
    $stmt->execute();

    $stmt->bind_result($userName, $userEmail);
    $stmt->fetch();

    $user = array(
        'userName'=>$userName,
        'userEmail'=>$userEmail
    );

    return $user;
}
```

Figure 4-23. APIOperation.php code snippet #2

As for *getUserDetails()* function, it fetches two columns from table *users*. The result is bind to variable *$userName* and *$userEmail* using *bind_result()* function. Then the result is fetched and stored in *$user* variable in a form of JSON array using *array()* function. Lastly, it returns the array *$user*.

4.3.4 – SEQUENCE DIAGRAM OF THE API

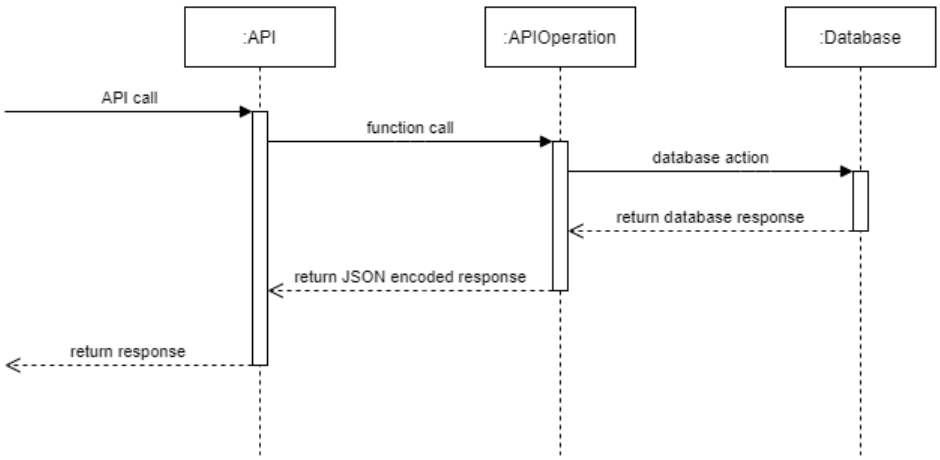


Figure 4-24. Overview of API

4.4 – ANDROID APPLICATION

The data collected and sent by the device is used to display in this application. This application has several sections which are Home, Usage and Profile. Each section has their own functionalities and will be discussed separately in their own subsections. There are also several utility classes used to execute the process. This subsection will discuss about several main sections for this application, the utility classes used, and the libraries used to develop this application.

4.4.1 – SIGN IN

At first, the users will be redirected to a sign in screen. The users need to register their account first to sign in. They can create an account by clicking the “Sign Up” text under the “Sign In” button.

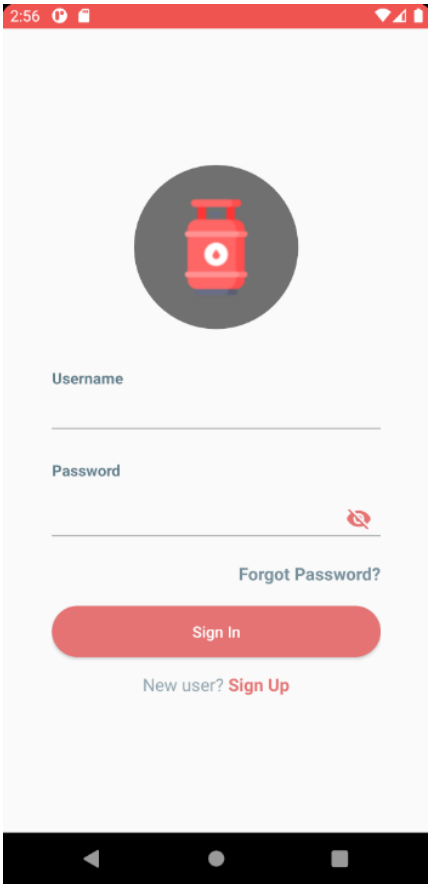


Figure 4-25. Sign In activity

Within this activity, the user needs to fill in his/her username and password to sign into the application. The username and password are then passed to a *signIn()* function to check whether the account credentials are correct.

```
public void signIn(final String userName, final String userPassword) {
    StringRequest stringRequest = new StringRequest(Request.Method.POST,
API.URL_SIGNIN, new Response.Listener<String>() {
        @Override
        public void onResponse(String response) {
            try {
                JSONObject jsonResponseObj = new JSONObject(response);

                if (!jsonResponseObj.getBoolean("error")) {
                    Toast.makeText(SignInActivity.this, jsonRe-
sponseObj.getString("message"), Toast.LENGTH_SHORT).show();

                    JSONObject jsonUserObj = jsonResponseObj.getJSONOb-
ject("user");
                    userSession.createSignInSession(jsonUse-
rObj.getString("userName"), jsonUserObj.getString("userEmail"));

                    finish();
                    startActivity(new Intent(getApplicationContext(), MainActiv-
ity.class));
                }
                else {
                    Toast.makeText(SignInActivity.this, jsonRe-
sponseObj.getString("message"), Toast.LENGTH_SHORT).show();
                }
            } catch (JSONException e) {
                e.printStackTrace();
            }
        }
    }, new Response.ErrorListener() {
        @Override
        public void onErrorResponse(VolleyError error) {
            Toast.makeText(SignInActivity.this, error.getMessage(),
Toast.LENGTH_SHORT).show();
        }
    }) {
        @Override
        protected Map<String, String> getParams() {
            Map<String, String> params = new HashMap<>();
            params.put("userName", userName);
            params.put("userPassword", userPassword);
            return params;
        }
    };
    VolleySingleton.getInstance(this).addToRequestQueue(stringRequest);
}
```

Figure 4-26. Sign In activity code snippet

When *signIn()* function is called, it will create an object of *StringRequest* with method POST and specified URL. It overrides the *getParams()* to modify the parameters passed which in this case is userName and userPassword. The function *onResponse()* is used to handle response get from executing the *stringRequest* object. In this case, at first it will check whether the return response is success or not by looking the value of *error* in the response. If it is false, means the account credentials is correct. Next, a sign in session is created to avoid user from having to keep on filling in the sign in credentials every time they run the application. Lastly, the user is redirected to main screen of the application which is *MainActivity*.

4.4.2 – SIGN UP

In Sign Up activity, the users are required to fill in all the text boxes to register the account.

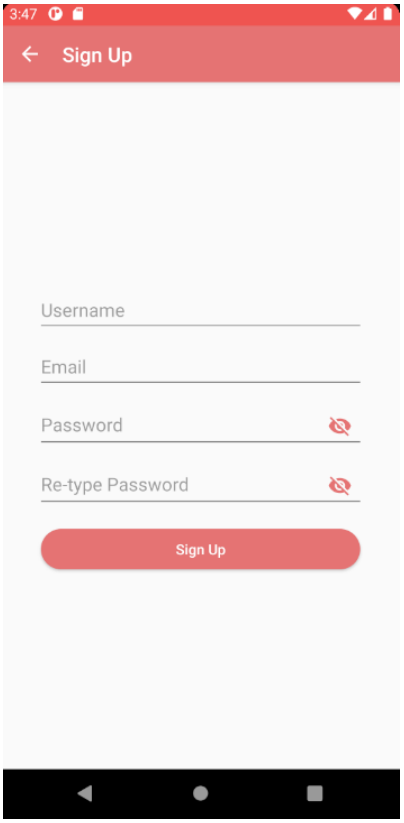


Figure 4-27. Sign Up activity

When the users click the “Sign Up” button, the application will check whether all the text boxes have been filled by checking the length of every text boxes input.

```
findViewById(R.id.Button_signUp).setOnClickListener(new View.OnClickListener() {
    @Override
    public void onClick(View view) {
        String userName =
autoCompleteTextView_userName.getText().toString().trim();
        String userEmail =
autoCompleteTextView_userEmail.getText().toString().trim();
        String userPassword = editText_password.getText().toString().trim();
        String userConfirmPassword =
editText_confirmPassword.getText().toString().trim();

        if (userName.length() <= 0 || userEmail.length() <= 0 ||
userPassword.length() <= 0 || userConfirmPassword.length() <= 0)
            Toast.makeText(SignUpActivity.this, "Please fill in all the text
boxes!", Toast.LENGTH_SHORT).show();
        else {
            if (userPassword.equals(userConfirmPassword))
                signUp(userName, userEmail, userPassword);
            else
                Toast.makeText(SignUpActivity.this, "The password and confirm
password does not match!", Toast.LENGTH_SHORT).show();
        }
    }
});
```

Figure 4-28. Sign Up code snippet #1

If all the text boxes are filled in, then it will check if the password and re-typed password is the same or not. If it is the same, then it will then call the *signUp()* function to register an account with the filled in credentials.

```
private void signUp(final String userName, final String userEmail, final String
userPassword) {
    StringRequest stringRequest = new StringRequest(Request.Method.POST,
API.URL_SIGNUP, new Response.Listener<String>() {
        @Override
        public void onResponse(String response) {
            try {
                JSONObject jsonResponseObj = new JSONObject(response);
                if (!jsonResponseObj.getBoolean("error")) {
                    Toast.makeText(SignUpActivity.this,
jsonResponseObj.getString("message"), Toast.LENGTH_SHORT).show();
                    finish();
                    startActivity(new Intent(getApplicationContext(),
SignInActivity.class));
                }
                else {
                    Toast.makeText(SignUpActivity.this,
jsonResponseObj.getString("message"), Toast.LENGTH_SHORT).show();
                }
            } catch (JSONException e) {
                e.printStackTrace();
            }
        }
    }, new Response.ErrorListener() {
        @Override
        public void onErrorResponse(VolleyError error) {
            Toast.makeText(SignUpActivity.this, error.getMessage(),
Toast.LENGTH_SHORT).show();
        }
    }) {
        @Override
        protected Map<String, String> getParams() {
            Map<String, String> params = new HashMap<>();
            params.put("userName", userName);
            params.put("userEmail", userEmail);
            params.put("userPassword", userPassword);
            return params;
        }
    };
    VolleySingleton.getInstance(this).addToRequestQueue(stringRequest);
}
```

Figure 4-29. Sign Up activity code snippet #2

In this function, it will send *userName*, *userEmail* and *userPassword* as parameters to the API URL called using POST request. If the registration is successful, the application will send a *toast* message and redirects the user back to the sign in activity. Else, an error message is shown to the user in a *toast*.

4.4.3 – HOME

Home is one of the fragments in *MainActivity*. It is the first fragment that will be loaded whenever the user is redirected to *MainActivity*. In this activity, the user is displayed with the current estimated LPG level of the LPG tank. The user can switch between the devices if more than one device is registered.

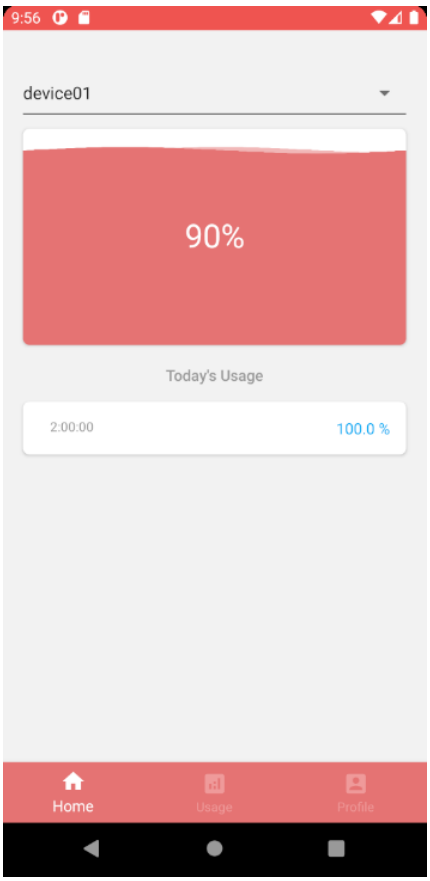


Figure 4-30. Home fragment

When the home fragment is being loaded, the application will fetch the list of devices that the user has and load them into the spinner. Based on the spinner's selection, it will load the *WaveLoadingView* according to the device's current estimated LPG level. The average usage for every hour is shown as cards under the "Today's Usage". In this case, at 2 AM the LPG level is still at 100%.

```
private void getTodayUsage(final String deviceID) {
    StringRequest stringRequest = new StringRequest(Request.Method.POST,
API.URL_GETUSAGEVALUES, new Response.Listener<String>() {
        @Override
        public void onResponse(String response) {
            try {
                JSONObject jsonResponseObj = new JSONObject(response);
                if (!jsonResponseObj.getBoolean("error")) {
                    JSONArray jsonUsageValuesArray =
jsonResponseObj.getJSONArray("usageValues");
                    usageHomeList.clear();

                    if (jsonUsageValuesArray.length() != 0) {
                        for (int i = 0; i < jsonUsageValuesArray.length(); i++) {
                            JSONObject jsonUsageValueObj =
jsonUsageValuesArray.getJSONObject(i);
                            usageHomeList.add(new
UsageHome(jsonUsageValueObj.getInt("readingValue"),
jsonUsageValueObj.getString("readingTime")));
                        }
                    }
                    else {
                        Toast.makeText(getContext(), "No today's usage data for "
+ appCompatSpinner_device.getSelectedItem().toString(),
Toast.LENGTH_SHORT).show();
                    }

                    recyclerView.setLayoutManager(new
LinearLayoutManager(getContext(), LinearLayoutManager.VERTICAL, false));
                    UsageHomeAdapter usageHomeAdapter = new
UsageHomeAdapter(getContext(), usageHomeList);
                    recyclerView.setAdapter(usageHomeAdapter);
                }
                else {
                    Toast.makeText(getContext(),
jsonResponseObj.getString("message"), Toast.LENGTH_SHORT).show();
                }
            }
            catch (JSONException e) {
                e.printStackTrace();
            }
        }
    }, new Response.ErrorListener() {
        @Override
        public void onErrorResponse(VolleyError error) {
            Toast.makeText(getContext(), error.getMessage(),
Toast.LENGTH_SHORT).show();
        }
    }) {
        @Override
        protected Map<String, String> getParams() {
            Map<String, String> params = new HashMap<>();
            params.put("deviceID", deviceID);
            params.put("usageMode", "0");
            return params;
        }
    };
    VolleySingleton.getInstance(getContext()).addToRequestQueue(stringRequest);
}
```

Figure 4-31. Home fragment code snippet #1

The *getTodayUsage()* function is used to get an array of the selected *deviceID*'s LPG level usages during that day. It accepts only *deviceID* as parameter but on *getParams()* function it sends *deviceID* and *usageMode* of 0. If the request is successful, the *stringRequest* will get an array response from the API. The array is passed to an adapter to be displayed into a *RecyclerView*.

```
public void loadWaveLoadingView(final String deviceID) {
    final FirebaseDatabase firebaseDatabase = FirebaseDatabase.getInstance();
    DatabaseReference databaseReference = firebaseDatabase.getReference();

    databaseReference.addValueEventListener(new ValueEventListener() {
        @Override
        public void onDataChange(@NonNull DataSnapshot dataSnapshot) {
            if (dataSnapshot.hasChild(deviceID + "/gas_level")) {
                int gasLevel = dataSnapshot.child(deviceID +
"/gas_level").getValue(Integer.class);
                waveLoadingView_gasLevelPercentage.setProgressValue(gasLevel);
                waveLoadingView_gasLevelPercentage.setCenterTitle(gasLevel + "%");
            }
            else {
                waveLoadingView_gasLevelPercentage.setProgressValue(0);
                waveLoadingView_gasLevelPercentage.setCenterTitle("No Data");
                Toast.makeText(getContext(), "No Data! Please reboot/start your
device to view data", Toast.LENGTH_SHORT).show();
            }
        }

        @Override
        public void onCancelled(@NonNull DatabaseError databaseError) {
            Log.w(TAG, "Failed to read value.", databaseError.toException());
        }
    });
}
```

Figure 4-32. Home fragment code snippet #2

As for the *WaveLoadingView*, it changes dynamically according to the spinner selection. If there is no node exist in the database, it will set the progress value to 0 and display out "No Data" message.

4.4.3 - USAGE

In Usage fragment, the user is displayed with a graph and the usage history cards underneath it. Here, the user can select to view the LPG usage according to *Spinner*'s selection. There are several choices which are to view today's usage, this week's usage, this month's usage, and this year's usage. The image below showing this month's usage data for device01.

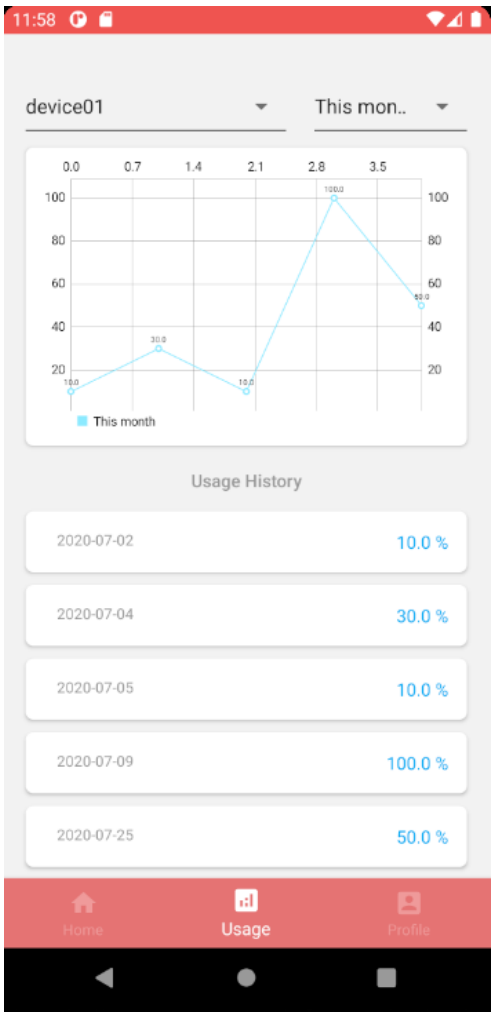


Figure 4-33. Usage fragment

The graph and the usage history cards are loaded using a function called *getUsageValues()*. This function accepts two parameters which are *deviceID* and *graphMode*. Both parameters are taken from the two spinners' selected item on top of the usage fragment.

```
try {
    JSONObject jsonResponseObj = new JSONObject(response);
    if (!jsonResponseObj.getBoolean("error")) {
        JSONArray jsonUsageValuesArray =
jsonResponseObj.getJSONArray("usageValues");
        usageList.clear();

        if (jsonUsageValuesArray.length() != 0) {
            for (int i = 0; i < jsonUsageValuesArray.length(); i++) {
                JSONObject jsonUsageValueObj =
jsonUsageValuesArray.getJSONObject(i);
                usageValues.add(new Entry(i,
jsonUsageValueObj.getInt("readingValue")));
                usageList.add(new Usage(jsonUsageValueObj.getInt("readingValue"),
jsonUsageValueObj.getString("readingTime")));
            }

            LineDataSet lineDataSet = new LineDataSet(usageValues, graphMode);
            ArrayList<ILineDataSet> dataSets = new ArrayList<>();
            dataSets.add(lineDataSet);

            LineData data = new LineData(dataSets);
            lineChart.setData(data);
            lineChart.invalidate();
        }
        else {
            Toast.makeText(getContext(), "No Data!", Toast.LENGTH_SHORT).show();
        }

        recyclerView.setLayoutManager(new LinearLayoutManager(getContext(),
LinearLayoutManager.VERTICAL, false));
        UsageAdapter usageAdapter = new UsageAdapter(getContext(), usageList);
        recyclerView.setAdapter(usageAdapter);
    }
    else {
        Toast.makeText(getContext(), jsonResponseObj.getString("message"),
Toast.LENGTH_SHORT).show();
    }
}
catch (JSONException e) {
    e.printStackTrace();
}
```

Figure 4-34. Usage fragment code snippet #1

If the parameters used to send API request returns a non-empty array, the values are added into two *ArrayList* where both is used to load the data for graph and the usage history *RecyclerView*.

4.4.4 – PROFILE

In profile fragment, the user can access settings for his/her account, application, and device. Those are “Profile”, “IoT Device”, “App”. However, “App” and “About App” are left as future work. The user can also sign out from the application by clicking the “Sign Out” button.

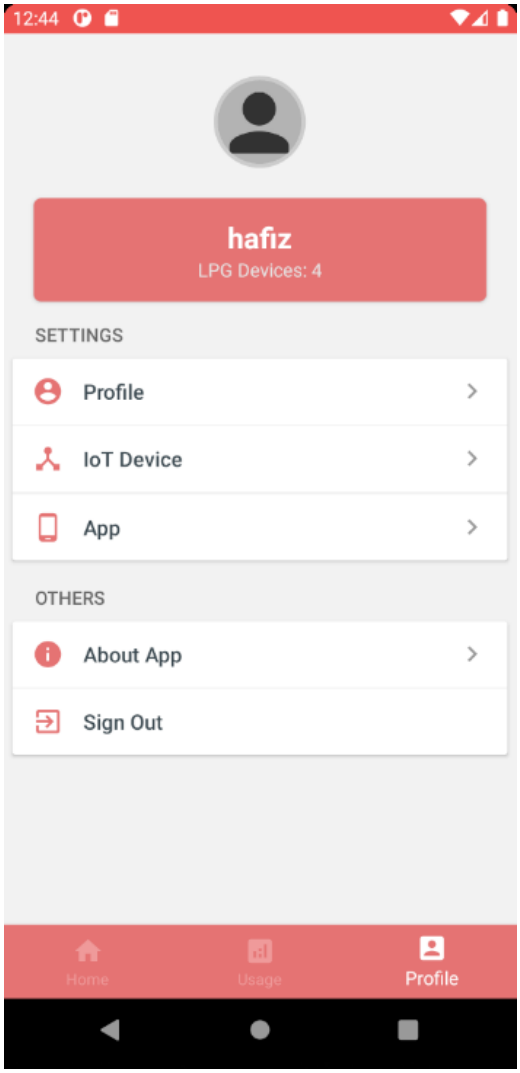


Figure 4-35. Profile fragment

The pink card on top side of the fragment shows the user’s username and how much devices have been registered.

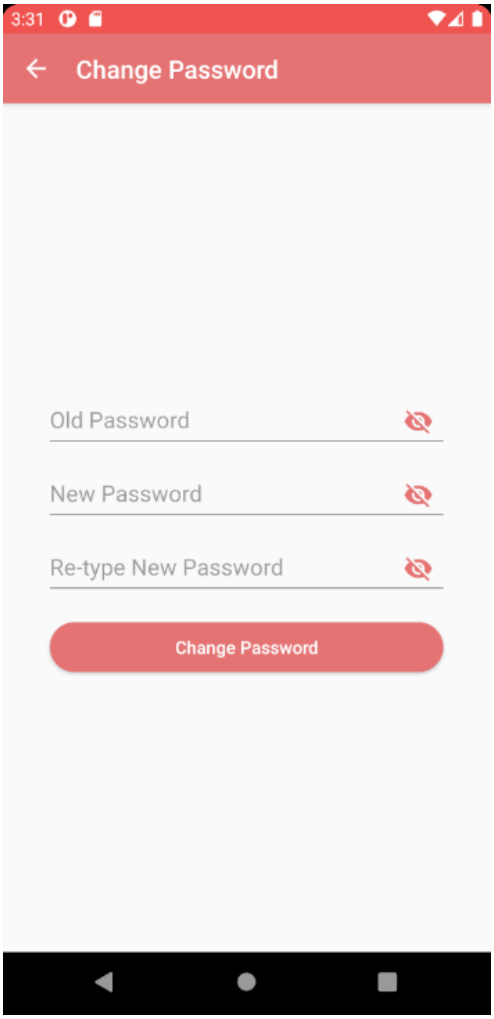


Figure 4-36. Change Password activity

In profile settings, the user can change the account password. The old password will be compared with the one in database using *checkOldPassword()* function to check whether it is correct or not.

```
private void checkOldPassword(final String oldPassword, final String newPassword,
final String userName) {
    StringRequest stringRequest = new StringRequest(Request.Method.POST,
API.URL_CHECKOLDPASSWORD, new Response.Listener<String>() {
        @Override
        public void onResponse(String response) {
            try {
                JSONObject jsonResponseObj = new JSONObject(response);
                if (!jsonResponseObj.getBoolean("error")) {
                    changePassword(newPassword, userName);
                } else {
                    Toast.makeText(SettingsProfileActivity.this,
jsonResponseObj.getString("message"), Toast.LENGTH_SHORT).show();
                }
            } catch (JSONException e) {
                e.printStackTrace();
            }
        }
    }, new Response.ErrorListener() {
        @Override
        public void onErrorResponse(VolleyError error) {
            Toast.makeText(SettingsProfileActivity.this, error.getMessage(),
Toast.LENGTH_SHORT).show();
        }
    }) {
        @Override
        protected Map<String, String> getParams() {
            Map<String, String> params = new HashMap<>();
            params.put("oldPassword", oldPassword);
            params.put("userName", userName);
            return params;
        }
    };
    VolleySingleton.getInstance(this).addToRequestQueue(stringRequest);
}
```

Figure 4-37. Change Password activity code snippet #1

If the API returns error as false, then it will execute another function called *changePassword()* to change the password in the database according to the one that the user filled in.

```
private void changePassword(final String newPassword, final String userName) {
    StringRequest stringRequest = new StringRequest(Request.Method.POST,
API.URL_CHANGEPASSWORD, new Response.Listener<String>() {
        @Override
        public void onResponse(String response) {
            try {
                JSONObject jsonResponseObj = new JSONObject(response);
                Toast.makeText(SettingsProfileActivity.this, jsonRe-
sponseObj.getString("message"), Toast.LENGTH_SHORT).show();
            } catch (JSONException e) {
                e.printStackTrace();
            }
        }
    }, new Response.ErrorListener() {
        @Override
        public void onErrorResponse(VolleyError error) {
            Toast.makeText(SettingsProfileActivity.this, error.getMessage(),
Toast.LENGTH_SHORT).show();
        }
    }) {
        @Override
        protected Map<String, String> getParams() {
            Map<String, String> params = new HashMap<>();
            params.put("newPassword", newPassword);
            params.put("userName", userName);
            return params;
        }
    };
    VolleySingleton.getInstance(this).addToRequestQueue(stringRequest);
}
```

Figure 4-38. Change Password activity code snippet #2

This function will toast the return message whether the request is a success or failed.

As for “IoT Device” settings, the user can add a new device, modify the device’s settings, or delete a device. Figure below shows the user has one device under his/her account which is named as “Kitchen’s Gas”.

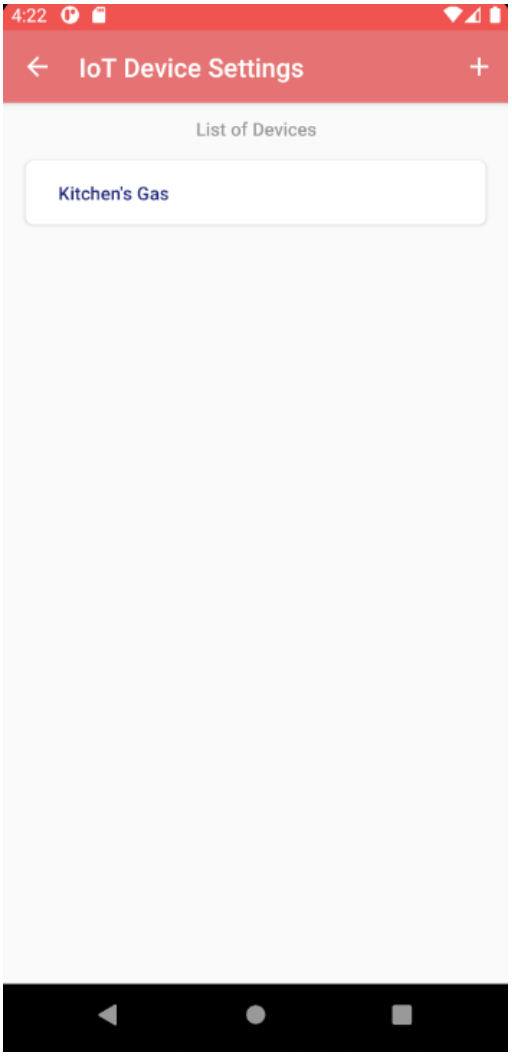


Figure 4-39. IoT Device settings activity

The device list cards are loaded using *loadDeviceList()* function. This function sends user's *userName* as parameter to the API. If the request is executed successfully, the API returns an array of device list for the *userName*. The array is then added into an *ArrayList* called *deviceList*. After that, the *deviceList* is passed to an adapter called *DeviceAdapter* that is used to load the *RecyclerView*.

```
private void loadDeviceList() {
    recyclerView_deviceList.setLayoutManager(new LinearLayoutManager(this,
LinearLayoutManager.VERTICAL, false));
    final List<Device> deviceList = new ArrayList<>();

    StringRequest stringRequest = new StringRequest(Request.Method.POST,
API.URL_GET_DEVICE_LIST, new Response.Listener<String>() {
        @Override
        public void onResponse(String response) {
            try {
                JSONObject jsonResponseObj = new JSONObject(response);
                JSONArray jsonDeviceListArray =
jsonResponseObj.getJSONArray("deviceList");

                for (int i=0; i<jsonDeviceListArray.length(); i++) {
                    JSONObject jsonDeviceObj =
jsonDeviceListArray.getJSONObject(i);
                    deviceList.add(new Device(jsonDeviceObj.getString("deviceID"),
jsonDeviceObj.getInt("thresholdPercentage"),
                            jsonDeviceObj.getInt("tickerTime"),
jsonDeviceObj.getString("deviceName")));
                }

                DeviceAdapter deviceAdapter = new
DeviceAdapter(getApplicationContext(), deviceList);
                recyclerView_deviceList.setAdapter(deviceAdapter);
            }
            catch (JSONException e) {
                e.printStackTrace();
            }
        }
    }, new Response.ErrorListener() {
        @Override
        public void onErrorResponse(VolleyError error) {
            Toast.makeText(SettingsIoTDeviceActivity.this, error.getMessage(),
Toast.LENGTH_SHORT).show();
        }
    }) {
        @Override
        protected Map<String, String> getParams() {
            Map<String, String> params = new HashMap<>();
            params.put("userName",
Objects.requireNonNull(userDetails.get(SessionManager.KEY_USERNAME)));
            return params;
        }
    };
    VolleySingleton.getInstance(this).addToRequestQueue(stringRequest);
}
```

Figure 4-40. IoT Device settings code snippet #1

When the “Kitchen’s Gas” is clicked, the user will be redirected to the “Kitchen’s Gas” settings activity.

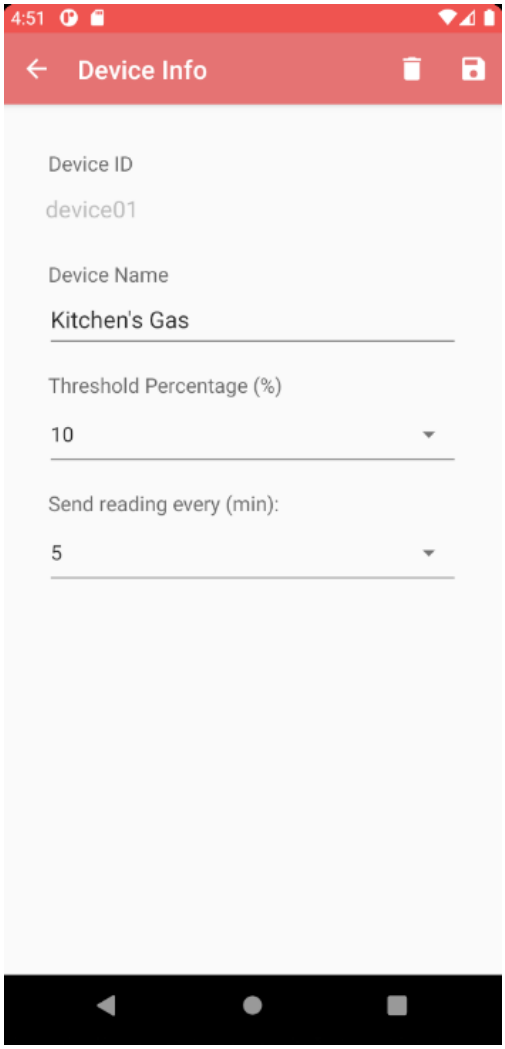


Figure 4-41. Device info activity

In this activity, the user can change the device's name, threshold percentage and send reading timer. The threshold percentage is for push notification service. The user will be notified when the "Kitchen's Gas" LPG level reaches 10% or below. As for the send reading, it refers to for every x minute does the user wants the device to send its reading. In this case, the device will send its reading for every 5 minutes. Finally, click the save button on top-right of the screen to save the settings. The settings will be updated to database using *saveDeviceInfo()* function.

```
public void saveDeviceInfo() {
    StringRequest stringRequest = new StringRequest(Request.Method.POST,
API.URL_SAVE_DEVICE_INFO, new Response.Listener<String>() {
        @Override
        public void onResponse(String response) {
            try {
                JSONObject jsonResponseObj = new JSONObject(response);
                if (!jsonResponseObj.getBoolean("error")) {
                    Toast.makeText(DeviceInfoActivity.this,
jsonResponseObj.getString("message"), Toast.LENGTH_SHORT).show();
                    finish();
                }
                else {
                    Toast.makeText(DeviceInfoActivity.this,
jsonResponseObj.getString("message"), Toast.LENGTH_SHORT).show();
                }
            }
            catch (JSONException e) {
                e.printStackTrace();
            }
        }
    }, new Response.ErrorListener() {
        @Override
        public void onErrorResponse(VolleyError error) {
            Toast.makeText(DeviceInfoActivity.this, error.getMessage(),
Toast.LENGTH_SHORT).show();
        }
    }) {
        @Override
        protected Map<String, String> getParams() {
            Map<String, String> params = new HashMap<>();
            params.put("deviceID", editText_deviceID.getText().toString().trim());
            params.put("thresholdPercentage",
appCompatSpinner_thresholdPercentage.getSelectedItem().toString().trim());
            params.put("tickerTime",
appCompatSpinner_tickerTime.getSelectedItem().toString().trim());
            params.put("deviceName",
editText_deviceName.getText().toString().trim());
            return params;
        }
    };
    VolleySingleton.getInstance(this).addToRequestQueue(stringRequest);
}
```

Figure 4-42. Device Info code snippet #1

The *saveDeviceInfo()* function sends four parameters which are *deviceID*, *thresholdPercentage*, *tickerTime* and *deviceName*. The function will *Toast* the status message after executing the request.

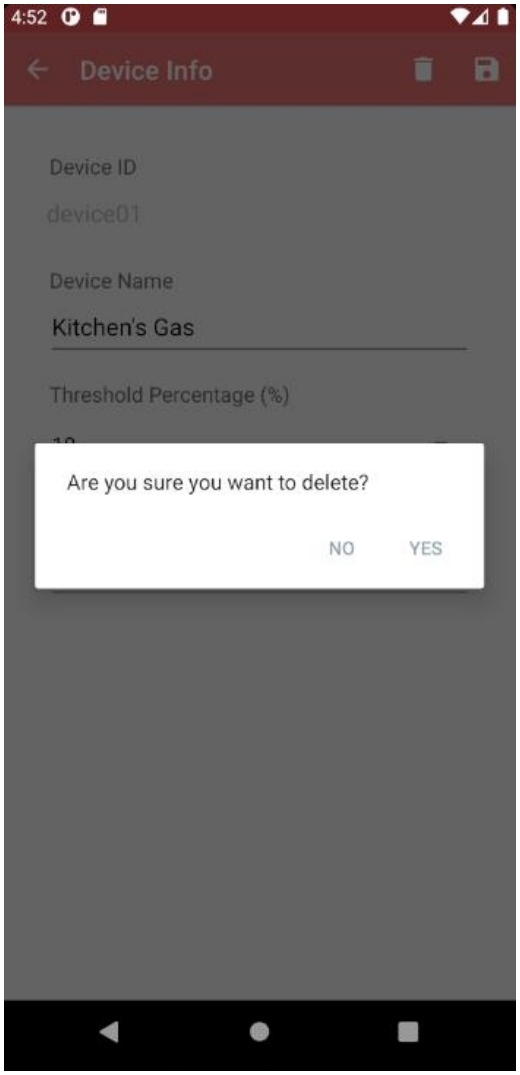


Figure 4-43. Device Info activity (Delete device)

The device record can also be deleted by simply clicking the delete button on top-right of the screen. The user will be prompted with an *AlertDialog* for confirmation. This process is executed using *deleteDevice()* function.

```
public void deleteDevice() {
    StringRequest stringRequest = new StringRequest(Request.Method.POST,
API.URL_DELETE_DEVICE, new Response.Listener<String>() {
        @Override
        public void onResponse(String response) {
            try {
                JSONObject jsonResponseObj = new JSONObject(response);
                if (!jsonResponseObj.getBoolean("error")) {
                    Toast.makeText(DeviceInfoActivity.this,
jsonResponseObj.getString("message"), Toast.LENGTH_SHORT).show();
                    finish();
                }
                else {
                    Toast.makeText(DeviceInfoActivity.this,
jsonResponseObj.getString("message"), Toast.LENGTH_SHORT).show();
                }
            }
            catch (JSONException e) {
                e.printStackTrace();
            }
        }
    }, new Response.ErrorListener() {
        @Override
        public void onErrorResponse(VolleyError error) {
            Toast.makeText(DeviceInfoActivity.this, error.getMessage(),
Toast.LENGTH_SHORT).show();
        }
    }) {
        @Override
        protected Map<String, String> getParams() {
            Map<String, String> params = new HashMap<>();
            params.put("deviceID", editText_deviceID.getText().toString().trim());
            return params;
        }
    };
    VolleySingleton.getInstance(this).addToRequestQueue(stringRequest);
}
```

Figure 4-44. Device Info code snippet #2

This function sends only *deviceID* as parameter. The request status is then displayed in *Toast*.

For sign out, the user can click the “Sign Out” button. This will sign the user out of the application and at the same time deleting the session. The user is then redirected to the *SignIn* activity. When the application is opened again in the future, the user must sign in again to use the application.

4.4.5 – UTILITY CLASSES

In this application, there are four utility classes namely *API*, *SessionManager*, *Tools*, and *VolleySingleton*. Each of these classes has its own functionality.

4.4.5.1 – API

API class is used to store the PHP REST API URL. There are nine API URLs and each of them has its own functionality and parameters needs to be passed.

```
public class API {
    private static final String ROOT_URL =
"https://myfyp.com/api/API.php?apicall=";

    public static final String URL_SIGNIN = ROOT_URL + "signin";
    public static final String URL_SIGNUP = ROOT_URL + "signup";
    public static final String URL_ADD_DEVICE = ROOT_URL + "adddevice";
    public static final String URL_GET_DEVICE_LIST = ROOT_URL + "getdevicelist";
    public static final String URL_CHECKOLDPASSWORD = ROOT_URL +
"checkoldpassword";
    public static final String URL_CHANGEPASSWORD = ROOT_URL + "changepassword";
    public static final String URL_GETUSAGEVALUES = ROOT_URL + "getusagevalues";
    public static final String URL_SAVE_DEVICE_INFO = ROOT_URL + "savedeviceinfo";
    public static final String URL_DELETE_DEVICE = ROOT_URL + "deletedevice";
}
```

Figure 4-45. API class

4.4.5.2 – SessionManager

SessionManager class on the other hand is used to handle the sign in or out session of the user. This class has several functions used to store and remove the sign in session.

```
public void createSignInSession(String userName, String userEmail) {
    editor.putBoolean(IS_SIGNED_IN, true);
    editor.putString(KEY_USERNAME, userName);
    editor.putString(KEY_USEREMAIL, userEmail);
    editor.commit();
}
```

Figure 4-46. SessionManager code snippet #1

The *createSignInSession()* function stores the *userName*, *userEmail* and the sign in status in a *SharedPreferences.Editor*. The *checkIsUserSignedIn()* function is used to check the user's sign in status.

```
public boolean checkIsUserSignedIn() {
    return sharedPreferences.getBoolean(IS_SIGNED_IN, false);
}
```

Figure 4-47. SessionManager code snippet #2

This function returns the current Boolean value that the *IS_SIGNED_IN* key is holding. The second parameter, *false* is for default value if the key is empty.

```
public HashMap<String, String> getUserDetails() {
    HashMap<String, String> user = new HashMap<>();
    user.put(KEY_USERNAME, sharedPreferences.getString(KEY_USERNAME, null));
    user.put(KEY_USEREMAIL, sharedPreferences.getString(KEY_USEREMAIL, null));
    return user;
}
```

Figure 4-48. SessionManager code snippet #3

The function *getUserDetails()* is used to return the current user details from the session. It fetches *KEY_USERNAME* and *KEY_USEREMAIL* in the *sharedPreferences* object. Then it stores and return them in a *HashMap<String, String>*.

```
public void signOutUser() {
    editor.clear();
    editor.commit();

    Intent intent = new Intent(context, SignInActivity.class);
    intent.addFlags(Intent.FLAG_ACTIVITY_CLEAR_TOP);
    intent.addFlags(Intent.FLAG_ACTIVITY_NEW_TASK);

    context.startActivity(intent);
}
```

Figure 4-49. SessionManager code snippet #4

For *signOutUser()* function, it is used to clear the user's session and redirects him/her to sign in activity screen.

4.4.5.3 – Tools

Tool class has only one function. It is used to set the system status bar colour according to the application primary colour.

```
public static void setSystemBarColor(Activity act) {
    if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.LOLLIPOP) {
        Window window = act.getWindow();

window.addFlags(WindowManager.LayoutParams.FLAG_DRAWS_SYSTEM_BAR_BACKGROUNDS);
        window.clearFlags(WindowManager.LayoutParams.FLAG_TRANSLUCENT_STATUS);

window.setStatusBarColor(act.getResources().getColor(R.color.colorPrimaryDark));
    }
}
```

Figure 4-50. Tool code snippet

4.4.5.4 – VolleySingleton

VolleySingleton class is used to handle API request from *StringRequest*. This main function for this class is to avoid from creating multiple objects when executing requests. It uses only one object from VolleySingleton class. Thus, this will save computing resources.

```java
public class VolleySingleton {
    private static VolleySingleton mInstance;
    private RequestQueue mRequestQueue;
    private static Context mCtx;

    private VolleySingleton(Context context) {
        mCtx = context;
        mRequestQueue = getRequestQueue();
    }

    public static synchronized VolleySingleton getInstance(Context context) {
        if (mInstance == null)
            mInstance = new VolleySingleton(context);

        return mInstance;
    }

    public RequestQueue getRequestQueue() {
        if (mRequestQueue == null)
            mRequestQueue = Volley.newRequestQueue(mCtx.getApplicationContext());

        return mRequestQueue;
    }

    public <T> void addToRequestQueue(Request<T> req) {
        getRequestQueue().add(req);
    }
}
```

Figure 4-51. VolleySingleton code snippet

This class has 3 functions. The constructor gets the current context and the current request queue for that context using *getRequestQueue()* function. The *addToRequestQueue()* function on the other hand is used to add the request into the *RequestQueue* in Volley. The request is then executed and removed from the queue once it has completed.

4.4.6 – LIBRARIES USED

In this application, there are several libraries used. The libraries are all imported using gradle.

```
implementation 'androidx.appcompat:appcompat:1.1.0'
implementation 'com.google.android.material:material:1.1.0'
implementation 'androidx.constraintlayout:constraintlayout:1.1.3'
```

Figure 4-52. Gradle code snippet #1

The libraries above are the stock libraries included when creating this application. These libraries are mostly used for developing the application's layout.

```
implementation 'com.google.android.gms:play-services-maps:17.0.0'
implementation('com.google.android.libraries.places:places:1.0.0') {
    exclude module: 'glide'
}
```

Figure 4-53. Gradle code snippet #2

For the libraries above, they are used to import the *color* values and for Volley library. Glide library is excluded as it is not being used in this application.

```
implementation 'com.balysv:material-ripple:1.0.2'                  // Ripple
effect
implementation 'com.mikhaellopez:circularimageview:3.2.0'          // Circular
image view
implementation 'com.github.PhilJay:MPAndroidChart:v3.1.0'          // Graph
implementation 'me.itangqi.waveloadingview:library:0.3.5'          // Wave Loading
implementation 'me.dm7.barcodescanner:zxing:1.9.13'                // Barcode
implementation 'com.android.support:multidex:1.0.3'
```

Figure 4-54. Gradle code snippet #3

Libraries above are the third-party libraries used in this application. The first library is used for applying ripple effect when pressing a button. The second library is used for circular image view layout. The third library is used to create graph. The fourth library is used for wave loading view, used in Home fragment. The fifth library is used for QR code camera. The last library is for the wave loading view library as it needs multidex for it to function.

4.5 – SUMMARY

This chapter has discussed about the implementation process of this project. There are three main categories involved in this project which are the IoT device, REST API, and the Android application. Each of them has its own important role. The IoT device is used to measure the LPG level and send the readings to database. REST API on the other hand is used for to provide an interface for the IoT device and Android application to communicate with the database. Lastly, the Android application is used to display the readings sent by the device in a presentable manner.

CHAPTER 5 - TESTING

5.1 - INTRODUCTION

This chapter discusses in detail about testing process of the three main categories namely the IoT device, REST API, and Android application. Each of them is tested according to some of ISO 9126 testing characteristics. The results of the testing process are summarised at the end of this chapter.

5.2 – IOT DEVICE

5.2.1 – FUNCTIONALITY

In this functionality test, each of the device's functions are tested by specified input data to check whether the actual results satisfy the expected results.

Test Case ID	TC_DEVICE_01				
Test Objective	To test the load cell functionality				
Test ID	**Test Case**	**Input Data**	**Expected Result**	**Actual Result**	**Status**
1	Measure the weight of LPG tank	Fully filled LPG tank	Around 275 grams	273 grams	Pass
2	Calculate the correct estimated LPG level in percentage	Half-filled LPG tank	50%	50%	Pass

Table 13. Test case TC_DEVICE_01

Test Case ID	TC_DEVICE_02				
Test Objective	To test the WiFi function				
Test ID	**Test Case**	**Input Data**	**Expected Result**	**Actual Result**	**Status**
1	Connecting to a WiFi/hotspot	Home's WiFi SSID and Password	Able to connect to WiFi	Connected to WiFi	Pass

Table 14. Test case TC_DEVICE_02

Test Case ID	TC_DEVICE_03				
Test Objective	To test the HTTP POST request function				
Test ID	**Test Case**	**Input Data**	**Expected Result**	**Actual Result**	**Status**
1	Fetching device settings from database	API URL for fetching device settings and its parameter (deviceID)	Successfully fetched device settings	Successfully fetched device settings	Pass
2	Sending reading to database	API URL for send readings and its parameters (readingValue and deviceID)	Successfully sent the reading into database	Successfully sent the reading into database	Pass

Table 15. Test case TC_DEVICE_03

Test Case ID	TC_DEVICE_04				
Test Objective	To test the Firebase function				
Test ID	**Test Case**	**Input Data**	**Expected Result**	**Actual Result**	**Status**
1	Sending reading to Firebase Realtime database	Current reading value	The data changes according to the current reading value	The data in the database is the same as the current reading value	Pass
2	Sending push notification to Firebase Cloud Messaging service via topic	Topic name = thresholdAlert and notification message	Push notification is successfully sent	Push notification is successfully sent	Pass

Table 16. Test case TC_DEVICE_04

Test Case ID	TC_DEVICE_05				
Test Objective	To test the ArduinoJson library function				
Test ID	**Test Case**	**Input Data**	**Expected Result**	**Actual Result**	**Status**
1	Parsing the return response for device settings from API	JSON encoded device settings response	The JSON encoded string is parsed correctly	The JSON encoded string is parsed correctly	Pass

Table 17. Test case TC_DEVICE_05

5.2.2 – RELIABILITY

In reliability testing, the device is tested with unwanted inputs to check whether the device can handle errors properly to ensure a failure-free operation throughout the whole process.

Test Objective	To test the reliability of the IoT device				
Test ID	**Test Case**	**Input Data**	**Expected Result**	**Actual Result**	**Status**
1	Entering an invalid WiFi SSID and Password	SSID = “test” Pass = “test”	The device will not be connected to the WiFi	The device is not connected to the WiFi	Pass
2	Unable to fetch device settings	-	Using the default device settings	Default device settings is used	Pass
3	Failed to initialize load cell sensor	-	Stop the booting process	Booting process is stopped	Pass
4	Ensuring the LPG level does not exceed 100% when the reading exceeds the predefined maximum weight	290 grams	LPG level will not exceed 100%	LPG level stays at 100%	Pass
5	Sending an invalid API call	Invalid API call URL	Printing out error response	Error response is printed out in monitor	Pass
6	Failed to send reading to Firebase Realtime Database	-	Printing out error response	Error response is printed out in monitor	Pass
7	Failed to send push notification to Firebase Cloud Messaging	-	Printing out error response	Error response is printed out in monitor	Pass

Table 18. Reliability testing for IoT device

5.2.3 – EFFICIENCY

In efficiency testing, the device is tested with how long it takes to execute a certain task. The actual result for execution time is taken using *micros()* function in Arduino. The function returns the execution time in microseconds.

Test Objective	To test the efficiency of the IoT device			
Test ID	**Test Case**	**Expected Result**	**Actual Result**	**Status**
1	Device full boot up	20s	14.77s	Pass
2	Connects to WiFi	15s	13.49s	Pass
3	Fetch device settings	2s	1.78s	Pass
4	Initialize Firebase	Below 1s	0.05s	Pass
5	Initialize load cell	1s	1.97s	Fail
6	Send reading to MySQL database	2s	1.21s	Pass
7	Send reading to Firebase database	2s	0.8s	Pass
8	Send push notification to Firebase	2s	0.34s	Pass

Table 19. Efficiency testing for IoT device

5.3 - API

5.3.1 – FUNCTIONALITY

In functionality test, the API functions are tested using a software called Postman. The APIs are tested using POST request in Postman.

Test Case ID	TC_API_01				
Test Objective	To test the sign in API call				
Test ID	**Test Case**	**Input Data**	**Expected Result**	**Actual Result**	**Status**
1	Entering valid sign in credentials	userName = “hafiz” userPassword = “hafiz123”	Successfully signed in JSON response	Successfully signed in JSON response	Pass
2	Entering invalid sign in credentials	userName = “hafiz” userPassword = “abc123”	Failed to sign in JSON response	Failed to sign in JSON response	Pass

Table 20. Test case TC_API_01

Test Case ID	TC_API_02				
Test Objective	To test the sign up API call				
Test ID	**Test Case**	**Input Data**	**Expected Result**	**Actual Result**	**Status**
1	Registering a non-registered account	userName = “azri” userEmail = “az@gmail.com” userPassword = “azri123”	Successfully signed up JSON response	Successfully signed up JSON response	Pass
2	Registering a registered account	userName = “hafiz” userEmail = “hafiz@gmail.com” userPassword = “abc123”	Failed to sign up JSON response	Failed to sign up JSON response	Pass

Table 21. Test case TC_API_02

Test Case ID	TC_API_03				
Test Objective	To test the get device settings API call				
Test ID	**Test Case**	**Input Data**	**Expected Result**	**Actual Result**	**Status**
1	Get device settings	deviceID = “device01”	An array of device settings in JSON encoded	Array of device settings in JSON encoded	Pass

Table 22. Test case TC_API_03

Test Case ID	TC_API_04				
Test Objective	To test the add device API call				
Test ID	**Test Case**	**Input Data**	**Expected Result**	**Actual Result**	**Status**
1	Add a non-registered device	userName = “hafiz” deviceID = “device02”	Successfully added the device in JSON encoded	Successfully added the device in JSON encoded	Pass
2	Add a registered device	userName = “hafiz” deviceID = “device01”	Failed to add the device in JSON encoded	Failed to add the device in JSON encoded	Pass

Table 23. Test case TC_API_04

Test Case ID	TC_API_05				
Test Objective	To test the get device list API call				
Test ID	**Test Case**	**Input Data**	**Expected Result**	**Actual Result**	**Status**
1	Send a registered userName	userName = “hafiz”	A device array list for “hafiz” in JSON encoded	An array list of device list for “hafiz” in JSON encoded	Pass
2	Send a non-registered userName	userName = “ali”	An empty device array list in JSON encoded	An empty device array list in JSON encoded	Pass

Table 24. Test case TC_API_05

Test Case ID	TC_API_06				
Test Objective	To test the check old password API call				
Test ID	**Test Case**	**Input Data**	**Expected Result**	**Actual Result**	**Status**
1	Send a valid old password	oldPassword = “hafiz1” userName = “hafiz”	Old password matches message in JSON encoded	Old password matches message in JSON encoded	Pass
2	Send an invalid old password	oldPassword = “hafiz2” userName = “hafiz”	Incorrect old password message in JSON encoded	Incorrect old password message in JSON encoded	Pass

Table 25. Test case TC_API_06

Test Case ID	TC_API_07				
Test Objective	To test the change password API call				
Test ID	**Test Case**	**Input Data**	**Expected Result**	**Actual Result**	**Status**
1	Change a registered user’s password	newPassword = “hafiz1” userName = “hafiz”	Successfully changed password message in JSON encoded	Successfully changed password message in JSON encoded	Pass
2	Change a non-registered user’s password	newPassword = “hafiz1” userName = “ali”	Failed to change password message in JSON encoded	Failed to change password message in JSON encoded	Pass

Table 26. Test case TC_API_07

Test Case ID	TC_API_08				
Test Objective	To test the get usage value API call				
Test ID	**Test Case**	**Input Data**	**Expected Result**	**Actual Result**	**Status**
1	Get a registered deviceID usage value for today	deviceID = “device01” usageMode = “0”	Today’s usage list for device01 in JSON encoded	Today’s usage list for device01 in JSON encoded	Pass
2	Get a non-registered usage value for today	deviceID = “device09” usageMode = “0”	An empty today’s usage list for device09 in JSON encoded	An empty today’s usage list for device09 in JSON encoded	Pass

Table 27. Test case TC_API_08

Test Case ID	TC_API_09				
Test Objective	To test the delete device API call				
Test ID	**Test Case**	**Input Data**	**Expected Result**	**Actual Result**	**Status**
1	Delete a registered deviceID	deviceID = “device01”	Successfully deleted device message in JSON encoded	Successfully deleted device message in JSON encoded	Pass
2	Delete a non-registered deviceID	deviceID = “device09”	Failed to delete device message in JSON encoded	Failed to delete device message in JSON encoded	Pass

Table 28. Test case TC_API_09

Test Case ID	TC_API_10				
Test Objective	To test the save device info API call				
Test ID	**Test Case**	**Input Data**	**Expected Result**	**Actual Result**	**Status**
1	Save a registered device's info	deviceID = "device01" thresholdPercentage = 10 tickerTime = 300 deviceName = "Kitchen's Gas"	Successfully saved device info message in JSON encoded	Successfully saved device info message in JSON encoded	Pass
2	Save a non-registered device's info	deviceID = "device09" thresholdPercentage = 10 tickerTime = 300 deviceName = "Kitchen's Gas #2"	Failed to save device info message in JSON encoded	Failed to save device info message in JSON encoded	Pass

Table 29. Test case TC_API_10

Test Case ID	TC_API_11				
Test Objective	To test the send readings API call				
Test ID	**Test Case**	**Input Data**	**Expected Result**	**Actual Result**	**Status**
1	Send a registered device reading	deviceID = "device01" readingValue = 90	Successfully sent reading message in JSON encoded	Successfully sent reading message in JSON encoded	Pass
2	Send a non-registered device reading	deviceID = "device09" readingValue = 90	Failed to send reading message in JSON encoded	Failed to send reading message in JSON encoded	Pass

Table 30. Test case TC_API_11

5.3.2 – RELIABILITY

In reliability testing, the API is tested with unwanted inputs to check whether it is capable of handling errors to ensure the API operation runs smoothly without interfering the other operations. Since all API calls have the same error handling mechanism, the testing is done on signin API call only.

Test Objective	To test the reliability of the API				
Test ID	**Test Case**	**Input Data**	**Expected Result**	**Actual Result**	**Status**
1	Entering an invalid API call	Apicall = “deletedatabase”	Invalid API call message in JSON encoded	Invalid API call message in JSON encoded	Pass
2	Entering invalid parameters for signin API call	userName = “hafiz” userEmail = “hafiz”	Parameter “userPassword” is missing message in JSON encoded	Parameter “userPassword” is missing message in JSON encoded	Pass
3	Calling signin API call with only userName as parameter	userName = “hafiz”	Parameter “userPassword” is missing message in JSON encoded	Parameter “userPassword” is missing message in JSON encoded	Pass

Table 31. Reliabiliy testing for API

5.3.3 – EFFICIENCY

In efficiency testing, every API call is tested using a software called Postman to check the time taken for each API calls.

Test Objective	To test the efficiency of API			
Test ID	**API Call**	**Expected Result**	**Actual Result**	**Status**
1	signin	2s	1.06s	Pass
2	signup	2s	1.27s	Pass
3	getdevicesettings	2s	0.95s	Pass
4	sendreadings	2s	1.38s	Pass
5	adddevice	2s	1.67s	Pass
6	getdevicelist	2s	1.20s	Pass
7	checkoldpassword	2s	1.00s	Pass
8	changepassword	2s	1.09s	Pass
9	getusagevalues	2s	1.00s	Pass
10	savedeviceinfo	2s	1.21s	Pass
11	deletedevice	2s	1.43s	Pass

Table 32. Efficiency testing for API

5.4 – ANDROID APPLICATION

5.4.1 – FUNCTIONALITY

In this testing process, only the core functions are tested.

Test Case ID	TC_APP_01				
Test Objective	To test the sign in function				
Test ID	**Test Case**	**Input Data**	**Expected Result**	**Actual Result**	**Status**
1	Signing in using valid credentials	userName = “hafiz” userPassword = “hafiz”	The user is signed in and redirected to MainActivity	User successfully signed in and redirected to MainActivity	Pass
2	Signing in using invalid credentials	userName = “hafiz” userPassword = “abc123”	Failed to sign in and a toast message is shown	Failed to sign in and a toast message is shown	Pass

Table 33. Test case TC_APP_01

Test Case ID	TC_APP_02				
Test Objective	To test the sign up function				
Test ID	**Test Case**	**Input Data**	**Expected Result**	**Actual Result**	**Status**
1	Sign up using unregistered userName	userName = “azri” userEmail = “azri@gmail.com” userPassword = “azri”	Successfully sign up and redirected back to sign in screen	Successfully sign up and redirected back to sign in screen	Pass
2	Signing in using invalid credentials	userName = “hafiz” userPassword = “abc123”	Failed to sign in and a toast message is shown	Failed to sign in and a toast message is shown	Pass

Table 34. Test case TC_APP_02

Test Case ID	TC_APP_03				
Test Objective	To test the Home fragment functions				
Test ID	**Test Case**	**Input Data**	**Expected Result**	**Actual Result**	**Status**
1	Loads device list into spinner	userName = “hafiz”	Spinner is populated by list of devices registered by “hafiz”	Spinner is populated by list of devices registered by “hafiz”	Pass
2	Wave loading view loads according to the spinner selection	device01	Loads device01’s current estimated LPG level from Firebase	Loads device01’s current estimated LPG level from Firebase	Pass

3	Loads selected device today's usage	device01	Loads today's usage as cards	Today's usage is loaded as cards	Pass

Table 35. Test case TC_APP_03

Test Case ID	TC_APP_04				
Test Objective	To test the Usage fragment functions				
Test ID	**Test Case**	**Input Data**	**Expected Result**	**Actual Result**	**Status**
1	Loads device list into device list spinner	userName = "hafiz"	Spinner is populated by list of devices registered by "hafiz"	Spinner is populated by list of devices registered by "hafiz"	Pass
2	Loads this week's graph for device01	deviceID = "device01" graphMode = "This Week"	A graph of this week's usage for device01 is loaded succesfully	A graph of this week's usage for device01 is loaded succesfully	Pass
3	Loads usage history cards according to Test ID 2 data.	deviceID = "device01" graphMode = "This Week "	A list of cards loaded into the recycler view successfully	A list of cards loaded into the recycler view successfully	Pass

Table 36. Test case TC_APP_04

Test Case ID	TC_APP_05				
Test Objective	To test the Profile fragment functions				
Test ID	**Test Case**	**Input Data**	**Expected Result**	**Actual Result**	**Status**
1	Loads number of devices registered under the user's userName	userName = "hafiz"	Number of devices is loaded successfully	Number of devices is loaded successfully	Pass
2	Change user's password	oldPassword = "hafiz" newPassword = "hafiz1"	Successfully changed password	Successfully changed password	Pass
3	Add a new device under the user's userName	JSON encoded QR code	Successfully added new device	Successfully added new device	Pass
4	Edit device's settings	deviceName = "Kitchen's Gas" deviceID = "device01" thresholdPercentage = "10" tickerTime = "5 min"	Successfully updated the device's settings	Successfully updated the device's settings	Pass
5	Delete a device under the user's userName	deviceID = "device04"	Successfully deleted the device04	Successfully deleted the device04	Pass
6	Sign out of the application	-	Signed out from the application and redirected to Sign In screen	Signed out from the application and redirected to Sign In screen	Pass

Table 37. Test case TC_APP_05

5.4.2 – RELIABILITY

In reliability testing for application, the functions are tested with unwanted inputs to check if the application capable of handling errors.

Test Objective	To test the reliability of application				
Test ID	**Test Case**	**Input Data**	**Expected Result**	**Actual Result**	**Status**
1	Signing in when there is no internet connection	userName = “hafiz” userPassword = “hafiz	Unable to sign into the application	Unable to sign into the application	Pass
2	Register an account with a registered userName	userName = “hafiz” userEmail = “hafiz@gmail.com” userPassword = “hafiz”	Failed to register the account	Failed to register the account	Pass
3	Signs in without having any registered device on Home fragment	-	Toast message is shown asking the user to register a device	Toast message is shown asking the user to register a device	Pass
4	Loads Usage fragment with no registered device	-	The graph shows “No Data available!” and usage history cards not loaded	The graph shows “No Data available!” and usage history cards not loaded	Pass
5	Scans an invalid QR code	Invalid format for adding device QR code	Toast “Invalid QR code” message and redirected user back to previous screen	Toast “Invalid QR code” message and redirected user back to previous screen	Pass

Table 38. Reliability testing for application

5.4.3 – EFFICIENCY

In efficiency testing for application, only the core functionalities are tested. The execution time is measured by Android Studio's Profiler.

Test Objective	To test the efficiency of application			
Test ID	**Test Case**	**Expected Result**	**Actual Result**	**Status**
1	Signing In	2s	1.01s	Pass
2	Registering an account	2s	1.22s	Pass
3	Loading wave loading view according to the selected deviceID in spinner	2s	1.00s	Pass
4	Loading today's usage cards according to the selected deviceID in spinner	1s	1.01s	Fail
5	Loading selected deviceID's graph for this month	2s	1.55s	Pass
6	Loading selected deviceID's this month usage cards	1s	1.20s	Fail
7	Changing password	2s	0.98s	Pass
8	Adding a new device	2s	1.89s	Pass
9	Changing device settings	2s	1.22s	Pass
10	Deleting a device	2s	1.01s	Pass
11	Signing Out	1s	0.52	Pass

Table 39. Efficiency testing for application

5.5 – SUMMARY

The device, API, and Android application are tested using three types of testings according to ISO 9126 which are functionality, reliability, and efficiency. Functionality testing is done by running specified functions to check whether it performs as expected. For reliability testing, it is to test whether the device, API, and application is capable of handling errors to ensure the process runs smoothly. For efficiency testing, the device, API, and application are tested to check whether they can perform specific function within a specified duration. Throughout the testing process, the device, API, and Android application have performed well in functionality, reliability, and efficiency testing. Even though there are some failures in efficiency testing, it is not because of the device, API, or application itself. It is due to the internet connection during the testing process. Therefore, it can be concluded that the device, API, and Android application satisfies the ISO 9126 software quality standard.

CHAPTER 6 – CONCLUSION AND FUTURE WORKS

6.1 – ACHIEVEMENTS

This project has successfully achieved all the objectives mentioned in Chapter 1. The details of the achievements are as follows:

1. Developed a device that utilizes load cell to measure the estimated LPG level.
2. Developed an Android application that allows the user to view the LPG usage statistics.
3. The application notifies the user when the LPG level reaches below a certain amount of threshold by sending push notification.

6.2 – PROJECT LIMITATIONS

Throughout the project, there are some limitations encountered. One of them is the device prototyping process had to be done using a donut board instead of a proper designed PCB. Besides that, the API is accessible by everyone if they know the URL and its API call. This is not secure because people with malicious intent can steal other user's data. Furthermore, some of the libraries used require an older version of Android SDK. Therefore, it will be difficult to upgrade the application for the newer version of Android phones. Finally, the application runs fully in English language. Therefore, it is difficult for users who do not understand English language to use the application.

6.3 – FUTURE WORK

Based on the limitations mentioned, there are some improvements can be made. Those improvements are:

- Use a PCB to attach the microcontroller and sensor together. This allows the microcontroller and sensor to be arranged and compressed in a more efficient way.
- Use an API key or token to access the API. This is to ensure that only the authorized user can access the API.
- Use other libraries that has the same functionality that allows to be used on a newer version of Android SDK.
- Allows users to change the application's language in the application's settings.

6.4 – CONCLUSION

In conclusion, the main purpose of this project is to provide a way to measure the LPG level in an LPG tank using IoT sensors and to use the readings collected from the sensor to form into a useful information. In this case, the LPG level is measured using a weight sensor called load cell and the readings are displayed in a graph in the application. All the objectives are achieved using the required hardware and software mentioned in Chapter 3. There are some improvements that can be made for this project to provide a more efficient way for the users to use the system.

REFERENCES

Anderson, K. (2019, September 4). *What is Rapid Application Development (RAD)?*. Retrieved from https://blog.capterra.com/what-is-rapid-application-development/

Athuraliya, A. (2019). Sequence Diagram Tutorial: Complete Guide with Examples. *Creately Blog.* Retrieved from https://creately.com/blog/diagrams/sequence-diagram-tutorial/

Computer Hope (2017). *Data flow diagram*. Retrieved from https://www.computerhope.com/jargon/d/data-flow-diagram.htm

Hahn, E. (2019, October 12). *How to Check Gas Bottle Level - Gas Bottle Level Indicator - How to Tell How Full a Gas Bottle Is.* Retrieved from https://www.elgas.com.au/blog/353-how-much-gas-is-left-in-the-gas-bottle

Juvanna, I. (2014). Gas Level Detection and Leakage Monitoring System using a Specific Technique. *International Journal of Computer Science and Mobile Computing, 3*(2), 591-595.

Omega. (n.d.). *Introduction to Load Cells*. Retrieved from https://sea.omega.com/my/prodinfo/loadcells.html

Rouse, M. (2019, July). *What is Internet of Things (IoT)?*. Retrieved from https://internetofthingsagenda.techtarget.com/definition/Internet-of-Things-IoT

Wu, H. P., Hung C., Chang S., Yang Z. (2016). Analysis and Implementation of Noncontact Level Sensing for a Pressurized Cylinder. *Journal of Sensors, 2016*(5027916), 1-10.

FIGURE REFERENCES

Hahn, E. (2019). The conventional technique of measuring gas level in a gas cylinder. Retrieved from https://www.elgas.com.au/blog/353-how-much-gas-is-left-in-the-gas-bottle

Juvanna, I., Meenakshi, N. (2014). *Overview diagram of the pressure sensor system*. Padur, Chennai: International Journal of Computer Science and Mobile Computing

Juvanna, I., Meenakshi, N. (2014). *LPG tank with RFID device*. Padur, Chennai: International Journal of Computer Science and Mobile Computing.

Loginov, D. (2018). *LoLin NodeMCU ESP8266 v3*, Arduino ecosystem and PlatformIO IDE. Retrieved from https://medium.com/@loginov_rocks/quick-start-with-nodemcu-v3-esp8266-arduino-ecosystem-and-platformio-ide-b8415bf9a038

Lucidchart Content Team. (2018, May 23). *Rapid Application Development (RAD)*[methodology diagram]. Retrieved from https://www.lucidchart.com/blog/rapid-application-development-methodology

PlatformIO. (2019). *PlatformIO IDE*. Retrieved from https://platformio.org/

Wu H. P., Hung C., Chang S., Yang Z. (2016). *Contents inside the device used to measure the gas level and how it's installed on a LPG tank.* Taiwan:Journal of Sensors.

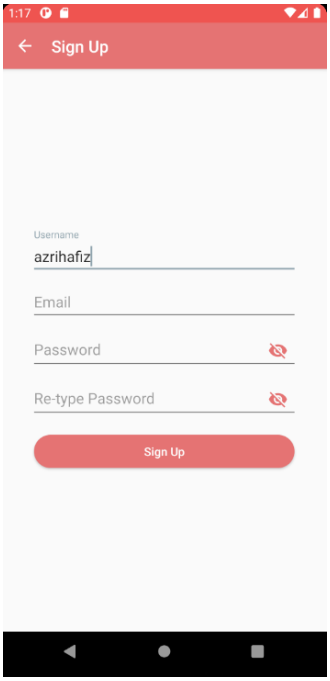
Wu H. P., Hung C., Chang S., Yang Z. (2016). *Knocking mechanism from pulling to pushing action.* Taiwan:Journal of Sensors.

APPENDIX

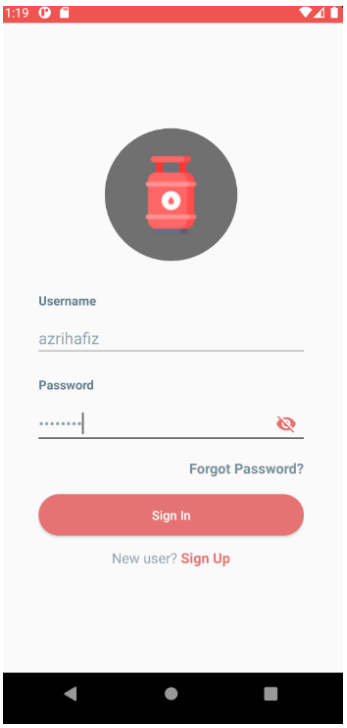
User Manual (LPG IoT Application)

1. Install the .apk file.

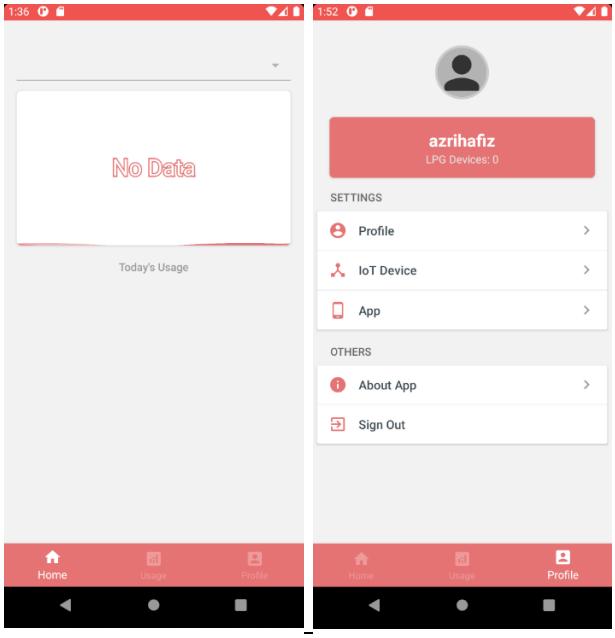
2. Create an account by tapping "Sign Up".



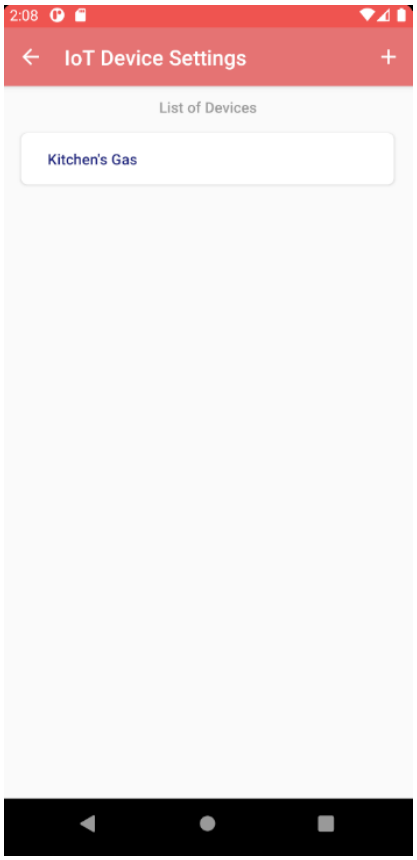
3. Sign in using the registered account



4. Add device by clicking the “IoT Settings” under Profile fragment

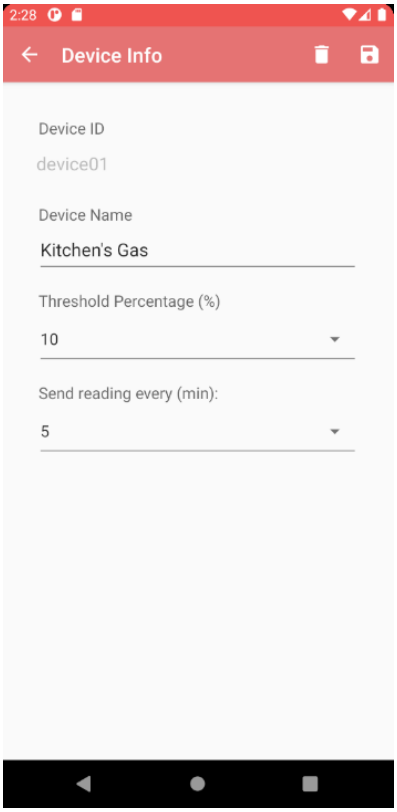


5. Click the add icon to add a device. Scan the device’s QR code. Once the QR code is scanned, you should be able to see your device in the list.

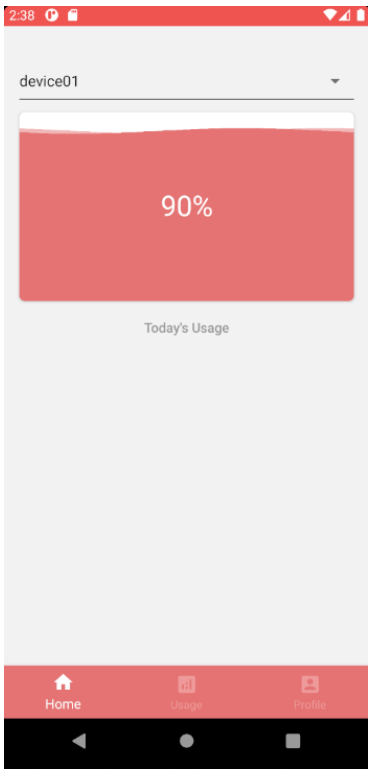


6. (Optional) You can change the device's settings according to what you want. "Threshold Percentage" refers to at below what percentage should the user gets notified if the gas level reaches that threshold percentage. As for "Send reading", it refers to every how many minutes should the device send its reading.

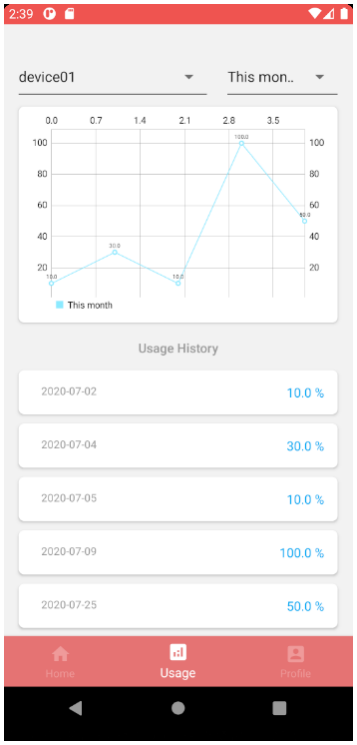
Click the save icon on top-right of the screen to save the settings. To delete the device, simply press the delete icon. You will be prompted for confirmation.



7. The estimated gas level for the registered device should appear on the home screen



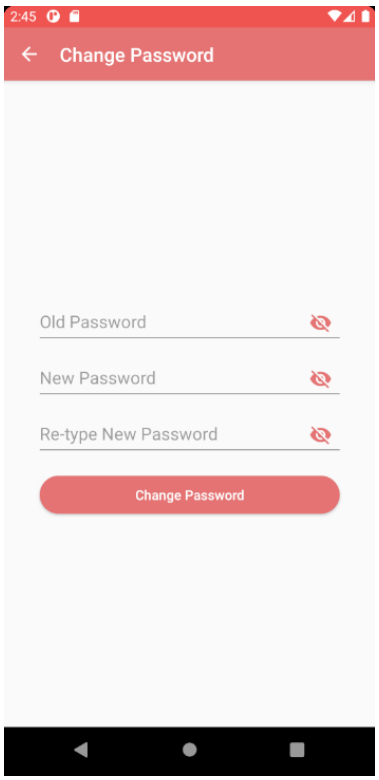
8. To view the usage statistics, simply tap the “Usage” button on the BottomNavigation panel.



Change Password

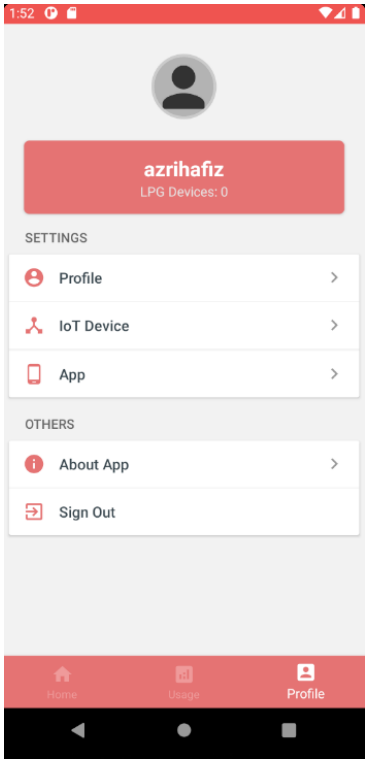
1. To change password, go to Profile > Profile.

Fill in the old password and your new password. Then click the “Change Password” button.



Sign Out

1. Go to Profile.
2. Click “Sign Out” button



IoT Device source code

Variables & Object declarations:

```
FirebaseData firebaseData;
FirebaseJson firebaseJson;
HX711_ADC LoadCell(D4, D5);
Ticker ticker;

//Load Cell & Gas Cylinder settings
const float calibrationFactor = 696.0;
const float fullWeight = 275.0;
const float emptyWeight = 5.0;
const float contentPerPercent = (fullWeight - emptyWeight)/100;
int readingVal;
int previousVal = 0;
long t;

//Default Device settings (will be changed according to data fetched online)
int thresholdPercentage = 10;
int tickerTime = 300; //300 seconds = 5 minutes
bool allowSendReading = false;
bool pushNotificationSent = false;

String httpPOSTRequest(String, String);
void toggleSendReading();
void sendReading();
void sendPushNotification();
void printReadings(float, int, int);
```

toggleSendReading() function:

```
/**
 * Toggles boolean value since ticker only accept function as parameter.
**/
void toggleSendReading() { allowSendReading = true; }
```

sendReading() function:

```
/**
 * Sends current reading to database online.
**/
void sendReading() {
    if (WiFi.status() != WL_CONNECTED) {
        Serial.println("[SEND_READING] Error sending reading! Reason: Wifi dis
connected!");
    }
    else {
        Serial.println();
        Serial.println("[SEND_READING] Sending reading to MySQL database...");

        String readingPercentage = "readingValue=" + String(readingVal) + "&de
viceID=" + DEVICE_ID;
        String reqResponse = httpPOSTRequest(DEVICE_SEND_READINGS, readingPerc
entage);

        size_t capacity = JSON_OBJECT_SIZE(2) + 50;
        DynamicJsonDocument doc(capacity);
        deserializeJson(doc, reqResponse);

        String message = doc["message"];
        Serial.println(message);

        Serial.println();
        Serial.println("[SEND_READING] Sending reading to Firebase Realtime da
tabase...");
        String path = DEVICE_ID + "/";
        if (Firebase.setInt(firebaseData, path + "gas_level", readingVal)) {
            Serial.println("[SEND_READING] Successfully sent reading to Fireba
se.");
        }
        else {
            Serial.println("[SEND_READING] Failed to send reading to Firebase!
");
            Serial.println("[SEND_READING] Reason: " + firebaseData.errorReaso
n());
        }

        if (readingVal <= thresholdPercentage && !pushNotificationSent) {
            sendPushNotification();
            pushNotificationSent = true;
        }
    }
}
```

sendPushNotification() function:

```
/**
 * Sends push notification to Firebase Cloud Messaging service
**/
void sendPushNotification() {
    if (WiFi.status() != WL_CONNECTED) {
        Serial.println("[PUSH_NOTIFICATION] Error sending reading! Reason: Wif
i disconnected!");
    }
    else {
        Serial.println("[PUSH_NOTIFICATION] Sending push notification...");
        firebaseData.fcm.setNotifyMessage("LPG IoT", "Your " + DEVICE_ID +" LP
G level has reached below " + String(thresholdPercentage) + "%");
        firebaseData.fcm.setTopic("thresholdAlert");

        if (Firebase.sendTopic(firebaseData)) {
            Serial.println("[PUSH_NOTIFICATION]" + firebaseData.fcm.getSendRes
ult());
        }
        else {
            Serial.println("[PUSH_NOTIFICATION]" + firebaseData.errorReason())
;
        }
    }
}
```

printReadings() function:

```
/**
 * Prints out the actual, rounded off and percentage readings. For debugging p
urposes.
**/
void printReadings(float actualVal, int roundedVal, int percentageVal) {
    Serial.print("[OUTPUT] Output val: ");
    Serial.print(actualVal);
    Serial.print("  Rounded val: ");
    Serial.println(roundedVal);
    Serial.print("[OUTPUT] Percentage: ");
    Serial.print(percentageVal);
    Serial.println("%");
    Serial.println();
}
```

httpPOSTRequest() function:

```
/**
 * Sends http POST request to specified url with paramaters.
 * @param url - URL of the specified target
 * @param params - Parameters for sending data
 * @return String - JSON response from server.
 *
**/
String httpPOSTRequest(String url, String params) {
    HTTPClient http;

    http.begin(url, API_URL_FINGERPRINT);
    http.addHeader("Content-Type", "application/x-www-form-urlencoded");

    int httpResponse = http.POST(params);
    String payload = "{}";

    if (httpResponse > 0) {
        Serial.println("[HTTP] Successfully executed POST request!");
        Serial.print("[HTTP] Response Code: ");
        Serial.println(httpResponse);
        Serial.print("[HTTP] URL: ");
        Serial.println(url);
        payload = http.getString();
    }
    else {
        Serial.print("[HTTP] Error: ");
        Serial.println(http.errorToString(httpResponse));
    }

    http.end();
    return payload;
}
```

setup() function:

```
void setup() {
    Serial.begin(9600); delay(10);
    Serial.println();
    Serial.println("[BOOT] Device booting up...");

    //Connecting to WiFi
    unsigned long startWifi = micros();
    WiFi.begin(WIFI_SSID, WIFI_PASS);
    Serial.print("[WIFI] Connecting to ");
    Serial.print(WIFI_SSID);
    while (WiFi.status() != WL_CONNECTED) {
        Serial.print(".");
        delay(300);
    }
    Serial.println();
    Serial.print("[WIFI] Connected! IP Address: ");
    Serial.println(WiFi.localIP());
    Serial.println();

    //Getting device settings on cloud
    if (WiFi.status() != WL_CONNECTED) {
        Serial.println("[DEVICE_SETTINGS] Error connecting to WiFi, using defa
ult settings..");
    }
    else {
        Serial.println("[DEVICE_SETTINGS] Fetching settings from online databa
se..");
        String deviceID = "deviceID=" + DEVICE_ID;
        String reqResponse = httpPOSTRequest(DEVICE_GET_SETTINGS, deviceID);

        size_t capacity = JSON_ARRAY_SIZE(2) + JSON_OBJECT_SIZE(3) + 110;
        DynamicJsonDocument doc(capacity);
        deserializeJson(doc, reqResponse);

        if (doc["deviceSettings"]["thresholdPercentage"] != NULL && doc["devic
eSettings"]["tickerTime"] != NULL) {
            String message = doc["message"];
            thresholdPercentage = doc["deviceSettings"]["thresholdPercentage"]
;
            tickerTime = doc["deviceSettings"]["tickerTime"];

            Serial.print("[DEVICE_SETTINGS] ");
            Serial.println(message);
        }
        else {
            Serial.println("[DEVICE_SETTINGS] Unable to fetch device settings,
 using default settings.");
        }
```

```
        Serial.print("[DEVICE_SETTINGS] Threshold Percentage (%): ");
        Serial.println(thresholdPercentage);
        Serial.print("[DEVICE_SETTINGS] Ticker Time (s): ");
        Serial.println(tickerTime);
        Serial.println("[DEVICE_SETTINGS] Done!");
    }
    Serial.println();

    //Initializing Firebase
    Serial.println("[FIREBASE] Setting up firebase..");
    Firebase.begin(FIREBASE_HOST, FIREBASE_AUTH);
    Firebase.reconnectWiFi(true);
    firebaseData.setBSSLBufferSize(1024, 1024);
    firebaseData.setResponseSize(1024);
    Firebase.setReadTimeout(firebaseData, 1000 * 60);
    Firebase.setwriteSizeLimit(firebaseData, "tiny");
    firebaseData.fcm.begin(FIREBASE_FCM_SERVER_KEY);
    firebaseData.fcm.setPriority("high");
    firebaseData.fcm.setTimeToLive(1000);
    Serial.println("[FIREBASE] Done!");
    Serial.println();


    //Initializing Load Cell
    Serial.println("[LOAD_CELL] Setting up load cell..");
    long stabilisingtime = 2000;
    boolean _tare = true;
    LoadCell.begin();
    LoadCell.start(stabilisingtime, _tare);

    if (LoadCell.getTareTimeoutFlag()) {
        Serial.println("[LOAD_CELL] Tare timeout, check MCU>HX711 wiring and p
in designations");
    }
    else {
        LoadCell.setCalFactor(calibrationFactor);
        Serial.println("[LOAD_CELL] Done!");
    }
    Serial.println();

    //Send Reading every "tickerTime" seconds
    ticker.attach(tickerTime, toggleSendReading);

    Serial.println("[BOOT] Booting up completed! Starting device..");
    delay(1500);
    Serial.println();
}
```

loop() function:

```
void loop() {
    static boolean newDataReady = 0;
    const int readingInterval = 50;

    if(LoadCell.update()) newDataReady = true;

    if (newDataReady) {
        if (millis() > t + readingInterval) {
            float i = LoadCell.getData();
            int roundedVal = round(i);

            if (roundedVal >= fullWeight) {
                readingVal = 100;
            }
            else if (roundedVal <= emptyWeight) {
                readingVal = 0;
            }
            else {
                readingVal = (roundedVal/contentPerPercent) - (emptyWeight/con
tentPerPercent);
            }

            newDataReady = 0;
            t = millis();
        }
    }

    if (Serial.available() > 0) {
        float i;
        char inByte = Serial.read();
        if (inByte == 't') LoadCell.tareNoDelay();
    }

    if (LoadCell.getTareStatus() == true) {
        Serial.println("Tare complete");
    }

    if (allowSendReading) {
        sendReading();
        allowSendReading = false;
    }
}
```

PHP REST API source code

Connect.php:

```php
<?php

define('host', 'localhost');
define('user', 'mydb');
define('pass', 'mydb123');
define('database', 'mydb');

class Connect {
    private $con;

    function __construct() { }

    function connect() {
        $this->con = new mysqli(host, user, pass, database);

        if (mysqli_connect_errno()) {
            echo "Failed to connect to MySQL: " . mysqli_connect_error();
        }
        return $this->con;
    }
}
```

API.php:

```php
<?php
require_once 'APIOperation.php';

function checkParams($params) {
    $available = true;
    $missingparams = "";
    foreach($params as $param){
        if(!isset($_POST[$param]) || strlen($_POST[$param])<=0) {
            $available = false;
            $missingparams = $missingparams . ", " . $param;
        }
    }
    if(!$available) {
        $response = array();
        $response['error'] = true;
        $response['message'] = 'Parameters' . substr($missingparams, 1, strlen
($missingparams)) . ' missing';
        echo json_encode($response);
        die();
    }
}
```

```
$response = array();

if(isset($_GET['apicall'])) {

    switch($_GET['apicall']) {

        case 'signin':
            checkParams(array('userName', 'userPassword'));
            $db = new APIOperation();

            $result = $db->signIn(
                $_POST['userName'],
                $_POST['userPassword']
            );

            if ($result) {
                $response['error'] = false;
                $response['message'] = "Successfully signed in!";
                $response['user'] = $db->getUserDetails($_POST['userName']);
            }
            else {
                $response['error'] = true;
                $response['message'] = "Failed to sign in!";
            }
        break;

        case 'signup':
            checkParams(array('userName', 'userEmail', 'userPassword'));
            $db = new APIOperation();

            $result = $db->signUp(
                $_POST['userName'],
                $_POST['userEmail'],
                $_POST['userPassword']
            );

            if ($result) {
                $response['error'] = false;
                $response['message'] = "Successfully signed up!";
            }
            else {
                $response['error'] = true;
                $response['message'] = "Failed to sign up!";
            }
        break;

        case 'getdevicesettings':
            checkParams(array('deviceID'));
```

```
            $db = new APIOperation();

            $response['error'] = false;
            $response['message'] = 'Successfully fetched '.$_POST['deviceID'].
' settings';
            $response['deviceSettings'] = $db-
>getDeviceSettings($_POST['deviceID']);
        break;

        case 'sendreadings':
            checkParams(array('readingValue', 'deviceID'));
            $db = new APIOperation();

            $result = $db->sendReadings(
                $_POST['readingValue'],
                $_POST['deviceID']
            );

            if ($result) {
                $response['error'] = false;
                $response['message'] = "Successfully sent the reading";
            }
            else {
                $response['error'] = true;
                $response['message'] = "Failed to send the reading";
            }
        break;

        case 'adddevice':
            checkParams(array('deviceID', 'userName'));
            $db = new APIOperation();

            $result = $db->addDevice(
                $_POST['deviceID'],
                $_POST['userName']
            );

            if ($result) {
                $response['error'] = false;
                $response['message'] = "Successfully added the device";
            }
            else {
                $response['error'] = true;
                $response['message'] = "Failed to add the device";
            }
        break;

        case 'getdevicelist':
```

```
            checkParams(array('userName'));
            $db = new APIOperation();

            $response['error'] = false;
            $response['message'] = "Successfully fetched the device list for "
.$_POST['userName'];
            $response['deviceList'] = $db->getDeviceList($_POST['userName']);
        break;

        case 'checkoldpassword':
            checkParams(array('oldPassword', 'userName'));
            $db = new APIOperation();

            $result = $db->checkOldPassword(
                $_POST['oldPassword'],
                $_POST['userName']
            );

            if ($result) {
                $response['error'] = false;
                $response['message'] = "oldPassword and userName matches!";
            }
            else {
                $response['error'] = true;
                $response['message'] = "Incorrect old password!";
            }
        break;

        case 'changepassword':
            checkParams(array('newPassword', 'userName'));
            $db = new APIOperation();

            $result = $db->changePassword(
                $_POST['newPassword'],
                $_POST['userName']
            );

            if ($result) {
                $response['error'] = false;
                $response['message'] = "Successfully changed password!";
            }
            else {
                $response['error'] = true;
                $response['message'] = "Failed to change password!";
            }
        break;

        case 'getusagevalues':
```

```
            checkParams(array('deviceID', 'usageMode'));
            $db = new APIOperation();

            $response['error'] = false;
            $response['message'] = 'Successfully fetched '.$_POST['deviceID'].
' usage values';
            $response['usageValues'] = $db-
>getUsageValues($_POST['deviceID'], $_POST['usageMode']);
        break;

        case 'savedeviceinfo':
            checkParams(array('deviceID', 'thresholdPercentage', 'tickerTime',
 'deviceName'));
            $db = new APIOperation();

            $result = $db->saveDeviceInfo(
                $_POST['deviceID'],
                $_POST['thresholdPercentage'],
                $_POST['tickerTime'],
                $_POST['deviceName']
            );

            if ($result) {
                $response['error'] = false;
                $response['message'] = "Successfully saved device info!";
            }
            else {
                $response['error'] = true;
                $response['message'] = "Failed to save device info!";
            }
        break;

        case 'deletedevice':
            checkParams(array('deviceID'));
            $db = new APIOperation();

            $result = $db->deleteDevice(
                $_POST['deviceID']
            );

            if ($result) {
                $response['error'] = false;
                $response['message'] = "Successfully deleted device!";
            }
            else {
                $response['error'] = true;
                $response['message'] = "Failed to delete device!";
            }
```

```
        break;

        default:
            $response['error'] = true;
            $response['message'] = 'Invalid API Call';
        break;
    }
}
else {
    $response['error'] = true;
    $response['message'] = 'Invalid API Call';
}

echo json_encode($response);
```

APIOperation.php:

```
<?php
date_default_timezone_set('Asia/Kuching');

class APIOperation {

    private $con;

    function __construct() {
        require_once 'Connect.php';

        $db = new Connect();
        $this->con = $db->connect();
    }

    function signIn($userName, $userPassword) {
        $stmt = $this->con-
>prepare("SELECT * FROM users WHERE userName=? AND userPassword=?");
        $userPassword = md5($userPassword);
        $stmt->bind_param("ss", $userName, $userPassword);
        $stmt->execute();
        $stmt->store_result();

        if ($stmt->num_rows == 1)
            return true;

        return false;
    }
```

```
function signUp($userName, $userEmail, $userPassword) {
        $stmt = $this->con-
>prepare("INSERT INTO users (userName, userEmail, userPassword) VALUES (?, ?,
?)");
        $userPassword = md5($userPassword);
        $stmt->bind_param("sss", $userName, $userEmail, $userPassword);

        if ($stmt->execute())
            return true;

        return false;
    }

    function getUserDetails($userName) {
        $stmt = $this->con-
>prepare("SELECT userName, userEmail FROM users WHERE userName=?");
        $stmt->bind_param("s", $userName);
        $stmt->execute();

        $stmt->bind_result($userName, $userEmail);
        $stmt->fetch();

        $user = array(
            'userName'=>$userName,
            'userEmail'=>$userEmail
        );

        return $user;
    }

    function getDeviceSettings($deviceID) {
        $stmt = $this->con-
>prepare("SELECT thresholdPercentage, tickerTime FROM devices WHERE deviceID=?
");
        $stmt->bind_param("s", $deviceID);
        $stmt->execute();
        $stmt->bind_result($thresholdPercentage, $tickerTime);
        $stmt->fetch();

        $deviceSettings = array(
                'thresholdPercentage'=>$thresholdPercentage,
                'tickerTime'=>$tickerTime
        );

        return $deviceSettings;
    }

    function sendReadings($readingValue, $deviceID) {
```

```
        $stmt = $this->con-
>prepare("INSERT INTO readings (readingValue, readingTime, deviceID) VALUES (?
, ?, ?)");
        $currentDateTime = date("Y-m-d H:i:s");
        $stmt->bind_param("iss", $readingValue, $currentDateTime, $deviceID);

        if ($stmt->execute())
            return true;

        return false;
    }

    function addDevice($deviceID, $userName) {
        $stmt = $this->con-
>prepare("INSERT INTO devices (deviceID, thresholdPercentage, tickerTime, user
Name) VALUES (?, ?, ?, ?)");
        $defaultThresholdValue = 10;
        $defaultTickerValue = 300;
        $stmt-
>bind_param("siis", $deviceID, $defaultThresholdValue, $defaultTickerValue, $u
serName);

        if ($stmt->execute())
            return true;

        return false;
    }

    function getDeviceList($userName) {
        $stmt = $this->con-
>prepare("SELECT deviceID, thresholdPercentage, tickerTime, deviceName FROM de
vices WHERE userName=?");
        $stmt->bind_param("s", $userName);
        $stmt->execute();
        $stmt-
>bind_result($deviceID, $thresholdPercentage, $tickerTime, $deviceName);

        $deviceList = array();
        while ($stmt->fetch()) {
            $deviceData = array(
                'deviceID'=>$deviceID,
                'thresholdPercentage'=>$thresholdPercentage,
                'tickerTime'=>$tickerTime,
                'deviceName'=>$deviceName
            );
            array_push($deviceList, $deviceData);
        }
        return $deviceList;
```

```
    }

    function checkOldPassword($oldPassword, $userName) {
        $stmt = $this->con-
>prepare("SELECT * FROM users WHERE userName=? AND userPassword=?");
        $oldPassword = md5($oldPassword);
        $stmt->bind_param("ss", $userName, $oldPassword);
        $stmt->execute();
        $stmt->store_result();

        if ($stmt->num_rows == 1)
            return true;

        return false;
    }

    function changePassword($newPassword, $userName) {
        $stmt = $this->con-
>prepare("UPDATE users SET userPassword=? WHERE userName=?");
        $newPassword = md5($newPassword);
        $stmt->bind_param("ss", $newPassword, $userName);

        if ($stmt->execute())
            return true;

        return false;
    }

    function getUsageValues($deviceID, $usageMode) {
        $usageList = array();

        if ($usageMode == "0") { //Today
            $todayDate = date("Y-m-d");
            $stmt = $this->con-
>prepare("SELECT readingValue, readingTime FROM readings WHERE deviceID=? AND
readingTime BETWEEN '".$todayDate." 00:00:00' AND '".$todayDate." 23:59:59'");
            $stmt->bind_param("s", $deviceID);
            $stmt->execute();
            $stmt->bind_result($readingValue, $readingTime);

            $unsortedUsageData = array();
            while ($stmt->fetch()) {
                $usageData = array(
                    'readingValue'=>$readingValue,
                    'readingTime'=>$readingTime
                );
                array_push($unsortedUsageData, $usageData);
            }
```

```
            for ($i=0; $i<24; $i++) {
                $start = $i.":00:00";
                $end = $i.":59:59";
                $usageData = array(
                    'readingValue'=>$this-
>calculateAverageReadings($unsortedUsageData, $todayDate, $start, $todayDate,
$end),
                    'readingTime'=>$start
                );

                if ($usageData['readingValue'] != 0)
                    array_push($usageList, $usageData);
            }
        }
        else if ($usageMode == "1") { //This week
            $todayDate = strtotime("today");
            $start_week = strtotime("last monday midnight",$todayDate);
            $end_week = strtotime("next sunday",$todayDate);
            $startDate = date("Y-m-d",$start_week);
            $endDate = date("Y-m-d",$end_week);

            $stmt = $this->con-
>prepare("SELECT readingValue, readingTime FROM readings WHERE deviceID=? AND
readingTime BETWEEN '".$startDate." 00:00:00' AND '".$endDate." 23:59:59'");
            $stmt->bind_param("s", $deviceID);
            $stmt->execute();
            $stmt->bind_result($readingValue, $readingTime);

            $unsortedUsageData = array();
            while ($stmt->fetch()) {
                $usageData = array(
                    'readingValue'=>$readingValue,
                    'readingTime'=>$readingTime
                );
                array_push($unsortedUsageData, $usageData);
            }

            $date = $start_week;
            for ($i=0; $i<7; $i++) {
                $date2 = date("Y-m-d",$date);
                $usageData = array(
                    'readingValue'=>$this-
>calculateAverageReadings($unsortedUsageData, $date2, "00:00:00", $date2, "23:
59:59"),
                    'readingTime'=>date("l", $date)
                );
                $date = strtotime("+1 day", $date);
```

```
                if ($usageData['readingValue'] != 0)
                    array_push($usageList, $usageData);
            }
        }
        else if ($usageMode == "2") { // This month
            $startDate = date("Y-m-1");
            $endDate = date("Y-m-t");

            $stmt = $this->con-
>prepare("SELECT readingValue, readingTime FROM readings WHERE deviceID=? AND
readingTime BETWEEN '".$startDate." 00:00:00' AND '".$endDate." 23:59:59'");
            $stmt->bind_param("s", $deviceID);
            $stmt->execute();
            $stmt->bind_result($readingValue, $readingTime);

            $unsortedUsageData = array();
            while ($stmt->fetch()) {
                $usageData = array(
                    'readingValue'=>$readingValue,
                    'readingTime'=>$readingTime
                );
                array_push($unsortedUsageData, $usageData);
            }

            $maxDay = date("t");
            $date = strtotime($startDate);
            for ($i=0; $i<$maxDay; $i++) {
                $date2 = date("Y-m-d", $date);
                $usageData = array(
                    'readingValue'=>$this-
>calculateAverageReadings($unsortedUsageData, $date2, "00:00:00", $date2, "23:
59:59"),
                    'readingTime'=>$date2
                );
                $date = strtotime("+1 day", $date);

                if ($usageData['readingValue'] != 0)
                    array_push($usageList, $usageData);
            }
        }
        else if ($usageMode == "3") { //This year
            $startDate = date("Y-01-1");
            $endDate = date("Y-12-31");

            $stmt = $this->con-
>prepare("SELECT readingValue, readingTime FROM readings WHERE deviceID=? AND
readingTime BETWEEN '".$startDate." 00:00:00' AND '".$endDate." 23:59:59'");
```

```
            $stmt->bind_param("s", $deviceID);
            $stmt->execute();
            $stmt->bind_result($readingValue, $readingTime);

            $unsortedUsageData = array();
            while ($stmt->fetch()) {
                $usageData = array(
                    'readingValue'=>$readingValue,
                    'readingTime'=>$readingTime
                );
                array_push($unsortedUsageData, $usageData);
            }

            $date = strtotime($startDate);
            $end = strtotime(date("Y-m-t", strtotime($startDate)));
            for ($i=0; $i<12; $i++) {
                $date2 = date("Y-m-d", $date);
                $end2 = date("Y-m-d", $end);
                $usageData = array(
                    'readingValue'=>$this-
>calculateAverageReadings($unsortedUsageData, $date2, "00:00:00", $end2, "23:5
9:59"),
                    'readingTime'=>date("F", $date)
                );
                $date = strtotime("+1 month", $date);
                $end = strtotime("+1 month", $end);

                if ($usageData['readingValue'] != 0)
                    array_push($usageList, $usageData);
            }
        }

        return $usageList;
    }

    function saveDeviceInfo($deviceID, $thresholdPercentage, $tickerTime, $dev
iceName) {
        $stmt = $this->con-
>prepare("UPDATE devices SET thresholdPercentage=?, tickerTime=?, deviceName=?
 WHERE deviceID=?");
        $intThresholdPercentage = (int)$thresholdPercentage;
        $intTickerTime = (int)$tickerTime * 60;
        $stmt-
>bind_param("iiss", $intThresholdPercentage, $intTickerTime, $deviceName, $dev
iceID);

        if ($stmt->execute())
            return true;
```

```
        return false;
    }

    function deleteDevice($deviceID) {
        $stmt = $this->con->prepare("DELETE FROM devices WHERE deviceID=?");
        $stmt->bind_param("s", $deviceID);

        $stmt2 = $this->con->prepare("DELETE FROM readings WHERE deviceID=?");
        $stmt2->bind_param("s", $deviceID);

        if ($stmt->execute() && $stmt2->execute())
            return true;

        return false;
    }

    function calculateAverageReadings($unsortedUsageData, $startDate, $startTi
me, $endDate, $endTime) {
        $reading = 0;
        $count = 0;
        $startTime2 = new DateTime($startDate." ".$startTime);
        $endTime2 = new DateTime($endDate." ".$endTime);
        for ($i=0; $i<sizeof($unsortedUsageData); $i++) {
            $arrData = new DateTime($unsortedUsageData[$i]['readingTime']);
            if ($arrData->getTimestamp() > $startTime2-
>getTimestamp() && $arrData->getTimestamp() < $endTime2->getTimestamp()) {
                $reading += $unsortedUsageData[$i]['readingValue'];
                $count++;
            }
        }

        if ($count != 0)
            $reading = $reading / $count;
        else
            $reading = 0;

        return $reading;
    }
}
```

Android application source code

SignInActivity class:

```
public class SignInActivity extends AppCompatActivity {

    private TextInputEditText textInputEditText_userName,
textInputEditText_password;
    SessionManager userSession;

    @Override
    protected void onCreate(@Nullable Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_signin);

        userSession = new SessionManager(getApplicationContext());
        if (userSession.checkIsUserSignedIn()) {
            Intent intent = new Intent(getApplicationContext(),
MainActivity.class);
            intent.addFlags(Intent.FLAG_ACTIVITY_CLEAR_TOP);
            intent.setFlags(Intent.FLAG_ACTIVITY_NEW_TASK);
            startActivity(intent);
            finish();
        }

        textInputEditText_userName =
findViewById(R.id.TextInputEditText_userName);
        textInputEditText_password =
findViewById(R.id.TextInputEditText_userPassword);

        findViewById(R.id.Button_signIn).setOnClickListener(new
View.OnClickListener() {
            @Override
            public void onClick(View view) {
                String userName =
textInputEditText_userName.getText().toString().trim();
                String userPassword =
textInputEditText_password.getText().toString().trim();

                if (userName.length() <= 0 || userPassword.length() <= 0)
                    Toast.makeText(SignInActivity.this, "Please fill in all the
text boxes!", Toast.LENGTH_SHORT).show();
                else
                    signIn(userName, userPassword);
            }
        });

        findViewById(R.id.TextView_signUp).setOnClickListener(new
View.OnClickListener() {
            @Override
            public void onClick(View view) {
                startActivity(new Intent(getApplicationContext(),
SignUpActivity.class));
            }
        });
    }
```

```
public void signIn(final String userName, final String userPassword) {
        StringRequest stringRequest = new StringRequest(Request.Method.POST,
API.URL_SIGNIN, new Response.Listener<String>() {
            @Override
            public void onResponse(String response) {
                try {
                    JSONObject jsonResponseObj = new JSONObject(response);

                    if (!jsonResponseObj.getBoolean("error")) {
                        Toast.makeText(SignInActivity.this,
jsonResponseObj.getString("message"), Toast.LENGTH_SHORT).show();

                        JSONObject jsonUserObj =
jsonResponseObj.getJSONObject("user");

userSession.createSignInSession(jsonUserObj.getString("userName"),
jsonUserObj.getString("userEmail"));

                        finish();
                        startActivity(new Intent(getApplicationContext(),
MainActivity.class));
                    }
                    else {
                        Toast.makeText(SignInActivity.this,
jsonResponseObj.getString("message"), Toast.LENGTH_SHORT).show();
                    }
                } catch (JSONException e) {
                    e.printStackTrace();
                }
            }
        }, new Response.ErrorListener() {
            @Override
            public void onErrorResponse(VolleyError error) {
                Toast.makeText(SignInActivity.this, error.getMessage(),
Toast.LENGTH_SHORT).show();
            }
        }) {
            @Override
            protected Map<String, String> getParams() {
                Map<String, String> params = new HashMap<>();
                params.put("userName", userName);
                params.put("userPassword", userPassword);
                return params;
            }
        };
        VolleySingleton.getInstance(this).addToRequestQueue(stringRequest);
    }
}
```

SignUpActivity class:

```
public class SignUpActivity extends AppCompatActivity {

    private AutoCompleteTextView autoCompleteTextView_userName,
autoCompleteTextView_userEmail;
    private EditText editText_password, editText_confirmPassword;

    @Override
    protected void onCreate(@Nullable Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_signup);
        initToolbar();

        autoCompleteTextView_userName =
findViewById(R.id.AutoCompleteTextView_userName);
        autoCompleteTextView_userEmail =
findViewById(R.id.AutoCompleteTextView_userEmail);
        editText_password = findViewById(R.id.EditText_password);
        editText_confirmPassword = findViewById(R.id.EditText_confirmPassword);

        findViewById(R.id.Button_signUp).setOnClickListener(new
View.OnClickListener() {
            @Override
            public void onClick(View view) {
                String userName =
autoCompleteTextView_userName.getText().toString().trim();
                String userEmail =
autoCompleteTextView_userEmail.getText().toString().trim();
                String userPassword =
editText_password.getText().toString().trim();
                String userConfirmPassword =
editText_confirmPassword.getText().toString().trim();

                if (userName.length() <= 0 || userEmail.length() <= 0 ||
userPassword.length() <= 0 || userConfirmPassword.length() <= 0)
                    Toast.makeText(SignUpActivity.this, "Please fill in all the
text boxes!", Toast.LENGTH_SHORT).show();
                else {
                    if (userPassword.equals(userConfirmPassword))
                        signUp(userName, userEmail, userPassword);
                    else
                        Toast.makeText(SignUpActivity.this, "The password and
confirm password does not match!", Toast.LENGTH_SHORT).show();
                }
            }
        });
    }

    private void initToolbar() {
        Toolbar toolbar = findViewById(R.id.toolbar);
        setSupportActionBar(toolbar);
        getSupportActionBar().setTitle("Sign Up");
        getSupportActionBar().setDisplayHomeAsUpEnabled(true);
        Tools.setSystemBarColor(this);
    }
```

```
private void signUp(final String userName, final String userEmail, final String
userPassword) {
        StringRequest stringRequest = new StringRequest(Request.Method.POST,
API.URL_SIGNUP, new Response.Listener<String>() {
            @Override
            public void onResponse(String response) {
                try {
                    JSONObject jsonResponseObj = new JSONObject(response);
                    if (!jsonResponseObj.getBoolean("error")) {
                        Toast.makeText(SignUpActivity.this,
jsonResponseObj.getString("message"), Toast.LENGTH_SHORT).show();
                        finish();
                        startActivity(new Intent(getApplicationContext(),
SignInActivity.class));
                    }
                    else {
                        Toast.makeText(SignUpActivity.this,
jsonResponseObj.getString("message"), Toast.LENGTH_SHORT).show();
                    }
                } catch (JSONException e) {
                    e.printStackTrace();
                }
            }
        }, new Response.ErrorListener() {
            @Override
            public void onErrorResponse(VolleyError error) {
                Toast.makeText(SignUpActivity.this, error.getMessage(),
Toast.LENGTH_SHORT).show();
            }
        }) {
            @Override
            protected Map<String, String> getParams() {
                Map<String, String> params = new HashMap<>();
                params.put("userName", userName);
                params.put("userEmail", userEmail);
                params.put("userPassword", userPassword);
                return params;
            }
        };
        VolleySingleton.getInstance(this).addToRequestQueue(stringRequest);
    }
}
```

HomeFragment class:

```
public class HomeFragment extends Fragment {

    private RecyclerView recyclerView;
    private AppCompatSpinner appCompatSpinner_device;
    private WaveLoadingView waveLoadingView_gasLevelPercentage;

    List<UsageHome> usageHomeList;
    HashMap<String, String> userDetails;

    @Nullable
    @Override
    public View onCreateView(@NonNull LayoutInflater inflater, @Nullable ViewGroup
container, @Nullable Bundle savedInstanceState) {
        View view = inflater.inflate(R.layout.fragment_home, container, false);
        recyclerView = view.findViewById(R.id.recyclerView);
        appCompatSpinner_device = view.findViewById(R.id.AppCompatSpinner_device);
        waveLoadingView_gasLevelPercentage =
view.findViewById(R.id.WaveLoadingView_gasLevelPercentage);

        appCompatSpinner_device.setEnabled(false);

        SessionManager userSession = new SessionManager(getContext());
        userDetails = userSession.getUserDetails();
        usageHomeList = new ArrayList<>();

        appCompatSpinner_device.setOnItemSelectedListener(new
AdapterView.OnItemSelectedListener() {
            @Override
            public void onItemSelected(AdapterView<?> parent, View view, int
position, long id) {

loadWaveLoadingView(appCompatSpinner_device.getSelectedItem().toString());

getTodayUsage(appCompatSpinner_device.getSelectedItem().toString());
            }

            @Override
            public void onNothingSelected(AdapterView<?> parent) {}
        });

        loadSpinner();

        return view;
    }
```

```
private void getTodayUsage(final String deviceID) {
    StringRequest stringRequest = new StringRequest(Request.Method.POST,
API.URL_GETUSAGEVALUES, new Response.Listener<String>() {
        @Override
        public void onResponse(String response) {
            try {
                JSONObject jsonResponseObj = new JSONObject(response);
                if (!jsonResponseObj.getBoolean("error")) {
                    JSONArray jsonUsageValuesArray =
jsonResponseObj.getJSONArray("usageValues");
                    usageHomeList.clear();

                    if (jsonUsageValuesArray.length() != 0) {
                        for (int i = 0; i < jsonUsageValuesArray.length(); i++) {
                            JSONObject jsonUsageValueObj =
jsonUsageValuesArray.getJSONObject(i);
                            usageHomeList.add(new
UsageHome(jsonUsageValueObj.getInt("readingValue"),
jsonUsageValueObj.getString("readingTime")));
                        }
                    }
                    else {
                        Toast.makeText(getContext(), "No today's usage data for "
+ appCompatSpinner_device.getSelectedItem().toString(),
Toast.LENGTH_SHORT).show();
                    }

                    recyclerView.setLayoutManager(new
LinearLayoutManager(getContext(), LinearLayoutManager.VERTICAL, false));
                    UsageHomeAdapter usageHomeAdapter = new
UsageHomeAdapter(getContext(), usageHomeList);
                    recyclerView.setAdapter(usageHomeAdapter);
                }
                else {
                    Toast.makeText(getContext(),
jsonResponseObj.getString("message"), Toast.LENGTH_SHORT).show();
                }
            }
            catch (JSONException e) {
                e.printStackTrace();
            }
        }
    }, new Response.ErrorListener() {
        @Override
        public void onErrorResponse(VolleyError error) {
            Toast.makeText(getContext(), error.getMessage(),
Toast.LENGTH_SHORT).show();
        }
    }) {
        @Override
        protected Map<String, String> getParams() {
            Map<String, String> params = new HashMap<>();
            params.put("deviceID", deviceID);
            params.put("usageMode", "0");
            return params;
        }
    };
    VolleySingleton.getInstance(getContext()).addToRequestQueue(stringRequest);
}
```

```
public void loadSpinner() {
    StringRequest stringRequest = new StringRequest(Request.Method.POST,
API.URL_GET_DEVICE_LIST, new Response.Listener<String>() {
        @Override
        public void onResponse(String response) {
            try {
                JSONObject jsonResponseObj = new JSONObject(response);
                JSONArray jsonDeviceListArray =
jsonResponseObj.getJSONArray("deviceList");
                List<String> deviceList = new ArrayList<>();
                for (int i=0; i<jsonDeviceListArray.length(); i++) {
                    JSONObject jsonDeviceObj =
jsonDeviceListArray.getJSONObject(i);
                    deviceList.add(jsonDeviceObj.getString("deviceID"));
                }
                ArrayAdapter<String> arrayAdapter_deviceList = new
ArrayAdapter<>(getContext(), R.layout.simple_spinner_item, deviceList);

arrayAdapter_deviceList.setDropDownViewResource(R.layout.simple_spinner_dropdown_i
tem);
                appCompatSpinner_device.setAdapter(arrayAdapter_deviceList);
                appCompatSpinner_device.setSelection(0);

                if (arrayAdapter_deviceList.getCount() != 0) {
                    appCompatSpinner_device.setEnabled(true);


loadWaveLoadingView(appCompatSpinner_device.getSelectedItem().toString());

getTodayUsage(appCompatSpinner_device.getSelectedItem().toString());
                }
                else {
                    waveLoadingView_gasLevelPercentage.setProgressValue(0);
                    waveLoadingView_gasLevelPercentage.setCenterTitle("No Data");
                    Toast.makeText(getContext(), "No Data! Please add a device in
IoT device settings", Toast.LENGTH_SHORT).show();
                }
            }
            catch (JSONException e) {
                e.printStackTrace();
            }
        }
    }, new Response.ErrorListener() {
        @Override
        public void onErrorResponse(VolleyError error) {
            Toast.makeText(getContext(), error.getMessage(),
Toast.LENGTH_SHORT).show();
        }
    }) {
        @Override
        protected Map<String, String> getParams() {
            Map<String, String> params = new HashMap<>();
            params.put("userName",
Objects.requireNonNull(userDetails.get(SessionManager.KEY_USERNAME)));
            return params;
        }
    };
    VolleySingleton.getInstance(getContext()).addToRequestQueue(stringRequest);
}
```

```
public void loadWaveLoadingView(final String deviceID) {
        final FirebaseDatabase firebaseDatabase = FirebaseDatabase.getInstance();
        DatabaseReference databaseReference = firebaseDatabase.getReference();

        databaseReference.addValueEventListener(new ValueEventListener() {
            @Override
            public void onDataChange(@NonNull DataSnapshot dataSnapshot) {
                if (dataSnapshot.hasChild(deviceID + "/gas_level")) {
                    int gasLevel = dataSnapshot.child(deviceID +
"/gas_level").getValue(Integer.class);
                    waveLoadingView_gasLevelPercentage.setProgressValue(gasLevel);
                    waveLoadingView_gasLevelPercentage.setCenterTitle(gasLevel +
"%");
                }
                else {
                    waveLoadingView_gasLevelPercentage.setProgressValue(0);
                    waveLoadingView_gasLevelPercentage.setCenterTitle("No Data");
                    Toast.makeText(getContext(), "No Data! Please reboot/start
your device to view data", Toast.LENGTH_SHORT).show();
                }
            }

            @Override
            public void onCancelled(@NonNull DatabaseError databaseError) {
                Log.w(TAG, "Failed to read value.", databaseError.toException());
            }
        });
    }
}
```

UsageFragment class:

```
public class UsageFragment extends Fragment implements
OnChartValueSelectedListener {
    private LineChart lineChart;
    private AppCompatSpinner appCompatSpinner_device, appCompatSpinner_graphMode;
    private RecyclerView recyclerView;
    private HashMap<String, String> userDetails;
    private List<Usage> usageList;
    @Nullable
    @Override
    public View onCreateView(@NonNull LayoutInflater inflater, @Nullable ViewGroup
container, @Nullable Bundle savedInstanceState) {
        View view = inflater.inflate(R.layout.fragment_usage, container, false);
        appCompatSpinner_device = view.findViewById(R.id.AppCompatSpinner_device);
        appCompatSpinner_graphMode =
view.findViewById(R.id.AppCompatSpinner_graphMode);
        lineChart = view.findViewById(R.id.usageLineChart);
        recyclerView = view.findViewById(R.id.RecyclerView_usageHistory);
        usageList = new ArrayList<>();
        SessionManager userSession = new
SessionManager(Objects.requireNonNull(getContext()));
        userDetails = userSession.getUserDetails();
        appCompatSpinner_device.setEnabled(false);
        appCompatSpinner_graphMode.setEnabled(false);
        lineChart.setBackgroundColor(Color.WHITE);
        lineChart.setOnChartValueSelectedListener(this);
        lineChart.setDrawGridBackground(false);
        lineChart.getDescription().setEnabled(false);
        lineChart.setNoDataText("No data available!");
        loadSpinner();
        appCompatSpinner_device.setOnItemSelectedListener(new
AdapterView.OnItemSelectedListener() {
            @Override
            public void onItemSelected(AdapterView<?> parent, View view, int
position, long id) {
                lineChart.clear();

getUsageValues(appCompatSpinner_device.getSelectedItem().toString(),
appCompatSpinner_graphMode.getSelectedItem().toString());
            }
            @Override
            public void onNothingSelected(AdapterView<?> parent) {}
        });
        appCompatSpinner_graphMode.setOnItemSelectedListener(new
AdapterView.OnItemSelectedListener() {
            @Override
            public void onItemSelected(AdapterView<?> parent, View view, int
position, long id) {
                lineChart.clear();
              getUsageValues(appCompatSpinner_device.getSelectedItem().toString(),
appCompatSpinner_graphMode.getSelectedItem().toString());
            }
            @Override
            public void onNothingSelected(AdapterView<?> parent) {}
        });
        return view;
    }
```

```
private void getUsageValues(final String deviceID, final String graphMode) {
    final ArrayList<Entry> usageValues = new ArrayList<>();
    StringRequest stringRequest = new StringRequest(Request.Method.POST,
API.URL_GETUSAGEVALUES, new Response.Listener<String>() {
        @Override
        public void onResponse(String response) {
            try {
                JSONObject jsonResponseObj = new JSONObject(response);
                if (!jsonResponseObj.getBoolean("error")) {
                    JSONArray jsonUsageValuesArray =
jsonResponseObj.getJSONArray("usageValues");
                    usageList.clear();

                    if (jsonUsageValuesArray.length() != 0) {
                        for (int i = 0; i < jsonUsageValuesArray.length(); i++) {
                            JSONObject jsonUsageValueObj =
jsonUsageValuesArray.getJSONObject(i);
                            usageValues.add(new Entry(i,
jsonUsageValueObj.getInt("readingValue")));
                            usageList.add(new
Usage(jsonUsageValueObj.getInt("readingValue"),
jsonUsageValueObj.getString("readingTime")));
                        }

                        LineDataSet lineDataSet = new LineDataSet(usageValues,
graphMode);
                        ArrayList<ILineDataSet> dataSets = new ArrayList<>();
                        dataSets.add(lineDataSet);

                        LineData data = new LineData(dataSets);
                        lineChart.setData(data);
                        lineChart.invalidate();
                    }
                    else {
                        Toast.makeText(getContext(), "No Data for " +
graphMode.toLowerCase() + "!", Toast.LENGTH_SHORT).show();
                    }

                    recyclerView.setLayoutManager(new
LinearLayoutManager(getContext(), LinearLayoutManager.VERTICAL, false));
                    UsageAdapter usageAdapter = new UsageAdapter(getContext(),
usageList);
                    recyclerView.setAdapter(usageAdapter);
                }
                else {
                    Toast.makeText(getContext(),
jsonResponseObj.getString("message"), Toast.LENGTH_SHORT).show();
                }
            }
            catch (JSONException e) {
                e.printStackTrace();
            }
        }
    }, new Response.ErrorListener() {
        @Override
        public void onErrorResponse(VolleyError error) {
            Toast.makeText(getContext(), error.getMessage(),
Toast.LENGTH_SHORT).show();
```

```
}
    }) {
        @Override
        protected Map<String, String> getParams() {
            String usageMode = "0";
            switch (graphMode) {
                case "Today":
                    usageMode = "0";
                    break;
                case "This week":
                    usageMode = "1";
                    break;
                case "This month":
                    usageMode = "2";
                    break;
                case "This year":
                    usageMode = "3";
                    break;
            }

            Map<String, String> params = new HashMap<>();
            params.put("deviceID", deviceID);
            params.put("usageMode", usageMode);
            return params;
        }
    };
    VolleySingleton.getInstance(getContext()).addToRequestQueue(stringRequest);
}
public void loadSpinner() {
        StringRequest stringRequest = new StringRequest(Request.Method.POST,
API.URL_GET_DEVICE_LIST, new Response.Listener<String>() {
            @Override
            public void onResponse(String response) {
                try {
                    JSONObject jsonResponseObj = new JSONObject(response);
                    JSONArray jsonDeviceListArray = jsonResponseObj.getJSO-
NArray("deviceList");

                    List<String> deviceList = new ArrayList<>();
                    for (int i=0; i<jsonDeviceListArray.length(); i++) {
                        JSONObject jsonDeviceObj = jsonDeviceListArray.getJSONOb-
ject(i);
                        deviceList.add(jsonDeviceObj.getString("deviceID"));
                    }

                    ArrayAdapter<String> arrayAdapter_deviceList = new ArrayA-
dapter<>(getContext(), R.layout.simple_spinner_item, deviceList);
                    arrayAdapter_deviceList.setDropDownViewResource(R.layout.sim-
ple_spinner_dropdown_item);
                    appCompatSpinner_device.setAdapter(arrayAdapter_deviceList);
                    appCompatSpinner_device.setSelection(0);

                    if (arrayAdapter_deviceList.getCount() != 0) {
                        appCompatSpinner_device.setEnabled(true);

                        String[] graphMode = new String[]{"Today", "This week",
"This month", "This year"};
```

```
ArrayAdapter<String> arrayAdapter_graphMode = new ArrayAdapter<>(getContext(),
R.layout.simple_spinner_item, graphMode);

arrayAdapter_graphMode.setDropDownViewResource(R.layout.simple_spinner_dropdown_it
em);

appCompatSpinner_graphMode.setAdapter(arrayAdapter_graphMode);
                        appCompatSpinner_graphMode.setSelection(0);

                        appCompatSpinner_graphMode.setEnabled(true);


getUsageValues(appCompatSpinner_device.getSelectedItem().toString(),
appCompatSpinner_graphMode.getSelectedItem().toString());
                    }
                    else {
                        Toast.makeText(getContext(), "No Data! Please add a device
in IoT device settings", Toast.LENGTH_SHORT).show();
                    }
                }
                catch (JSONException e) {
                    e.printStackTrace();
                }
            }
        }, new Response.ErrorListener() {
            @Override
            public void onErrorResponse(VolleyError error) {
                Toast.makeText(getContext(), error.getMessage(),
Toast.LENGTH_SHORT).show();
            }
        }) {
            @Override
            protected Map<String, String> getParams() {
                Map<String, String> params = new HashMap<>();
                params.put("userName",
Objects.requireNonNull(userDetails.get(SessionManager.KEY_USERNAME)));
                return params;
            }
        };

VolleySingleton.getInstance(getContext()).addToRequestQueue(stringRequest);
    }

    @Override
    public void onValueSelected(Entry e, Highlight h) {}

    @Override
    public void onNothingSelected() {}
}
```

ProfileFragment class:

```
public class ProfileFragment extends Fragment {

    TextView textView_lpgDevices;

    @Nullable
    @Override
    public View onCreateView(@NonNull LayoutInflater inflater, @Nullable ViewGroup
container, @Nullable Bundle savedInstanceState) {
        View view = inflater.inflate(R.layout.fragment_profile, container, false);

        final SessionManager userSession = new SessionManager(getContext());
        final HashMap<String, String> userDetails = userSession.getUserDetails();

        TextView textView_userName = view.findViewById(R.id.TextView_userName);
        textView_userName.setText(userDetails.get(SessionManager.KEY_USERNAME));

        textView_lpgDevices = view.findViewById(R.id.TextView_lpgDevices);

        view.findViewById(R.id.LinearLayout_profile).setOnClickListener(new
View.OnClickListener() {
            @Override
            public void onClick(View v) {
                startActivity(new Intent(getContext(),
SettingsProfileActivity.class));
            }
        });

        view.findViewById(R.id.LinearLayout_iotDevice).setOnClickListener(new
View.OnClickListener() {
            @Override
            public void onClick(View v) {
                startActivity(new Intent(getContext(),
SettingsIoTDeviceActivity.class));
            }
        });

        view.findViewById(R.id.LinearLayout_app).setOnClickListener(new
View.OnClickListener() {
            @Override
            public void onClick(View v) {
                startActivity(new Intent(getContext(),
SettingsAppActivity.class));
            }
        });

        view.findViewById(R.id.LinearLayout_aboutApp).setOnClickListener(new
View.OnClickListener() {
            @Override
            public void onClick(View v) {
                startActivity(new Intent(getContext(),
SettingsAboutAppActivity.class));
            }
        });
```

```
        view.findViewById(R.id.LinearLayout_signOut).setOnClickListener(new
View.OnClickListener() {
            @Override
            public void onClick(View view) {
                getActivity().finish();
                userSession.signOutUser();
            }
        });

        getNoOfDevices(userDetails.get(SessionManager.KEY_USERNAME));

        return view;
    }

    public void getNoOfDevices(final String userName) {
        StringRequest stringRequest = new StringRequest(Request.Method.POST,
API.URL_GET_DEVICE_LIST, new Response.Listener<String>() {
            @Override
            public void onResponse(String response) {
                try {
                    JSONObject jsonResponseObj = new JSONObject(response);
                    JSONArray jsonDeviceListArray =
jsonResponseObj.getJSONArray("deviceList");
                    int noOfDevices = jsonDeviceListArray.length();

                    textView_lpgDevices.setText("LPG Devices: " + noOfDevices);
                }
                catch (JSONException e) {
                    e.printStackTrace();
                }
            }
        }, new Response.ErrorListener() {
            @Override
            public void onErrorResponse(VolleyError error) {
                Toast.makeText(getContext(), error.getMessage(),
Toast.LENGTH_SHORT).show();
            }
        }) {
            @Override
            protected Map<String, String> getParams() {
                Map<String, String> params = new HashMap<>();
                params.put("userName", userName);
                return params;
            }
        };

VolleySingleton.getInstance(getContext()).addToRequestQueue(stringRequest);
    }
}
```